



物联网虚拟仿真实验

Python 部分

(V1.2)

华清远见教育集团 • 研发中心



170-9108-5953



2306275952/2668462267



<http://www.fsdev.com.cn>



support@farsight.com.cn



目 录

目 录.....	- 1 -
第 1 章 MQTT 简介	- 1 -
1.1 MQTT 基础知识	- 1 -
1.2 本系统 MQTT 架构	- 5 -
第 2 章 Python 环境搭建	- 8 -
第 3 章 Pyinstaller 工具安装和使用.....	- 10 -
3.1 安装方法.....	- 10 -
3.2 生成可执行程序.....	- 11 -
第 4 章 基于 Python 的各种传感器数据采集实验	- 13 -
4.1 火焰传感器数据采集.....	- 13 -
4.1.1 实验目的.....	- 13 -
4.1.2 实验原理.....	- 13 -
4.1.3 实验步骤.....	- 13 -
4.2 空气温湿度传感系统数据采集.....	- 21 -
4.2.1 实验目的.....	- 21 -
4.2.2 实验原理.....	- 22 -
4.2.3 实验步骤.....	- 22 -
4.3 CO2 传感系统数据采集	- 24 -
4.3.1 实验目的.....	- 24 -
4.3.2 实验原理.....	- 25 -
4.3.3 实验步骤.....	- 25 -
4.4 土壤温湿度传感系统数据采集.....	- 27 -
4.4.1 实验目的.....	- 27 -
4.4.2 实验原理.....	- 27 -
4.4.3 实验步骤.....	- 27 -



4.5 PM2.5 传感系统数据采集.....	- 29 -
4.5.1 实验目的.....	- 29 -
4.5.2 实验原理.....	- 30 -
4.5.3 实验步骤.....	- 30 -
4.6 人体红外传感系统数据采集.....	- 32 -
4.6.1 实验目的.....	- 32 -
4.6.2 实验原理.....	- 32 -
4.6.3 实验步骤.....	- 32 -
4.7 烟雾传感系统数据采集.....	- 34 -
4.7.1 实验目的.....	- 34 -
4.7.2 实验原理.....	- 35 -
4.7.3 实验步骤.....	- 35 -
4.8 可燃气体传感系统数据采集.....	- 37 -
4.8.1 实验目的.....	- 37 -
4.8.2 实验原理.....	- 37 -
4.8.3 实验步骤.....	- 37 -
4.9 雨雪传感系统数据采集.....	- 39 -
4.9.1 实验目的.....	- 39 -
4.9.2 实验原理.....	- 40 -
4.9.3 实验步骤.....	- 40 -
4.10 光照传感系统数据采集.....	- 42 -
4.10.1 实验目的.....	- 42 -
4.10.2 实验原理.....	- 42 -
4.10.3 实验步骤.....	- 42 -
第 5 章 基于 Python 的各种执行器控制实验	- 45 -
5.1 声光报警器执行系统控制实验.....	- 45 -



5.1.1 实验目的.....	- 45 -
5.1.2 实验原理.....	- 45 -
5.1.3 实验步骤.....	- 45 -
5.2 喷淋执行系统控制实验.....	- 48 -
5.2.1 实验目的.....	- 48 -
5.2.2 实验原理.....	- 49 -
5.2.3 实验步骤.....	- 49 -
5.3 电灯执行系统控制实验.....	- 52 -
5.3.1 实验目的.....	- 52 -
5.3.2 实验原理.....	- 52 -
5.3.3 实验步骤.....	- 52 -
5.4 遮阳板执行系统控制实验.....	- 55 -
5.4.1 实验目的.....	- 55 -
5.4.2 实验原理.....	- 55 -
5.4.3 实验步骤.....	- 56 -
5.5 微型电控锁执行系统控制实验.....	- 59 -
5.5.1 实验目的.....	- 59 -
5.5.2 实验原理.....	- 59 -
5.5.3 实验步骤.....	- 59 -
5.6 风扇执行系统控制实验.....	- 62 -
5.6.1 实验目的.....	- 62 -
5.6.2 实验原理.....	- 62 -
5.6.3 实验步骤.....	- 63 -
第 6 章 基于 Python 的各种人工智能实验	- 65 -
6.1 人脸识别.....	- 65 -
6.1.1 实验目的.....	- 65 -



6.1.2 实验原理.....	- 65 -
6.1.3 实验步骤.....	- 66 -
I 申请步骤.....	- 66 -
II 调用 API 实现识别	- 69 -
III 人脸检测方式.....	- 70 -
IV 使用 URL 方式进行人脸检测	- 73 -
V 使用 BASE64 方式进行人脸检测	- 75 -
VI 添加人脸库.....	- 77 -
VII 开始识别.....	- 78 -
VIII 改进返回值.....	- 81 -
6.2 语音识别.....	- 84 -
6.2.1 实验目的.....	- 84 -
6.2.2 实验原理.....	- 84 -
6.2.3 实验步骤.....	- 84 -
I 申请步骤.....	- 85 -
II 调用 API 实现识别	- 87 -
III 语音识别.....	- 88 -
6.3 通用物体和场景识别.....	- 90 -
6.3.1 实验目的.....	- 90 -
6.3.2 实验原理.....	- 90 -
I 接口描述.....	- 90 -
II 请求说明.....	- 90 -
6.3.3 实验步骤.....	- 91 -
6.4 蔬菜识别.....	- 93 -
6.4.1 实验目的.....	- 93 -
6.4.2 实验原理.....	- 93 -
I 接口描述.....	- 93 -
II 请求说明.....	- 93 -



6.4.3 实验步骤.....	- 93 -
6.5 动物识别.....	- 94 -
6.5.1 实验目的.....	- 94 -
6.5.2 实验原理.....	- 94 -
I 接口描述.....	- 94 -
II 请求说明.....	- 94 -
6.5.3 实验步骤.....	- 95 -
6.6 植物识别.....	- 96 -
6.6.1 实验目的.....	- 96 -
6.6.2 实验原理.....	- 96 -
I 接口描述.....	- 96 -
II 请求说明.....	- 96 -
6.6.3 实验步骤.....	- 96 -
6.7 果蔬识别.....	- 97 -
6.7.1 实验目的.....	- 97 -
6.7.2 实验原理.....	- 97 -
I 接口描述.....	- 97 -
II 请求说明.....	- 98 -
6.7.3 实验步骤.....	- 98 -
6.8 图像主体检测.....	- 99 -
6.8.1 实验目的.....	- 99 -
6.8.2 实验原理.....	- 99 -
I 接口描述.....	- 99 -
II 请求示例.....	- 99 -
6.8.3 实验步骤.....	- 99 -
6.9 地标识别.....	- 100 -
6.9.1 实验目的.....	- 100 -



6.9.2 实验原理.....	- 100 -
I 接口描述.....	- 100 -
II 请求示例.....	- 100 -
6.9.3 实验步骤.....	- 101 -
第 7 章 基于 Python 的智能系统实验	- 102 -
7.1 智能火焰报警系统.....	- 102 -
7.1.1 实验目的.....	- 102 -
7.1.2 实验原理.....	- 102 -
7.1.3 实验步骤.....	- 102 -
7.2 智能烟雾报警系统.....	- 111 -
7.2.1 实验目的.....	- 111 -
7.2.2 实验原理.....	- 111 -
7.2.3 实验步骤.....	- 112 -
7.3 智能光照控制系统.....	- 116 -
7.3.1 实验目的.....	- 116 -
7.3.2 实验原理.....	- 117 -
7.3.3 实验步骤.....	- 117 -
7.4 智能风扇控制系统.....	- 122 -
7.4.1 实验目的.....	- 122 -
7.4.2 实验原理.....	- 122 -
7.4.3 实验步骤.....	- 122 -
7.5 智能土壤灌溉系统.....	- 125 -
7.5.1 实验目的.....	- 125 -
7.5.2 实验原理.....	- 126 -
7.5.3 实验步骤.....	- 126 -
7.6 智能灯光控制系统.....	- 129 -



7.6.1 实验目的.....	- 129 -
7.6.2 实验原理.....	- 129 -
7.6.3 实验步骤.....	- 129 -
7.7 智能窗帘控制系统.....	- 132 -
7.7.1 实验目的.....	- 132 -
7.7.2 实验原理.....	- 132 -
7.7.3 实验步骤.....	- 132 -
7.8 智能燃气报警系统.....	- 135 -
7.8.1 实验目的.....	- 135 -
7.8.2 实验原理.....	- 135 -
7.8.3 实验步骤.....	- 136 -



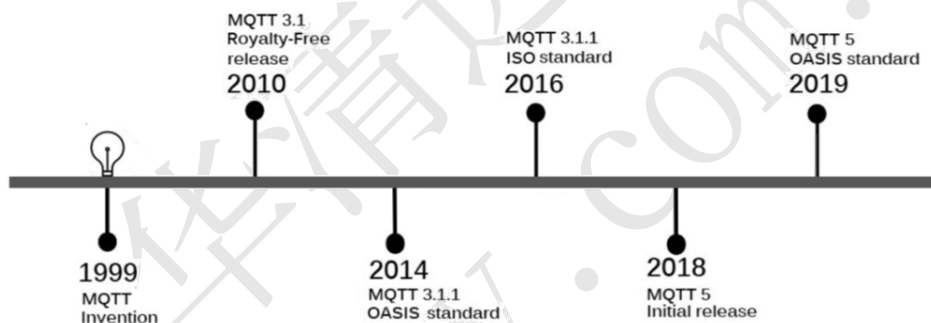
第 1 章 MQTT 简介

1.1 MQTT 基础知识

MQTT 协议由 Andy Stanford-Clark (IBM) 和 Arlen Nipper (Arcom, 现为 Cirrus Link) 于 1999 年发明。他们需要一种通过卫星连接石油管道的协议, 以最大限度地减少电池损耗和带宽。所以他们为这个协议规定了几种要求:

- 这个协议必须易于实现;
- 这个协议中的数据必须易于传输, 消耗成本小;
- 这个协议必须提供服务质量管理;
- 这个协议必须支持连续的会话控制
- 假设数据不可知, 不强求传输数据的类型与格式, 保持灵活性。

该规定也适合目前的物联网信息传输场景。因此, 经过多年的发展, MQTT 成为了物联网消息传输的主要技术之一。



MQTT 所具有的适用于物联网特定需求的特点和功能, 使其成为物联网领域最佳的协议之一。它的主要特点包括:

轻量级: 物联网设备通常在处理能力、内存和能耗方面受到限制。MQTT 开销低、报文小的特点使其非常适合这些设备, 因为它消耗更少的资源, 即使在有限的能力下也能实现高效的通信。

可靠: 物联网网络常常面临高延迟或连接不稳定的情况。MQTT 支持多种 QoS 等级、会话感知和持久连接, 即使在困难的条件下也能保证消息的可靠传递, 使其非常适合物联网应用。

安全通信: 安全对于物联网网络至关重要, 因为其经常涉及敏感数据的传输。为确保数据在传输过程中的机密性, MQTT 提供传输层安全 (TLS) 和安全套接层 (SSL) 加密功能。此外, MQTT 还通过用户名/密码凭证或客户端证书提供身份验证和授权机制, 以保护网络及其资源的访问。



双向通信：MQTT 的发布-订阅模式为设备之间提供了无缝的双向通信方式。客户端既可以向主题发布消息，也可以订阅接收特定主题上的消息，从而实现了物联网生态系统中的高效数据交换，而无需直接将设备耦合在一起。这种模式也简化了新设备的集成，同时保证了系统易于扩展。

连续、有状态的会话：MQTT 提供了客户端与 Broker 之间保持有状态会话的能力，这使得系统即使在断开连接后也能记住订阅和未传递的消息。此外，客户端还可以在建立连接时指定一个保活间隔，这会促使 Broker 定期检查连接状态。如果连接中断，Broker 会储存未传递的消息（根据 QoS 级别确定），并在客户端重新连接时尝试传递它们。这个特性保证了通信的可靠性，降低了因间断性连接而导致数据丢失的风险。

大规模物联网设备支持：物联网系统往往涉及大量设备，需要一种能够处理大规模部署的协议。MQTT 的轻量级特性、低带宽消耗和对资源的高效利用使其成为大规模物联网应用的理想选择。通过采用发布-订阅模式，MQTT 实现了发送者和接收者的解耦，从而有效地减少了网络流量和资源使用。此外，协议对不同 QoS 等级的支持使得消息传递可以根据需求进行定制，确保在各种场景下获得最佳的性能表现。

语言支持：物联网系统包含使用各种编程语言开发的设备和应用。MQTT 具有广泛的语言支持，使其能够轻松与多个平台和技术进行集成，从而实现了物联网生态系统中的无缝通信和互操作性。您可以阅读我们的 MQTT 客户端编程系列文章，学习如何在 PHP、Node.js、Python、Golang、Node.js 等编程语言中使用 MQTT。

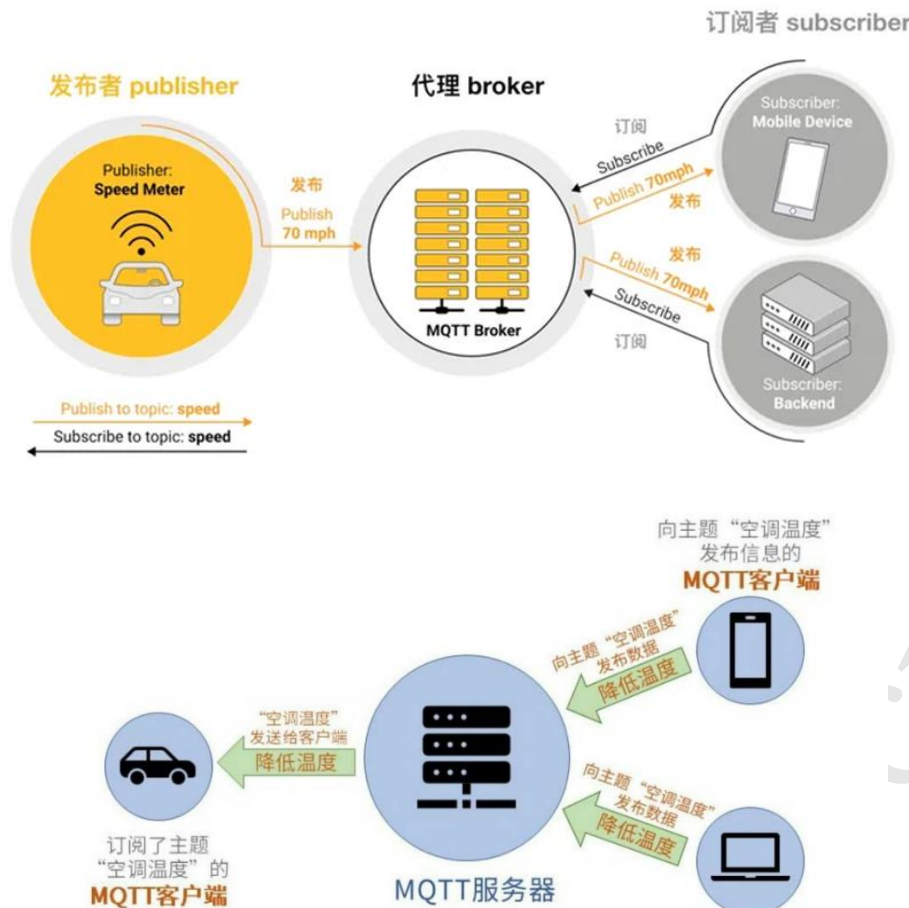
要了解 MQTT 的工作原理，首先需要掌握以下几个概念：MQTT 客户端、MQTT Broker、发布-订阅模式、主题、QoS。

MQTT 客户端：任何运行 MQTT 客户端库的应用或设备都是 MQTT 客户端。例如，使用 MQTT 的即时通讯应用是客户端，使用 MQTT 上报数据的各种传感器是客户端，各种 MQTT 测试工具也是客户端。

MQTT Broker：MQTT Broker 是负责处理客户端请求的关键组件，包括建立连接、断开连接、订阅和取消订阅等操作，同时还负责消息的转发。一个高效强大的 MQTT Broker 能够轻松应对海量连接和百万级消息吞吐量，从而帮助物联网服务提供商专注于业务发展，快速构建可靠的 MQTT 应用。

发布-订阅模式：发布-订阅模式与客户端-服务器模式的不同之处在于，它将发送消息的客户端（发布者）和接收消息的客户端（订阅者）进行了解耦。发布者和订阅者之间无需建立直接连接，而是通过 MQTT Broker 来负责消息的路由和分发。

下图展示了 MQTT 发布/订阅过程。



pub/sub 最重要的方面是 publisher 与 subscriber 的解耦，这种耦合度有下面三个维度：

空间解耦：publisher 与 subscriber 并不知道对方的存在，例如不会有 IP 地址和端口的交互，也更不会有消息的交互。

时间解耦：publisher 与 subscriber 并不一定需要同时运行。

同步解耦：两个组件的操作都不会在发布或者接收过程中产生中断。

这种特点会带来如下的好处：

发布者与订阅者只需要知道同一个消息代理即可。

发布者和订阅者不需要直接交互。

发布者和订阅者不需要同时在线。

在 MQTT 中，话题名是由一串 UTF-8 编码的字符串组成的，例如：

kingdom/phylum/class/order/family/genus/species（界门纲目科属种）

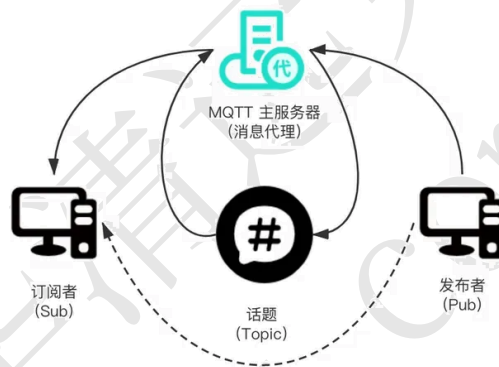
这同时也是一个多级主题的示例，在 MQTT 中，允许存在多级话题，可以向任何一级发布消息，每一个话题级别由/分割。也允许用通配符进行灵活的话题订阅和发布。



通配符分为两种，分别是单层通配符(+)和多层通配符(#):

单层通配符(+): 单层通配符可以替代主题名中的任意一个层级用来指代本层级的所有话题层级名，单层通配符前后必须是正斜杠/(单层通配符处于主题名末尾时除外)。如使用 kingdom/phylum/+/order 作为主题名订阅时，匹配结果如下

Topic	Status
kingdom/phylum/class/order	Yes
kingdom/phylum/aaaaa/order	Yes
kingdom/phylum/aaaaa/bbbbbb	No
kingdom/bbbbbbb/aaaaa/order	No
kingdom/phylum/aaaaa/bbbbbb/order	No



在 pub/sub 的架构模式中，Broker 扮演着至关重要的作用，其中非常重要的一点就是 Broker 能够对消息进行过滤，使每个订阅者只接收自己感兴趣的消息。Broker 有几个可以过滤的选项：

基于主题的过滤：MQTT 是基于 subject 的消息过滤的，每条消息都会有一个 topic，接收客户端会向 Broker 订阅感兴趣的 topic，订阅后，Broker 就会确保客户端收到发布到 topic 中的消息。

基于内容的过滤：在基于内容的过滤中，Broker 会根据特定的内容过滤消息，接收客户端会经过过滤他们感兴趣的内容。这种方法的一个显著的缺点就是必须事先知道消息的内容，不能加密或者轻易修改。

基于类型的过滤：在使用面向对象的语言时，基于消息（事件）的类型过滤是一种比较常见的过滤方式。

MQTT 提供了三种服务质量（QoS），在不同网络环境下保证消息的可靠性。

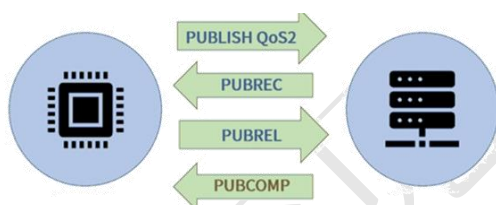


QoS 0 : 消息最多发 1 次, 占用的网络资源最低。发送端一旦发送完消息后, 就完成任务了。发送端不会检查发出的消息能否被正确接收到。

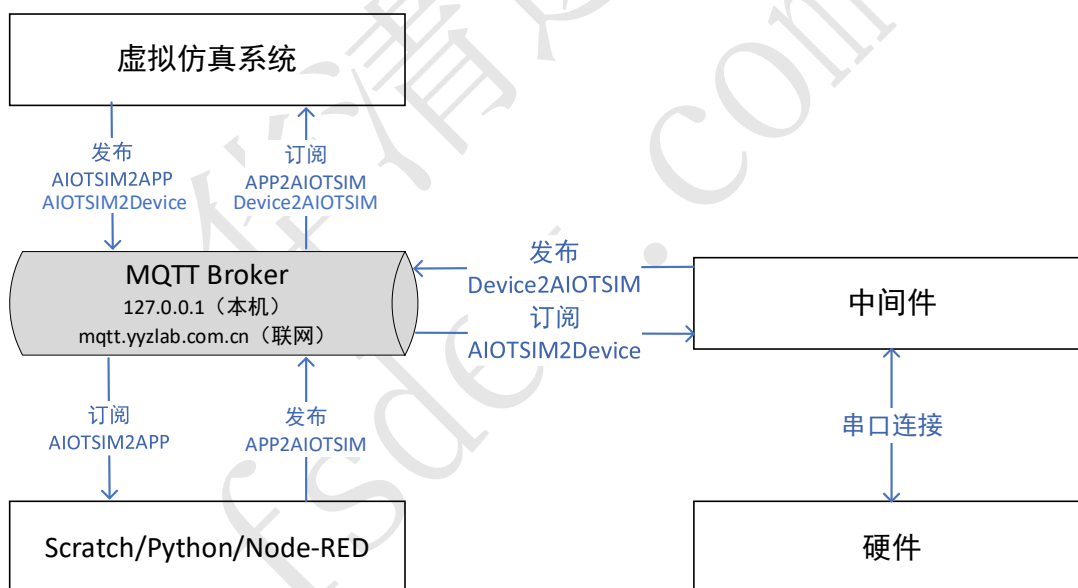
QoS 1 : 消息最少发 1 次, 但是有可能出现接收端反复接收同一消息的情况。



QoS 2 : 消息保证收 1 次。

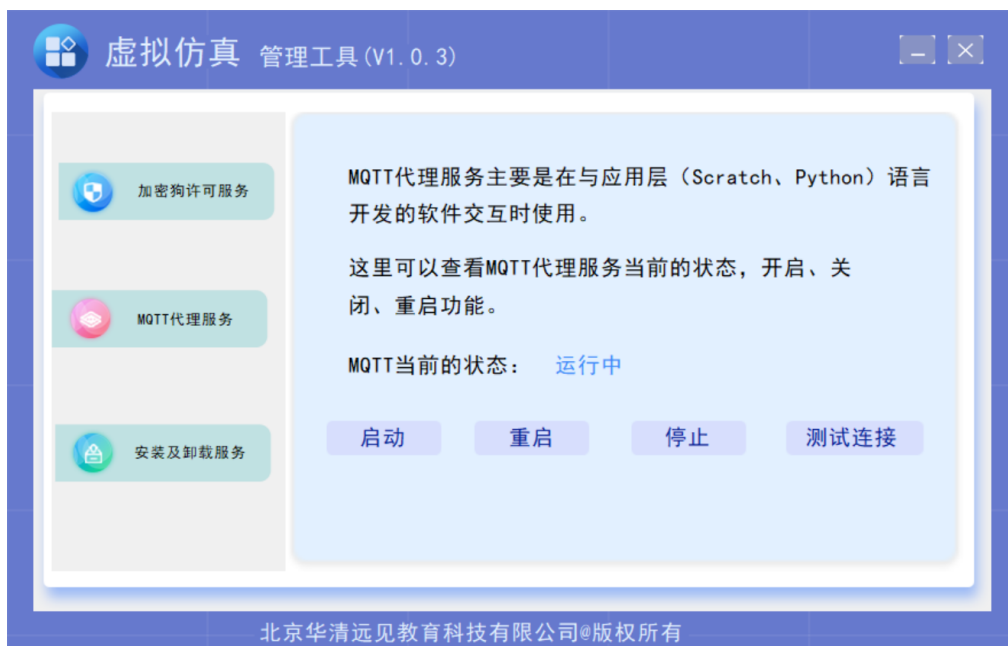


1.2 本系统 MQTT 架构



本系统包括: 虚拟仿真系统、MQTT Broker、Scratch、Node-RED、Python、中间件、实际硬件等。

在安装单机版虚拟仿真系统时, 已自动安装 MQTT Broker 服务如下图所示。网页版未在本机安装, 也不能将本机作为 MQTT Broker 服务器, 只能将 mqtt.yyzlab.com.cn 作为服务器。



因此，本机可作为 MQTT Broker 使用，从而存储转发其他软件订阅或发送的对应话题内的消息。

虚拟仿真系统中，网关向 Broker 订阅 APP2AIOTSIM 话题，从而接收来自 Scratch 或 Python 发布的消息，网关向 Broker 发布 AIOTSIM2APP 话题，从而让 Scratch、Node-RED 或 Python 接收相关的话题。注意，当使用 127.0.0.1 本机地址作为 Broker 时，使用默认话题即可，当使用 mqtt.yyzlab.com.cn 作为 Broker 服务器时，话题和 ClientID 必须能够被唯一识别，从而避免与他人使用的话题和 ID 冲突。例如，可以通过点击“生成”按钮将话题和 ClientID 设置为下面的形式。在使用 Scratch、Node-RED、Python 时，ClientID 也不应与此相同，必须独立生成。

MQTT:

IP地址: wss:// mqtt.yyzlab.com.cn

Path: mqtt

端口: 9002 MQTT版本: 3.1.1

ClientID: 33574cfa21e85f0e7a505ccabd84575 生成

用户名: 请输入用户名

密码: 请输入密码

连接

与应用层、云端交互的Topic:

订阅: 1698225072428/APP2AIOTSIM 复制

发布: 1698225072428/AIOTSIM2APP 复制

生成topic 订阅

与硬件交互的Topic:

订阅: 1698218726605/Device2AIOTSIM 复制

发布: 1698218726605/AIOTSIM2Device 复制

生成topic 订阅



本系统可以通过中间件连接真实的硬件设备。中间件界面如下。

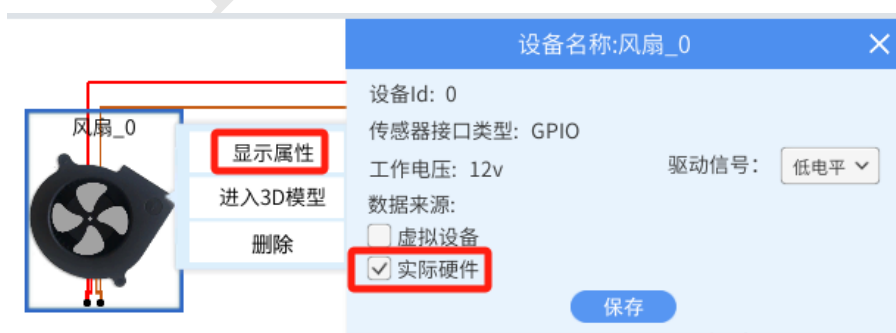


当需要接收硬件传感器发来的数据时，首先需要将虚拟仿真系统中的器件设置为实际硬件件。



然后中间件会将通过串口接收到的数据转发给 Device2AIOTSIM 这个话题，供虚拟仿真系统接收。

当需要向硬件执行器发送命令时，首先需要将虚拟仿真系统中的器件设置为实际硬件，然后当利用 Scratch、Node-RED 或 Python 通过 APP2AIOTSIM 话题向该器件发送控制指令时，该指令会自动转发到 AIOTSIM2Device 上，供中间件接收，并通过串口转发给真实硬件。





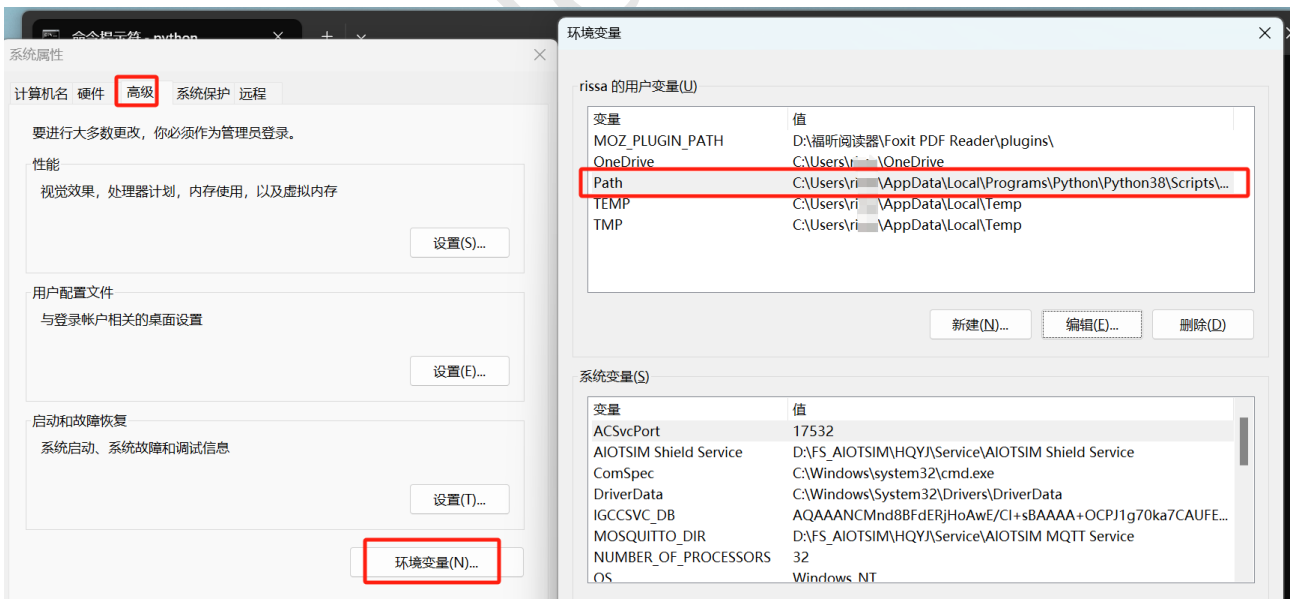
第2章 Python 环境搭建

通过 CMD 命令提示符和添加删除程序我们可以检查系统内有无安装 Python。在命令提示符中输入 Python，若正常响应，则说明已安装 Python。

```
命令提示符 - python
Microsoft Windows [版本 10.0.22621.2861]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\r\>python
Python 3.8.10 (tags/v3.8.10:3d8993a, May 3 2021, 11:48:03) [MSC v.1928 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

在高级系统设置-系统属性-高级-环境变量中可以看到包含了 Python 所在的路径。

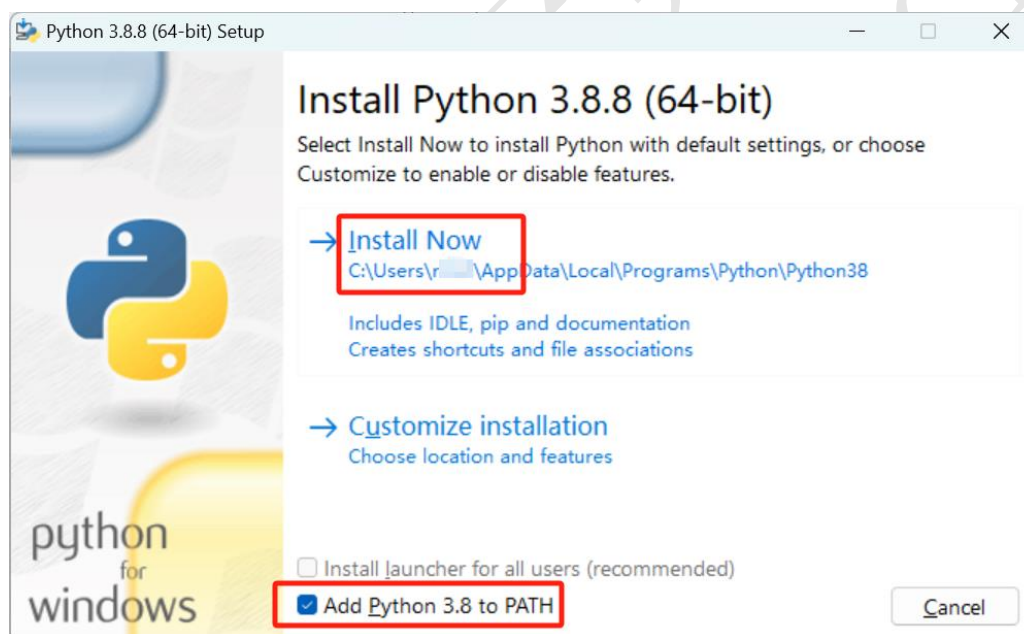




如果提示找不到 Python，则需要打开 Python 安装程序进行安装。



注意勾选 Add Python 3.8 to PATH。然后点击 Install Now。



为了便于使用 Python 进行编译，除了使用命令提示行使用 `python *.py` 命令编译 Python 文件外，还可以使用 VS Code、Geany 等代码编辑软件进行编译。此时需要修改这类软件的编译选项。具体方法请参考网络。



第3章 Pyinstaller 工具安装和使用

为了将 Python 程序打包成在其他电脑上也可运行的*.exe 程序，我们需要使用 pyinstaller 工具。

3.1 安装方法

Python 默认并不包含 PyInstaller 模块，因此需要自行安装 PyInstaller 模块。

安装 PyInstaller 模块与安装其他 Python 模块一样，使用 pip 命令安装即可。在命令行输入如下命令：

```
pip install pyinstaller
```

强烈建议使用 pip 在线安装的方式来安装 PyInstaller 模块，不要使用离线包的方式来安装，因为 PyInstaller 模块还依赖其他模块，pip 在安装 PyInstaller 模块时会先安装它的依赖模块。

运行上面命令，应该看到如下输出结果：

强烈建议使用 pip 在线安装的方式来安装 PyInstaller 模块，不要使用离线包的方式来安装，因为 PyInstaller 模块还依赖其他模块，pip 在安装 PyInstaller 模块时会先安装它的依赖模块。

```
C:\WINDOWS\system32\cmd.exe - pip install pyinstaller
Microsoft Windows [版本 10.0.18362.657]
(c) 2019 Microsoft Corporation. 保留所有权利。

C:\Users\ASUS>pip list
Package            Version
-----
baidu-aip          2.2.13.0
certifi            2019.3.9
chardet            3.0.4
cyclor             0.10.0
idna               2.8
kiwisolver         1.1.0
matplotlib         3.1.0
numpy             1.16.4
pip               19.1.1
pyparsing          2.4.0
python-dateutil    2.8.0
requests          2.21.0
setuptools        41.0.1
six               1.12.0
urllib3           1.24.3
web.py            0.40.dev1

C:\Users\ASUS>pip install pyinstaller
Collecting pyinstaller
  WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ReadTimeoutError("HTTPConnectionPool(host='files.pythonhosted.org', port=443): Read timed out. (read timeout=15)")': /packages/3c/c9/c3f9bc64eb11eee6a824686deba6129884c8cbdf70e750661773b9865ee0/PyInstaller-3.6.tar.gz
  WARNING: Retrying (Retry(total=3, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ReadTimeoutError("HTTPConnectionPool(host='files.pythonhosted.org', port=443): Read timed out. (read timeout=15)")': /packages/3c/c9/c3f9bc64eb11eee6a824686deba6129884c8cbdf70e750661773b9865ee0/PyInstaller-3.6.tar.gz
  Downloading https://files.pythonhosted.org/packages/3c/c9/c3f9bc64eb11eee6a824686deba6129884c8cbdf70e750661773b9865ee0/PyInstaller-3.6.tar.gz (3.5MB)
    |#####| 2.5MB 12kB/s eta 0:01:29
```

运行上面命令，应该看到如下输出结果：

Successfully installed pyinstaller-x.x.x 其中的 x.x.x 代表 PyInstaller 的版本。



录，并在该目录下看到有一个 `app.exe` 文件，这就是使用 `PyInstaller` 工具生成的 EXE 程序。

在命令行窗口中进入 `dist` 目录下，在该目录执行 `app.exe`，将会看到该程序生成如下输出结果：

程序开始执行

123，您好！

由于该程序没有图形用户界面，因此如果读者试图通过双击来运行该程序，则只能看到程序窗口一闪就消失了，这样将无法看到该程序的输出结果。

在上面命令中使用了 `-F` 选项，该选项指定生成单独的 EXE 文件，因此，在 `dist` 目录下生成了一个单独的大约为 6MB 的 `app.exe` 文件（在 Mac OS X 平台上生成的文件就叫 `app`，没有后缀）；与 `-F` 选项对应的是 `-D` 选项（默认选项），该选项指定生成一个目录（包含多个文件）来作为程序。

下面先将 `PyInstaller` 工具在 `app` 目录下生成的 `build`、`dist` 目录删除，并将 `app.spec` 文件也删除，然后使用如下命令来生成 EXE 文件。

```
pyinstaller -D app.py
```

执行上面命令，将看到详细的生成过程。当生成完成后，将会在 `app` 目录下看到多了一个 `dist` 目录，并在该目录下看到有一个 `app` 子目录，在该子目录下包含了大量 `.dll` 文件和 `.pyz` 文件，它们都是 `app.exe` 程序的支撑文件。在命令行窗口中运行该 `app.exe` 程序，同样可以看到与前一个 `app.exe` 程序相同的输出结果。

`PyInstaller` 不仅支持 `-F`、`-D` 选项，而且也支持如表 1 所示的常用选项。

<code>-h, --help</code>	查看该模块的帮助信息
<code>-F, --onefile</code>	产生单个的可执行文件
<code>-D, --onedir</code>	产生一个目录（包含多个文件）作为可执行程序
<code>-a, --ascii</code>	不包含 Unicode 字符集支持
<code>-d, --debug</code>	产生 debug 版本的可执行文件
<code>-w, --windowed, --noconsole</code>	指定程序运行时不显示命令行窗口（仅对 Windows 有效）
<code>-c, --nowindowed, --console</code>	指定使用命令行窗口运行程序（仅对 Windows 有效）
<code>-o DIR, --out=DIR</code>	指定 <code>spec</code> 文件的生成目录。如果没有指定，则默认使用当前目录来生成 <code>spec</code> 文件
<code>-p DIR, --path=DIR</code>	设置 Python 导入模块的路径（和设置 <code>PYTHONPATH</code> 环境变量的作用相似）。也可使用路径分隔符（Windows 使用分号，Linux 使用冒号）来分隔多个路径
<code>-n NAME, --name=NAME</code>	指定项目（产生的 <code>spec</code> ）名字。如果省略该选项，那么第一个脚本的主文件名将作为 <code>spec</code> 的名字



第 4 章 基于 Python 的各种传感器数据采集实验

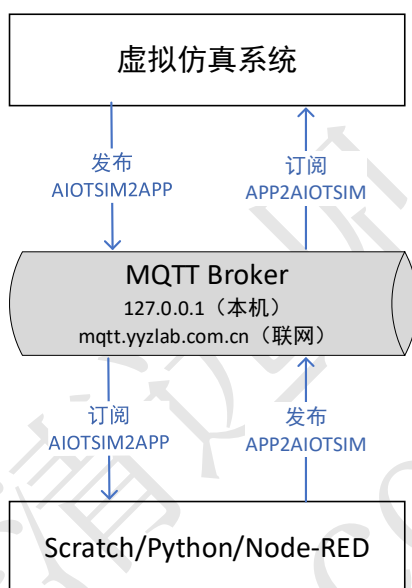
本章节的 Python 代码执行源码需要在 Python 环境下执行。执行 exe 则不需要。

4.1 火焰传感器数据采集

4.1.1 实验目的

通过 Python 实现对虚拟仿真软件上的火焰传感器的数据进行采集。

4.1.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

4.1.3 实验步骤

打开物联网虚拟仿真系统，从 01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-ZigBee 传感系统和执行通信协议中导入连线文件。

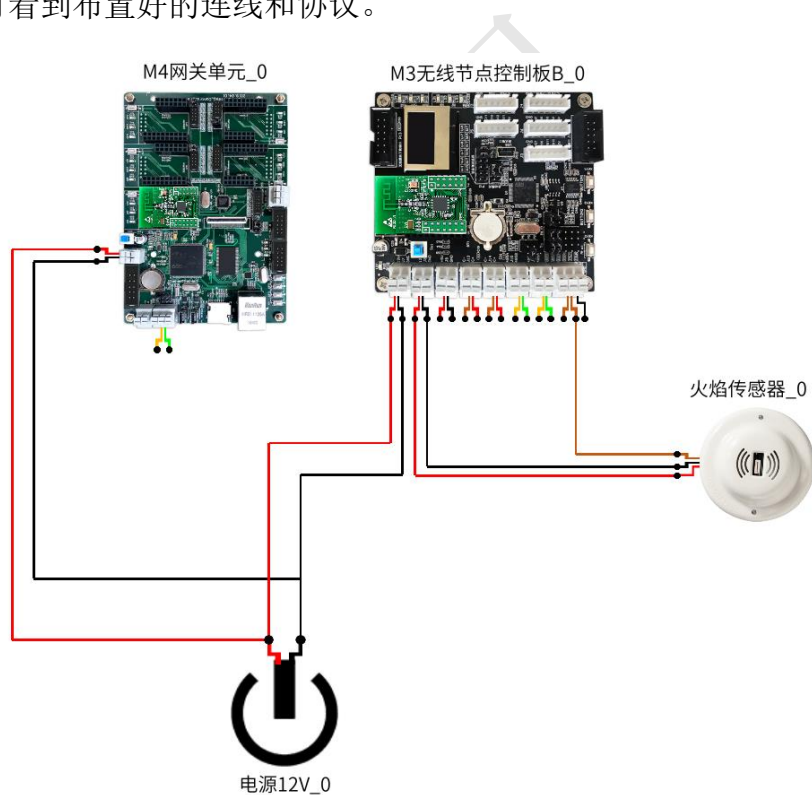


↑ 文件夹 « 01-程序源码 » 物联网虚拟仿真软件实验 » 03-ZigBee传感系统和执行通信协议

文件夹

名称	修改日期
07-ZigBee雨雪传感系统通信协议实验.AIOTSIM	11 10:15
08-ZigBee烟雾传感系统通信协议实验.AIOTSIM	11 11:03
09-ZigBee PM2.5传感系统通信协议实验.AIO...	11 11:05
10-ZigBee火焰传感系统通信协议实验.AIOTSIM	15 11:49
11-ZigBee声光报警执行系统通信协议实验.AIO...	11 11:09

打开后即可看到布置好的连线和协议。



下面配置一下 MQTT 协议。设置好相关的话题之后点击连接和订阅。



Mqtt配置

MQTT:

与应用层、云端交互的Topic:

与硬件交互的Topic:

在 Python 文件中输入如下代码并执行，可以连接到 MQTT broker 上自动获得虚拟仿真系统中的火焰传感器状态。

```

1  # -*- coding:utf-8 -*-
2  import paho.mqtt.client as mqtt
3  from queue import Queue
4  import json
5  import base64
6  import requests
7
8  import time
9
10 class HQYJMQttClient:
11     def __init__(self, broker_ip: str, broker_port: int):
12         self.mqtt_queue = Queue(255)
13         self.is_connected = False
14         self.client = mqtt.Client()
15         self.client.on_message = self.on_message
16         self.client.on_connect = self.on_connect
17         self.rc = 100
18         try:
19             self.client.connect(broker_ip, broker_port, 3)
20         except Exception as e:
21             print(e)
22
23     def on_message(self, client, userdata, message):
24         msg = json.loads(message.payload.decode())

```



```

25     self.mqtt_queue.put(msg)
26
27     def on_connect(self, client, userdata, flags, rc):
28         print("result code:", rc)
29         self.rc = rc
30
31
32 # 注意: 在 websocket 端口是 9001, 如使用 Python 代码, 端口需手动改成 1883。
33 hqyj_mqtt_clt = HQYJMqttClient('127.0.0.1', 1883)
34 hqyj_mqtt_clt.client.loop_start()
35 time.sleep(3)
36 if hqyj_mqtt_clt.rc == 0:
37     print('MQTT Connect')
38     hqyj_mqtt_clt.client.subscribe('AIOTSIM2APP', qos=0)
39
40     while True:
41         mqtt_data = hqyj_mqtt_clt.mqtt_queue.get()
42
43         if ('infrared' in mqtt_data) and ('id' in mqtt_data) and (mqtt_data['id'] == 0):
44             if (mqtt_data["infrared"]) > 0:
45                 hqyj_mqtt_clt.client.publish('APP2AIOTSIM', payload=json.dumps({"doorLock":True, "i
46 d":0}),ensure_ascii=False))
47             if (mqtt_data["infrared"]) < 1:
48                 hqyj_mqtt_clt.client.publish('APP2AIOTSIM', payload=json.dumps({"doorLock":False,
49 "id":0}),ensure_ascii=False))

```

运行该程序后, 可以在命令行界面显示接收到的数据。

```

# -*- coding:utf-8 -*-
import paho.mqtt.client as mqtt
from queue import Queue
import json
import base64
import requests

import time

class HQYJMqttCli
    def __init__(se
        self.mqtt_que
        self.is_conne

```

```

C:\Windows\SYSTEM32\cmd.exe
result code: 0
MQTT Connected
{'flame': True, 'id': 0}
{'flame': True, 'id': 0}
{'flame': True, 'id': 0}

```

我们也可以用 tkinter 来构造基于 Python 的应用程序, 代码如下:

```

1     from tkinter import *
2 import tkinter.messagebox
3 import paho.mqtt.client as mqtt
4 import threading
5 import json

```



```

6
7 class MQTTClient:
8     """
9     mqtt
10    """
11    def __init__(self):
12        self.message_flame = ""
13
14    def param_set(self, target_ip, default_port, sub_topic, pub_topic):
15        """
16        初始化
17        :param target_ip: mqtt 服务器 IP
18        :param default_port: mqtt 端口号
19        :param sub_topic: sub 的 topic
20        :param pub_topic: pub 的 topic
21        """
22        self.client = mqtt.Client()
23        self.client.on_connect = self.on_connect
24        self.client.on_message = self.on_message
25        self.client.connect(target_ip, default_port, 5)
26        self.pub_topic = pub_topic
27        self.client.subscribe(sub_topic, qos=0)
28        self.client.loop_start()
29
30    def on_connect(self, client, userdata, flags, rc):
31        """
32        连接功能
33        :param client:
34        :param userdata:
35        :param flags:
36        :param rc:
37        :return:
38        """
39        tkinter.messagebox.showinfo("MQTT 始化", "MQTT 连接成功")
40
41    def on_message(self, client, userdata, msg):
42        """
43        接收数据
44        :param client:
45        :param userdata:
46        :param msg: 接收到的信息
47        :return: none
48        """
49        msg = json.loads(str(msg.payload.decode('utf-8')))
50    try:

```



```

51         self.message_flame = msg['flame']
52         # print(msg)
53     except Exception as e:
54         pass
55
56     def send_mqtt(self, data):
57         """
58         发送 mqtt
59         :param data: 发送的数据, 字符串
60         :return: none
61         """
62         self.client.publish(self.pub_topic, payload='{{"{}"}}'.format(str(data)), qos=0)
63
64 class InterfaceStyle:
65     def __init__(self):
66         self.ip = ""
67         self.end_of_code = True
68
69     def on_closing(self):
70         if tkinter.messagebox.askokcancel("退出", "确定要退出吗?"):
71             self.end_of_code = False
72             self.root.destroy()
73
74
75     def type(self):
76         self.root = Tk()
77         self.root.title("火焰传感器数据采集")
78         self.root.geometry("430x320")
79         self.type_text = "火焰传感器"
80         # 背景设置
81         background_img = PhotoImage(file="image/背景.png")
82         background_label = Label(self.root, image=background_img)
83         background_label.pack()
84
85         # 空气温湿度传感器
86         img = PhotoImage(file="image/火焰传感器.png")
87         frame = Frame(self.root, bd=10, width=270, height=200, bg="ivory")
88         frame.place(x=70, y=42)
89
90         self.var = StringVar(frame, "状态: 未连接")
91
92         name = Label(frame, text=self.type_text, bg="ivory")
93         name.place(x=10, y=135)
94
95         value = Label(frame, textvariable=self.var, bg="ivory")

```

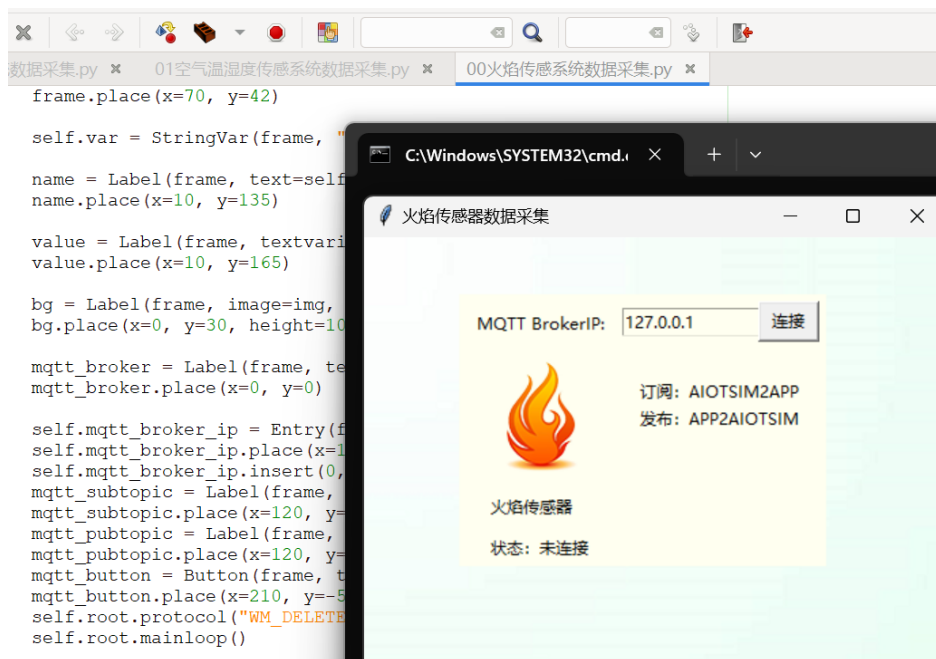


```

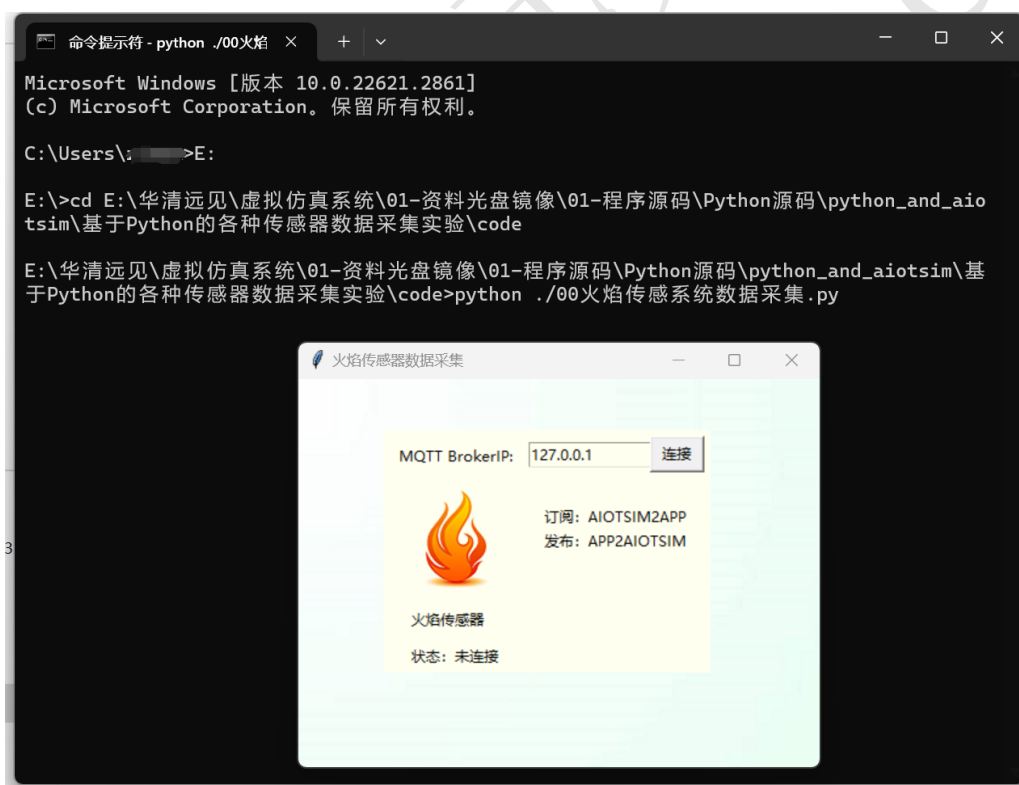
96         value.place(x=10, y=165)
97
98         bg = Label(frame, image=img, height=100, width=100, bg="ivory")
99         bg.place(x=0, y=30, height=100, width=100)
100
101         mqtt_broker = Label(frame, text="MQTT BrokerIP:", bg="ivory")
102         mqtt_broker.place(x=0, y=0)
103
104         self.mqtt_broker_ip = Entry(frame, width=15, bg="ivory")
105         self.mqtt_broker_ip.place(x=110, y=0)
106         self.mqtt_broker_ip.insert(0, "127.0.0.1")
107         mqtt_subtopic = Label(frame, text="订阅: AIOTSIM2APP", bg="ivory")
108         mqtt_subtopic.place(x=120, y=50)
109         mqtt_pubtopic = Label(frame, text="发布: APP2AIOTSIM", bg="ivory")
110         mqtt_pubtopic.place(x=120, y=70)
111         mqtt_button = Button(frame, text="连接", width=5, command=self.mqtt_init)
112         mqtt_button.place(x=210, y=-5)
113         self.root.protocol("WM_DELETE_WINDOW", self.on_closing)
114         self.root.mainloop()
115
116     def mqtt_init(self):
117         self.ip = self.mqtt_broker_ip.get()
118
119 if __name__ == '__main__':
120     interface_style = InterfaceStyle()
121     t_interface_style = threading.Thread(target=interface_style.type)
122     t_interface_style.start()
123     mqtt_client = MQTTClient()
124
125     while interface_style.end_of_code:
126         if interface_style.ip != "":
127             try:
128                 mqtt_client.param_set(interface_style.ip, 1883, "AIOTSIM2APP", "APP2AIOTSI
129 M")
130             except Exception as e:
131                 tkinter.messagebox.showwarning("MQTT 初始化失败", e)
132                 interface_style.ip = ""
133             elif mqtt_client.message_flame != "":
134                 tempval = "状态: {}".format(mqtt_client.message_flame)
135                 interface_style.var.set(tempval)
136                 mqtt_client.message_flame = ""

```

运行这段程序后，可以生成一个窗口：



我们也可以用命令行的方式运行这段程序：



在该窗口中，可以设置连接的 MQTT Broker 的 IP 地址，订阅的话题为 AIOTSIM2APP，发布的话题为 APP2AIOTSIM。我们点击连接按钮，可以提示连接成功。



此时运行虚拟仿真系统，即可在此窗口上获得传感器的状态。



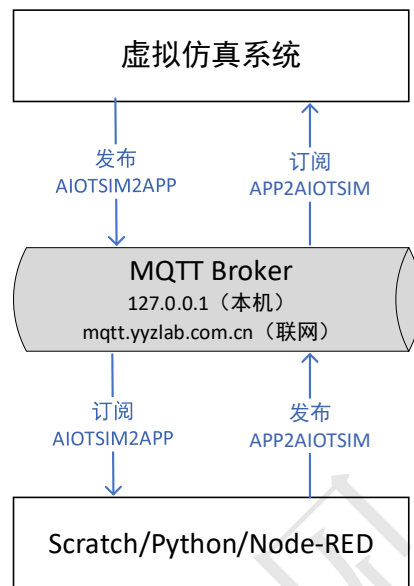
4.2 空气温湿度传感系统数据采集

4.2.1 实验目的

通过 Python 实现对虚拟仿真软件上的空气温湿度传感器的数据进行采集。



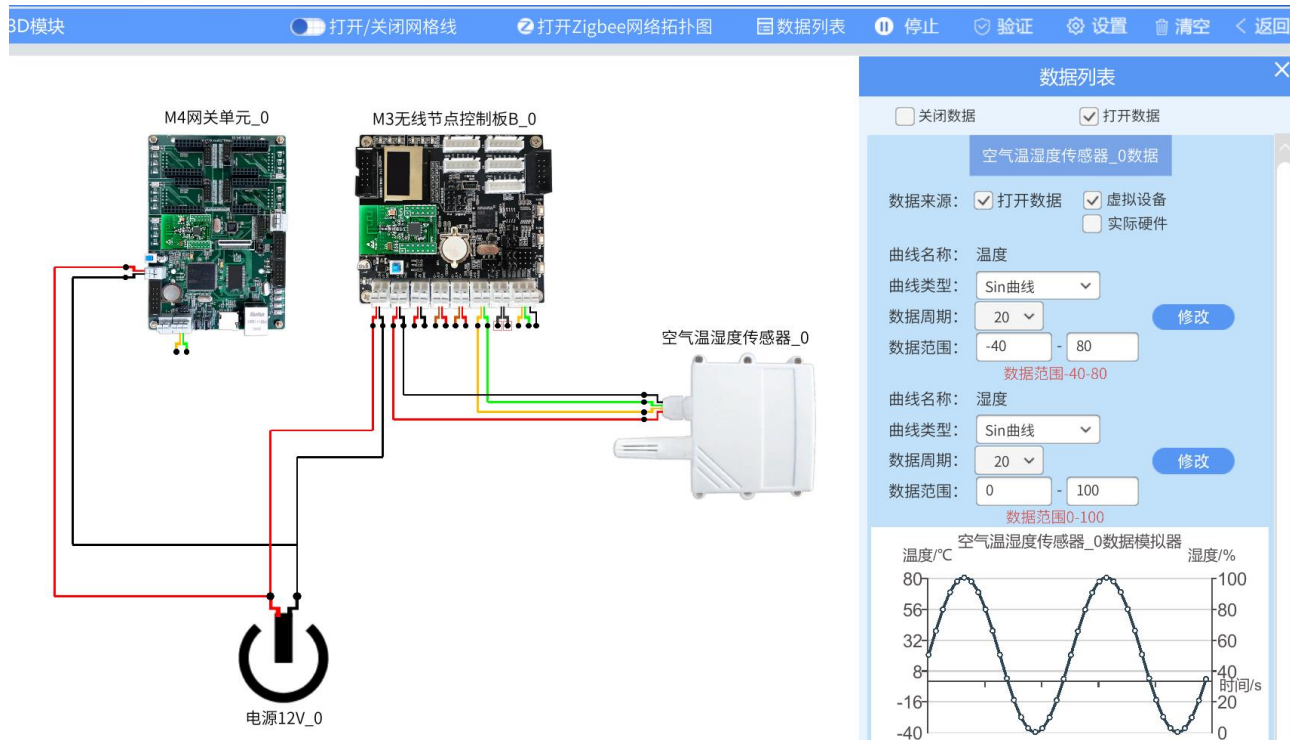
4.2.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

4.2.3 实验步骤

打开虚拟仿真软件，导入\01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-ZigBee 传感系统和执行通信协议下的 ZigBee 空气温湿度传感系统通信协议实验.AIOTSIM。打开数据并链接订阅好 MQTT 后，运行系统。注意订阅的话题要与 Python 程序对应。

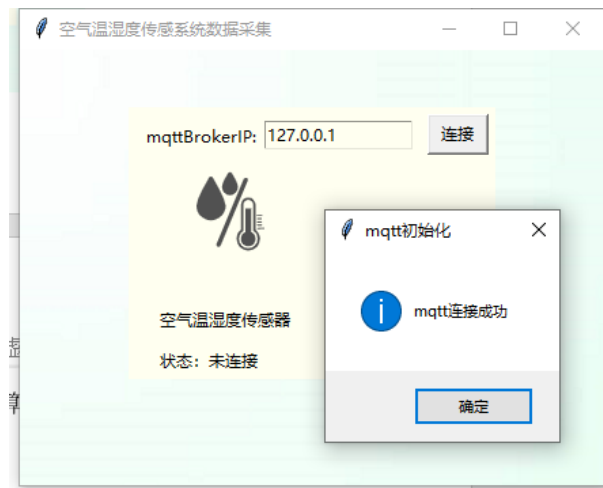


运行 Python 代码或者执行空气温湿度传感系统数据采集的 exe 文件。路径位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\dist。程序对应的代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\code。



输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



然后就可以看到上传的虚拟仿真的数据了。



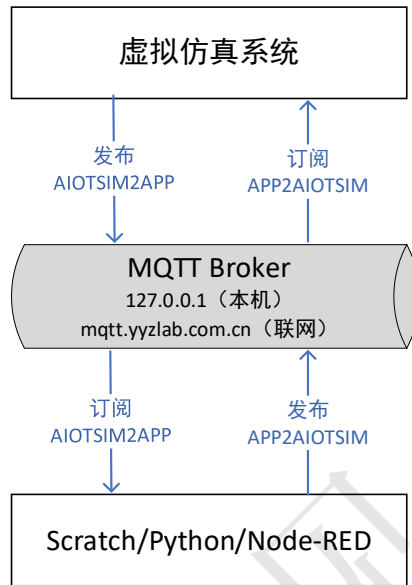
4.3 CO2 传感系统数据采集

4.3.1 实验目的

通过 Python 实现对虚拟仿真软件上的二氧化碳传感器的数据进行采集。



4.3.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

4.3.3 实验步骤

打开虚拟仿真软件，导入\01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-ZigBee 传感系统和执行通信协议下的 ZigBeeCO2 传感系统通信协议实验.AIOTSIM。打开数据并链接订阅好 MQTT 后，运行系统。注意订阅的话题要与 Python 程序对应。

运行 Python 代码或者执行 CO2 传感系统数据采集的 exe 文件。路径位于 01-资料光盘镜



像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\dist。
程序对应的代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于
Python 的各种传感器数据采集实验\code。



输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



然后就可以看到上传的虚拟仿真的数据了。

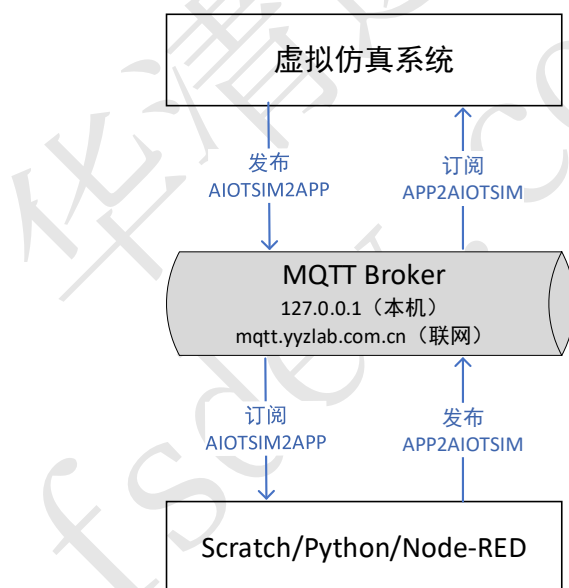


4.4 土壤温湿度传感系统数据采集

4.4.1 实验目的

通过 Python 实现对虚拟仿真软件上的土壤温湿度传感器的数据进行采集。

4.4.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

4.4.3 实验步骤

打开虚拟仿真软件，导入 \01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-



ZigBee 传感系统和执行通信协议下的 ZigBee 土壤温湿度传感系统通信协议实验.AIOTSIM。
打开数据并链接订阅好 MQTT 后，运行系统。注意订阅的话题要与 Python 程序对应。

MQTT配置

MQTT:

IP地址: 127.0.0.1 端口: 1883

ClientID: 请输入或者生成ClientID 生成

用户名: 请输入用户名

密码: 请输入密码

断开

与应用层、云端交互的Topic:

订阅: APP2AIOTSIM 复制

发布: AIOTSIM2APP 复制

生成Topic 取消订阅

与硬件交互的Topic:

运行 Python 代码或者执行土壤温湿度传感系统数据采集的 exe 文件。路径位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\dist。程序对应的代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\code。

土壤温湿度传感系统数据采集

mqttBrokerIP: 127.0.0.1 连接

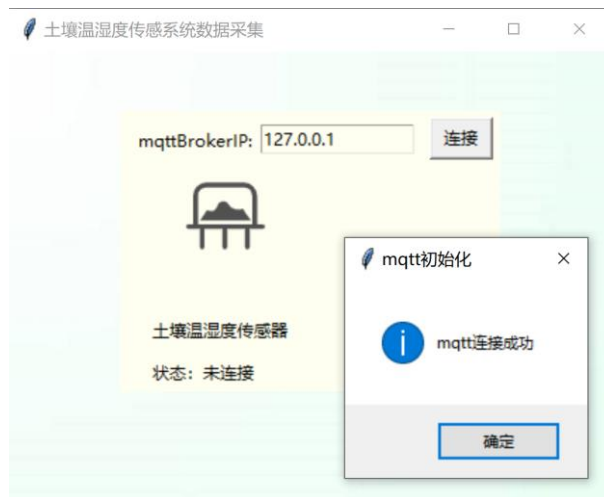
订阅: AIOTSIM2APP
发布: APP2AIOTSIM

土壤温湿度传感器

状态: 未连接

输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



然后就可以看到上传的虚拟仿真的数据了。



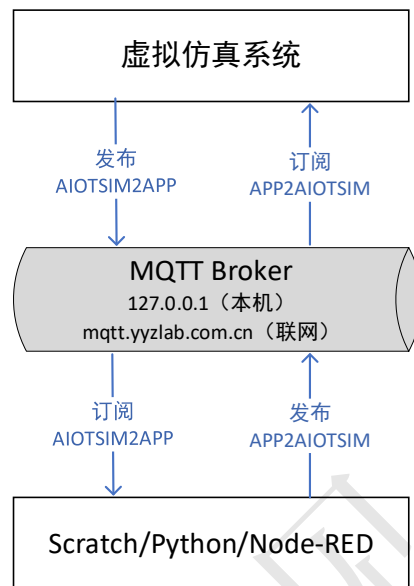
4.5 PM2.5 传感系统数据采集

4.5.1 实验目的

通过 Python 实现对虚拟仿真软件上的 PM2.5 传感器的数据进行采集。



4.5.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

4.5.3 实验步骤

打开虚拟仿真软件，导入\01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-ZigBee 传感系统和执行通信协议下的 ZigBeePM2.5 传感系统通信协议实验.AIOTSIM。打开数据并链接订阅好 MQTT 后，运行系统。注意订阅的话题要与 Python 程序对应。

运行 Python 代码或者执行 PM2.5 传感系统数据采集的 exe 文件。路径位于 01-资料光盘



镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\dist。程序对应的代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\code。

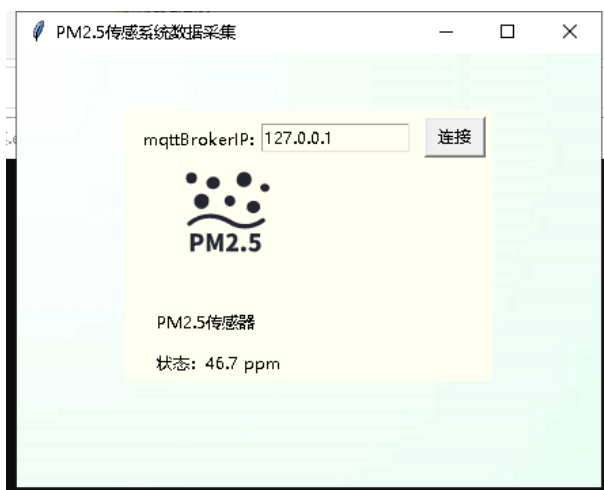


输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



然后就可以看到上传的虚拟仿真的数据了。

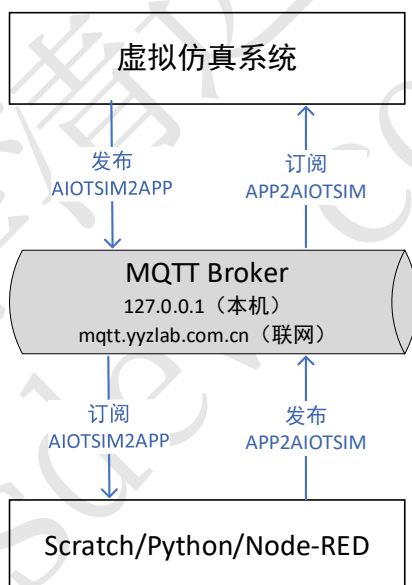


4.6 人体红外传感系统数据采集

4.6.1 实验目的

通过 Python 实现对虚拟仿真软件上的人体红外传感器的数据进行采集。

4.6.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

4.6.3 实验步骤

打开虚拟仿真软件，导入\01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-ZigBee 传感系统和执行通信协议下的 ZigBee 人体红外传感系统通信协议实验.AIOTSIM。打



开数据并链接订阅好 MQTT 后，运行系统。注意订阅的话题要与 Python 程序对应。

MQTT配置窗口包含以下配置项：

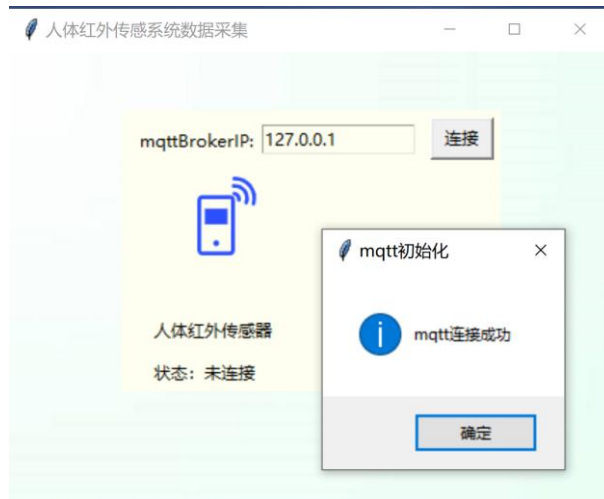
- MQTT:
 - IP地址: 127.0.0.1
 - 端口: 1883
 - ClientID: 请输入或者生成ClientID (生成按钮)
 - 用户名: 请输入用户名
 - 密码: 请输入密码
 - 断开按钮
- 与应用层、云端交互的Topic:
 - 订阅: APP2AIOTSIM (复制按钮)
 - 发布: AIOTSIM2APP (复制按钮)
 - 生成Topic按钮
 - 取消订阅按钮
- 与硬件交互的Topic:

运行 Python 代码或者执行人体红外传感系统数据采集的 exe 文件。路径位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\dist。程序对应的代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\code。

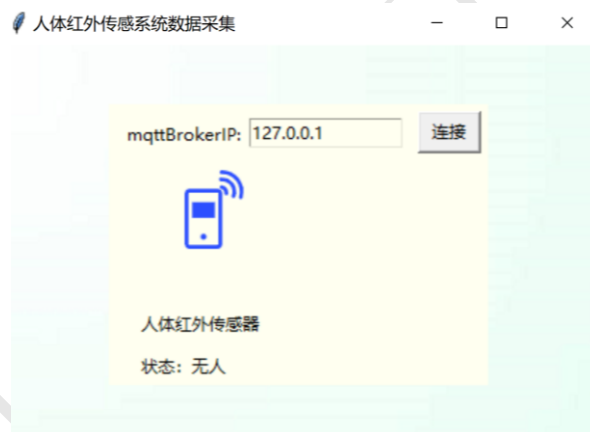


输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



然后就可以看到上传的虚拟仿真的数据了。



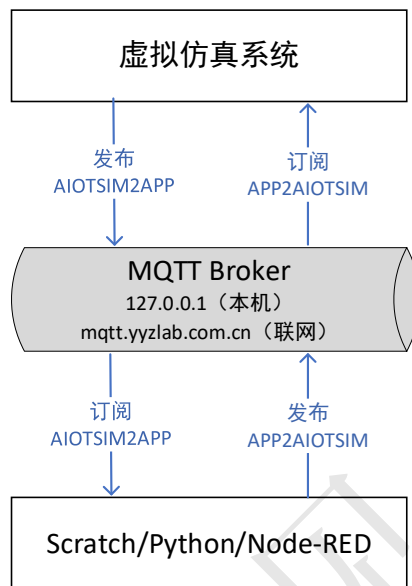
4.7 烟雾传感系统数据采集

4.7.1 实验目的

通过 Python 实现对虚拟仿真软件上的烟雾传感器的数据进行采集。



4.7.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

4.7.3 实验步骤

打开虚拟仿真软件，导入\01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-ZigBee 传感系统和执行通信协议下的 ZigBee 烟雾传感系统通信协议实验.AIOTSIM。打开数据并链接订阅好 MQTT 后，运行系统。注意订阅的话题要与 Python 程序对应。

The screenshot shows the 'Mqtt配置' (MQTT Configuration) window. It contains the following fields and buttons:

- MQTT:**
 - IP地址: 127.0.0.1
 - 端口: 1883
 - ClientID: 请输入或者生成ClientID (with a '生成' button)
 - 用户名: 请输入用户名
 - 密码: 请输入密码
 - '断开' button
- 与应用层、云端交互的Topic:**
 - 订 阅: APP2AIOTSIM (with a '复制' button)
 - 发 布: AIOTSIM2APP (with a '复制' button)
 - '生成Topic' button
 - '取消订阅' button
- 与硬件交互的Topic:** (This section is partially visible at the bottom)

运行 Python 代码或者执行烟雾传感系统数据采集的 exe 文件。路径位于 01-资料光盘镜



像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\dist。
程序对应的代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于
Python 的各种传感器数据采集实验\code。



输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



然后就可以看到上传的虚拟仿真的数据了。

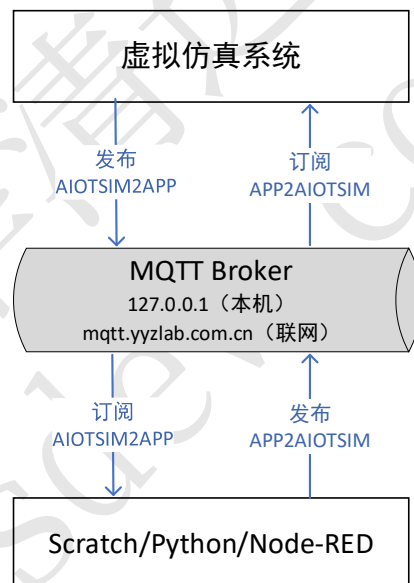


4.8 可燃气体传感系统数据采集

4.8.1 实验目的

通过 Python 实现对虚拟仿真软件上的可燃气体传感器的数据进行采集。

4.8.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

4.8.3 实验步骤

打开虚拟仿真软件，导入 \01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-ZigBee 传感系统和执行通信协议下的 ZigBee 可燃气体传感系统通信协议实验.AIOTSIM。打开



数据并链接订阅好 MQTT 后，运行系统。注意订阅的话题要与 Python 程序对应。

The image shows a 'Mqtt配置' (MQTT Configuration) dialog box. It contains the following fields and buttons:

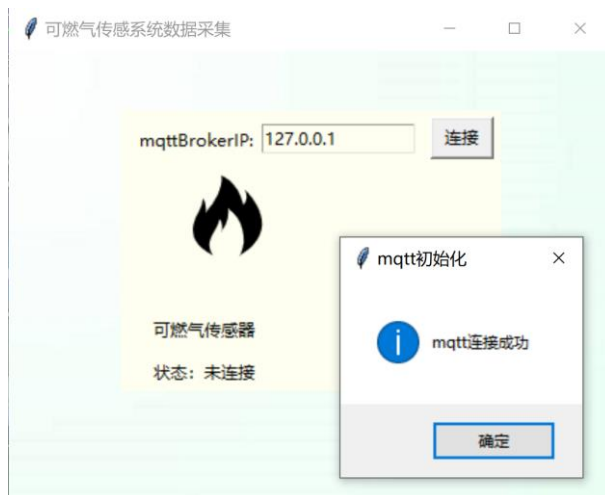
- MQTT:**
 - IP地址: 127.0.0.1
 - 端口: 1883
 - ClientID: 请输入或者生成ClientID (with a '生成' button)
 - 用户名: 请输入用户名
 - 密码: 请输入密码
 - 断开 button
- 与应用层、云端交互的Topic:**
 - 订阅: APP2AIOTSIM (with a '复制' button)
 - 发布: AIOTSIM2APP (with a '复制' button)
 - 生成Topic button
 - 取消订阅 button
- 与硬件交互的Topic:**

运行 Python 代码或者执行可燃气体传感系统数据采集的 exe 文件。路径位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\dist。程序对应的代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\code。



输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



然后就可以看到上传的虚拟仿真的数据了。



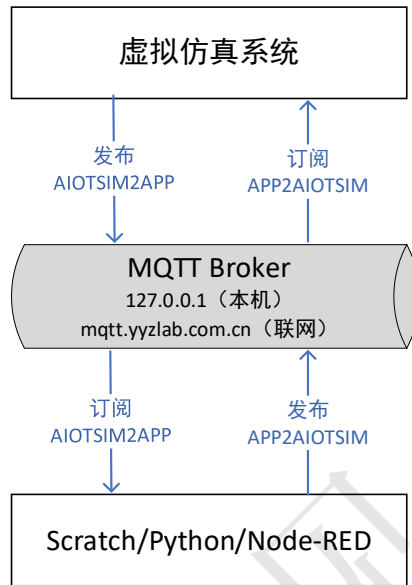
4.9 雨雪传感系统数据采集

4.9.1 实验目的

通过 Python 实现对虚拟仿真软件上的雨雪传感器的数据进行采集。



4.9.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

4.9.3 实验步骤

打开虚拟仿真软件，导入\01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-ZigBee 传感系统和执行通信协议下的 ZigBee 雨雪传感系统通信协议实验.AIOTSIM。打开数据并链接订阅好 MQTT 后，运行系统。注意订阅的话题要与 Python 程序对应。

The screenshot shows the 'Mqtt配置' (MQTT Configuration) window. It includes fields for IP address (127.0.0.1), Port (1883), ClientID, Username, and Password. There are buttons for '生成' (Generate), '断开' (Disconnect), '订阅' (Subscribe), '发布' (Publish), '生成Topic' (Generate Topic), and '取消订阅' (Unsubscribe). The '订阅' field is set to 'APP2AIOTSIM' and the '发布' field is set to 'AIOTSIM2APP'.

运行 Python 代码或者执行可燃气体传感系统数据采集的 exe 文件。路径位于 01-资料光盘



镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\dist。程序对应的代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\code。



输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



然后就可以看到上传的虚拟仿真的数据了。

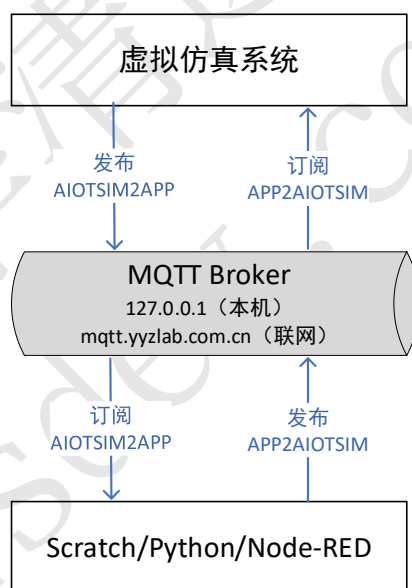


4.10 光照传感系统数据采集

4.10.1 实验目的

通过 Python 实现对虚拟仿真软件上的光照传感器的数据进行采集。

4.10.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

4.10.3 实验步骤

打开虚拟仿真软件，导入 \01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-



ZigBee 传感系统和执行通信协议下的 ZigBee 光照传感系统通信协议实验.AIOTSIM。打开数据并链接订阅好 MQTT 后，运行系统。注意订阅的话题要与 Python 程序对应。

The screenshot displays the AIOTSIM software interface, which is divided into several sections:

- Top Bar:** Contains navigation buttons such as "打开/关闭网络线" (Open/Close Network Line), "打开Zigbee网络拓扑图" (Open Zigbee Network Topology Diagram), "数据列表" (Data List), "停止" (Stop), "验证" (Verify), "设置" (Settings), "清空" (Clear), and "< 返回" (Back).
- Hardware Diagram:** Shows the physical connection between components:
 - M4网关单元_0** (M4 Gateway Unit_0): A green circuit board.
 - M3无线节点控制板B_0** (M3 Wireless Node Control Board B_0): A black circuit board.
 - 光照传感器_0** (Light Sensor_0): A white sensor module.
 - 电源12V_0** (12V Power Source_0): A power supply symbol.
 Wires connect the power source to the gateway and node boards, and the node board to the light sensor.
- 数据列表 (Data List) Panel:**
 - Buttons: "关闭数据" (Close Data), "打开数据" (Open Data).
 - Section: "光照传感器_0数据" (Light Sensor_0 Data).
 - Options: "数据来源:" (Data Source) with "打开数据" (Open Data) and "虚拟设备" (Virtual Device) checked, and "实际硬件" (Actual Hardware) unchecked.
 - Curve Name: "光照值" (Light Value).
 - Curve Type: "Sin曲线" (Sin Curve).
 - Data Period: "20".
 - Data Range: "0" to "65535".
 - Graph: A line graph titled "光照传感器_0数据模拟器" (Light Sensor_0 Data Simulator) showing a sine wave. The y-axis is "光照值/Lux" (Light Value/Lux) ranging from 0 to 65535. The x-axis is "时间/s" (Time/s) ranging from 0 to 32. A data point is highlighted at "光照值:6145".
- Mqtt配置 (MQTT Configuration) Panel:**
 - MQTT:
 - IP地址: 127.0.0.1
 - 端口: 1883
 - ClientID: 请输入或者生成ClientID (Please enter or generate ClientID)
 - 用户名: 请输入用户名 (Please enter username)
 - 密码: 请输入密码 (Please enter password)
 - Buttons: "生成" (Generate), "断开" (Disconnect).
 - 与应用层、云端交互的Topic: (Topic for interaction with application layer and cloud)
 - 订阅: APP2AIOTSIM (Subscribe)
 - 发布: AIOTSIM2APP (Publish)
 - Buttons: "生成Topic" (Generate Topic), "取消订阅" (Cancel Subscription).
 - 与硬件交互的Topic: (Topic for interaction with hardware)
 - 订阅: Device2AIOTSIM (Subscribe)
 - Buttons: "复制" (Copy).

运行 Python 代码或者执行光照传感系统数据采集的 exe 文件。路径位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\dist。程序对应的代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种传感器数据采集实验\code。



输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



然后就可以看到上传的虚拟仿真的数据了。





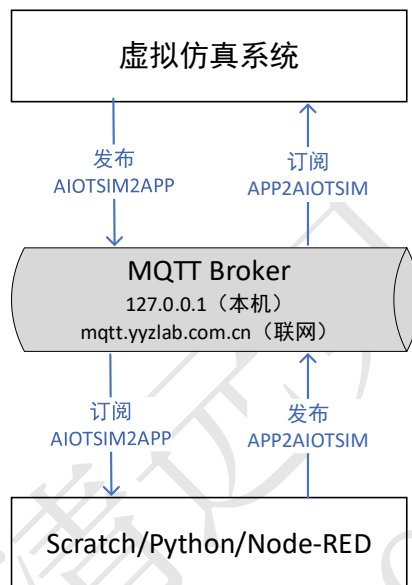
第 5 章 基于 Python 的各种执行器控制实验

5.1 声光报警器执行系统控制实验

5.1.1 实验目的

通过 Python 实现对虚拟仿真软件上的声光报警传感器的数据进行采集。

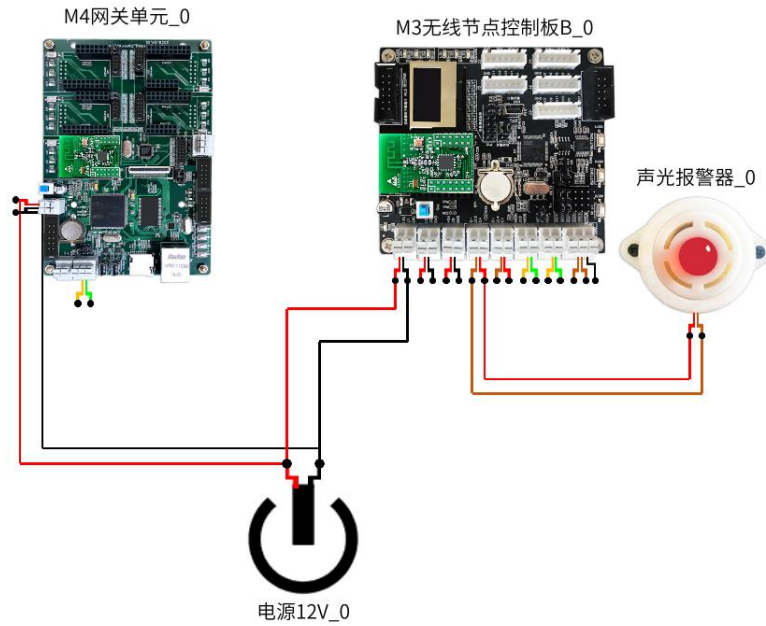
5.1.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

5.1.3 实验步骤

打开虚拟仿真软件，导入\01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-ZigBee 传感系统和执行通信协议下的 11-ZigBee 声光报警执行系统通信协议实验.AIOTSIM。打开数据并链接订阅好 MQTT 后，运行系统。注意订阅的话题要与 Python 程序对应。



Mqtt配置 ✕

MQTT:

IP地址: 127.0.0.1

端口: 1883

ClientID: 请输入或者生成ClientID

生成

用户名: 请输入用户名

密 码: 请输入密码

断开

与应用层、云端交互的Topic:

订 阅: APP2AIOTSIM

复制

发 布: AIOTSIM2APP

复制

生成Topic

取消订阅

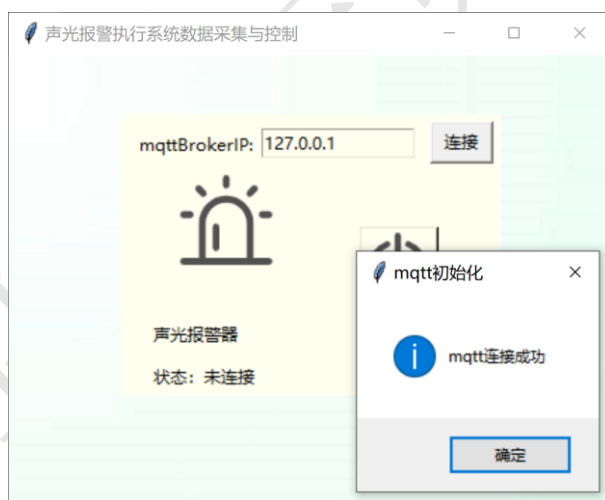
与硬件交互的Topic:

运行 Python 代码或者执行声光报警执行系统数据采集的 exe 文件。路径位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种执行器控制实验\dist。代码路径位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种执行器控制实验\code。



输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。

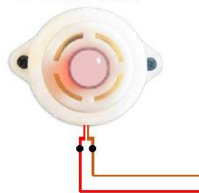


点击按钮开关,可观察到按钮开关变为绿色,”状态”变为打开;物联网虚拟仿真系统中声光报警器_0 开始闪烁





声光报警器_0



再次点击按钮开关,可观察到按钮开关变为黑色,”状态”变为关闭;物联网虚拟仿真系统中声光报警器_0 停止闪烁



声光报警器_0



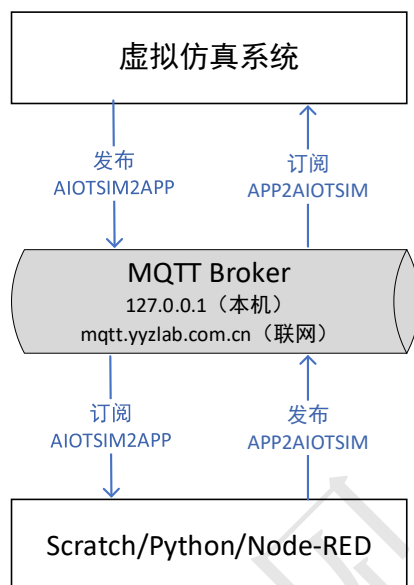
5.2 喷淋执行系统控制实验

5.2.1 实验目的

通过 Python 实现对虚拟仿真软件上的喷淋传感器的数据进行采集。



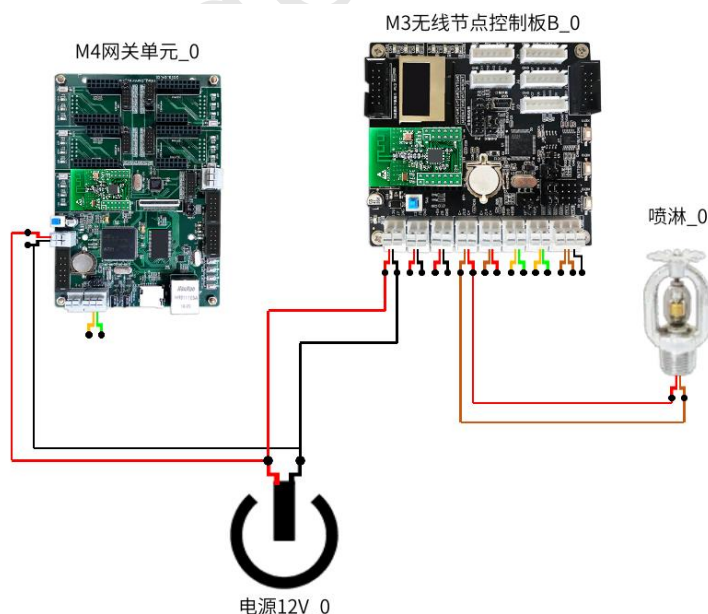
5.2.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

5.2.3 实验步骤

打开虚拟仿真软件，导入\01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-ZigBee 传感系统和执行通信协议下的 ZigBee 喷淋执行系统通信协议实验.AIOTSIM。打开数据并链接订阅好 MQTT 后，运行系统。注意订阅的话题要与 Python 程序对应。





Mqtt配置
✕

MQTT:

与应用层、云端交互的Topic:

与硬件交互的Topic:

运行 Python 代码或者执行喷淋执行系统数据采集的 exe 文件。路径位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种执行器控制实验\dist



输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。





点击按钮开关,可观察到按钮开关变为绿色,”状态”变为打开;物联网虚拟仿真系统中喷淋_0 开始冒”水花”



再次点击按钮开关,可观察到按钮开关变为黑色,”状态”变为关闭;物联网虚拟仿真系统中喷淋_0 停止冒”水花”。





喷淋_0

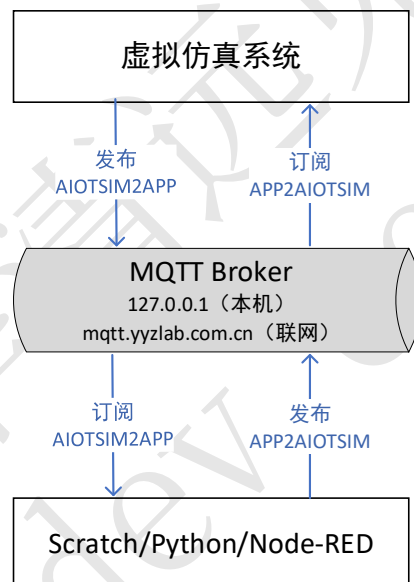


5.3 电灯执行系统控制实验

5.3.1 实验目的

通过 Python 实现对虚拟仿真软件上的灯光传感器的数据进行采集。

5.3.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

5.3.3 实验步骤

打开虚拟仿真软件，导入\01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-ZigBee 传感系统和执行通信协议下的 ZigBee 灯光执行系统通信协议实验.AIOTSIM。打开数据并链接订阅好 MQTT 后，运行系统。注意订阅的话题要与 Python 程序对应。



Mqtt配置
✕

MQTT:

IP地址: 127.0.0.1

端口: 1883

ClientID: 请输入或者生成ClientID

生成

用户名: 请输入用户名

密 码: 请输入密码

断开

与应用层、云端交互的Topic:

订 阅: APP2AIOTSIM

复制

发 布: AIOTSIM2APP

复制

生成Topic

取消订阅

与硬件交互的Topic:

运行 Python 代码或者执行灯光执行系统数据采集的 exe 文件。路径位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种执行器控制实验\dist

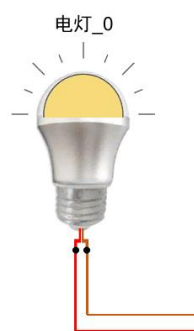


输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



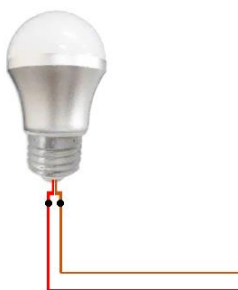
点击按钮开关,可观察到按钮开关变为绿色,”状态”变为打开;物联网虚拟仿真系统中电灯_0 变”亮”



再次点击按钮开关,可观察到按钮开关变为黑色, 状态变为关闭; 物联网虚拟仿真系统中电灯_0 变灭。



电灯_0

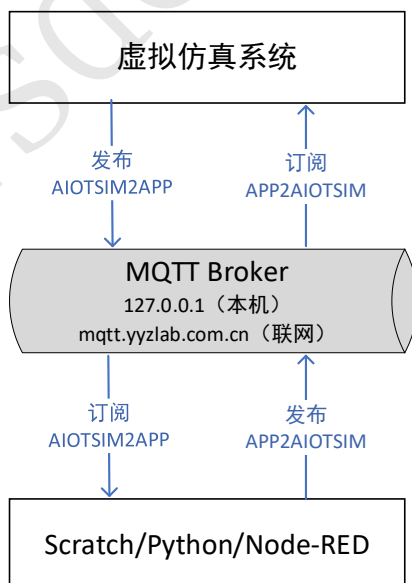


5.4 遮阳板执行系统控制实验

5.4.1 实验目的

通过 Python 实现对虚拟仿真软件上的遮阳板传感器的数据进行采集。

5.4.2 实验原理





在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

5.4.3 实验步骤

打开虚拟仿真软件，导入 \01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-ZigBee 传感系统和执行通信协议下的 ZigBee 遮阳板执行系统通信协议实验.AIOTSIM。打开数据并链接订阅好 MQTT 后，运行系统。注意订阅的话题要与 Python 程序对应。

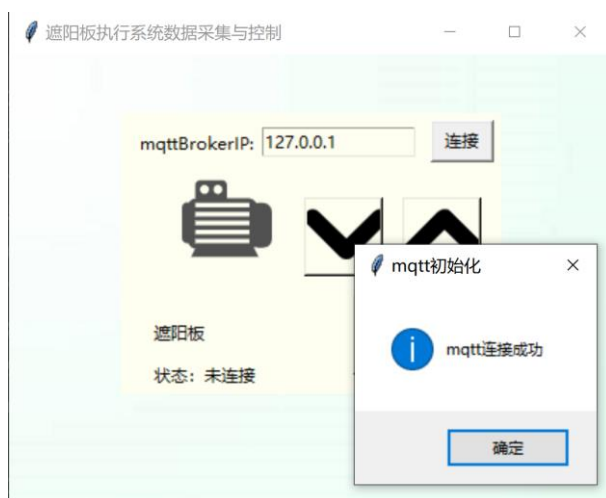
运行 Python 代码或者执行遮阳板执行系统数据采集的 exe 文件。路径位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种执行器控制实验\dist



输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。



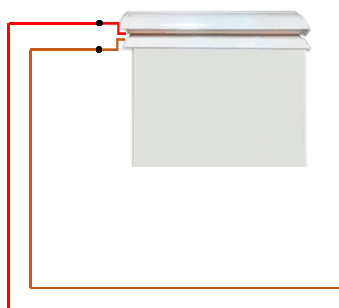
点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



点击”正转”按钮(向下箭头),可观察到”状态”变为正转,物联网虚拟仿真系统中电动遮阳板_0 会将”遮阳布”拉下来



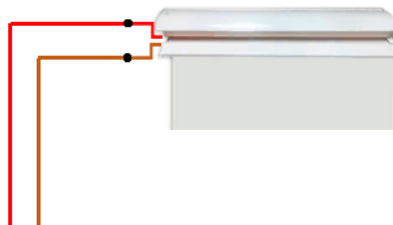
电动遮阳板_0



点击 “反转”按钮(向上箭头),可观察到 “状态”变为反转,物联网虚拟仿真系统中电动遮阳板_0 会将 “遮阳布”拉上去。



电动遮阳板_0



点击”停止”按钮,可观察到”状态”变为停止,物联网虚拟仿真系统中电动遮阳板_0 会将”遮阳布” 停下



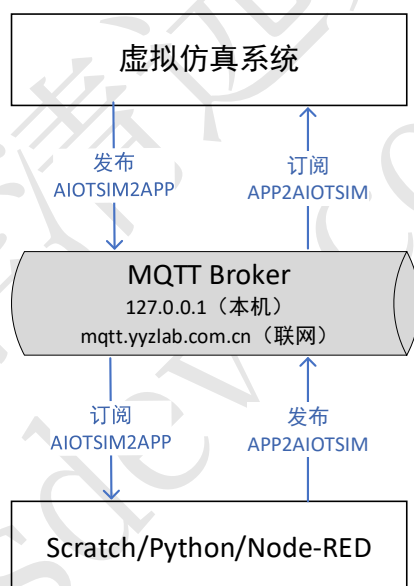


5.5 微型电控锁执行系统控制实验

5.5.1 实验目的

通过 Python 实现对虚拟仿真软件上的门锁传感器的数据进行采集。

5.5.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

5.5.3 实验步骤

打开虚拟仿真软件，导入\01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-ZigBee 传感系统和执行通信协议下的 ZigBee 门锁执行系统通信协议实验.AIOTSIM。打开数



据并链接订阅好 MQTT 后，运行系统。注意订阅的话题要与 Python 程序对应。

MQTT配置

MQTT:

IP地址: 127.0.0.1 端口: 1883

ClientID: 请输入或者生成ClientID 生成

用户名: 请输入用户名

密 码: 请输入密码

断开

与应用层、云端交互的Topic:

订 阅: APP2AIOTSIM 复制

发 布: AIOTSIM2APP 复制

生成Topic 取消订阅

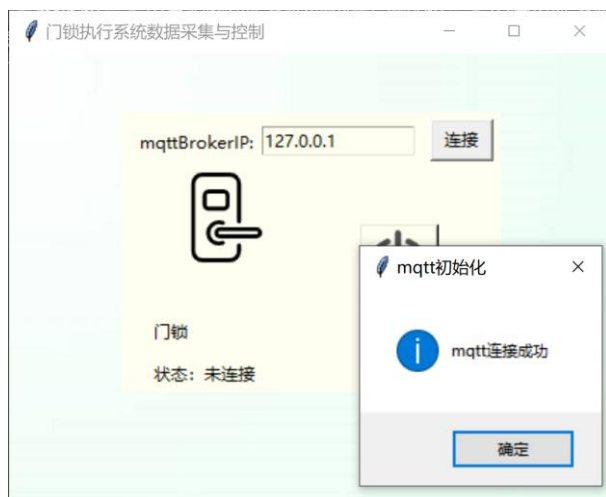
与硬件交互的Topic:

运行 Python 代码或者执行门锁执行系统数据采集的 exe 文件。路径位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种执行器控制实验\dist

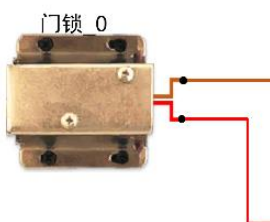


输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

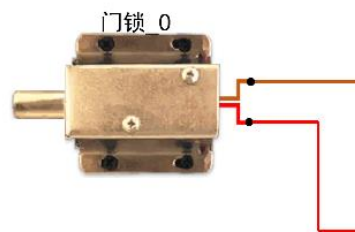
点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



点击按钮开关,可观察到按钮开关变为绿色,”状态”变为打开;物联网虚拟仿真系统中门锁_0”打开”



再次点击按钮开关,可观察到按钮开关变为黑色,”状态”变为关闭;物联网虚拟仿真系统中门锁_0”关闭”

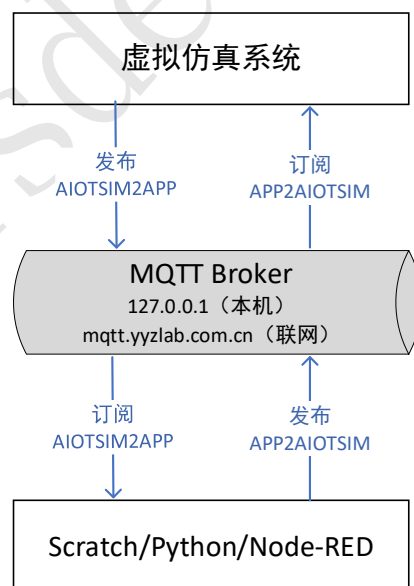


5.6 风扇执行系统控制实验

5.6.1 实验目的

通过 Python 实现对虚拟仿真软件上的风扇传感器的数据进行采集。

5.6.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相



关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

5.6.3 实验步骤

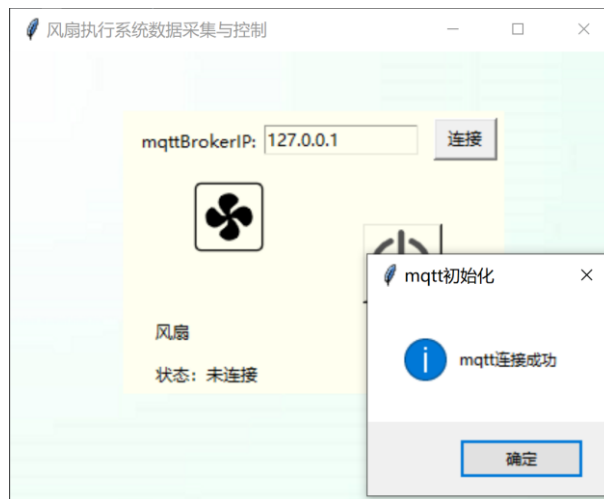
打开虚拟仿真软件，导入 01-资料光盘镜像\01-程序源码\物联网虚拟仿真软件实验\03-ZigBee 传感系统和执行通信协议下的 ZigBee 风扇执行系统通信协议实验.AIOTSIM。打开数据并链接订阅好 MQTT 后，运行系统。注意订阅的话题要与 Python 程序对应。

运行 Python 代码或者执行风扇执行系统数据采集的 exe 文件。路径位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种执行器控制实验\dist



输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



点击按钮开关,可观察到按钮开关变为绿色,”状态”变为打开;物联网虚拟仿真系统中风扇_0”转动”



再次点击按钮开关，可观察到按钮开关变为黑色，“状态”变为关闭；物联网虚拟仿真系统中风扇_0 停止转动。





第 6 章 基于 Python 的各种人工智能实验

6.1 人脸识别

6.1.1 实验目的

通过调用百度 AI 开放平台的接口，实现人脸识别项目。

6.1.2 实验原理

接口能力

- **人脸检测**：检测图片中的人脸并标记出位置信息；
- **人脸关键点**：展示人脸的核心关键点信息，及 150 个关键点信息。
- **人脸属性值**：展示人脸属性信息，如年龄、性别等。
- **人脸质量信息**：返回人脸各部分的遮挡、光照、模糊、完整度、置信度等信息。

业务应用

典型应用场景：如人脸属性分析，基于人脸关键点的加工分析，人脸营销活动等。

说明：检测响应速度，与图片中人脸数量相关，人脸数量较多时响应时间会有些许延长。

质量检测

如果需要判断一张图片中的人脸，是否符合后续识别或者对比的条件，可以使用此接口，在请求时在 `face_field` 参数中请求 `quality`。基于返回结果 `quality` 中，以下字段及对应阈值，进行质量检测的判断，以保证人脸质量符合后续业务操作要求。

指标	字段与解释	推荐数值界限
遮挡范围	occlusion ，取值范围[0~1]，0 为无遮挡，1 是完全遮挡 含有多个具体子字段，表示脸部多个部位 通常用作判断头发、墨镜、口罩等遮挡	left_eye : 0.6, #左眼被遮挡的阈值 right_eye : 0.6, #右眼被遮挡的阈值 nose : 0.7, #鼻子被遮挡的阈值 mouth : 0.7, #嘴巴被遮挡的阈值 left_cheek : 0.8, #左脸颊被遮挡的阈值 right_cheek : 0.8, #右脸颊被遮挡的阈值 chin_contour : 0.6, #下巴被遮挡阈值
模糊度范围	blur ，取值范围[0~1]，0 是最清晰，1 是最模糊	小于 0.7



指标	字段与解释	推荐数值界限
光照范围	illumination , 取值范围[0~255] 脸部光照的灰度值, 0 表示光照不好 以及对应客户端 SDK 中, YUV 的 Y 分量	大于 40
姿态角度	Pitch : 三维旋转之俯仰角度[-90(上), 90(下)] Roll : 平面内旋转角[-180(逆时针), 180(顺时针)] Yaw : 三维旋转之左右旋转角[-90(左), 90(右)]	分别小于 20 度
人脸完整度	completeness (0 或 1), 0 为人脸溢出图像边界, 1 为人脸都在图像边界内	视业务逻辑判断
人脸大小	人脸部分的大小 建议长宽像素值范围: 80*80~200*200	人脸部分不小于 100*100 像素

6.1.3 实验步骤

本节实验代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种人工智能实验

I 申请步骤

- 1、首先登陆 <https://ai.baidu.com/tech/face>, 进入到通用人脸识别的界面。选择立即使用。



- 2、登陆自己的百度账号, 可用手机号申请。



百度账号
云账号(推广账号)

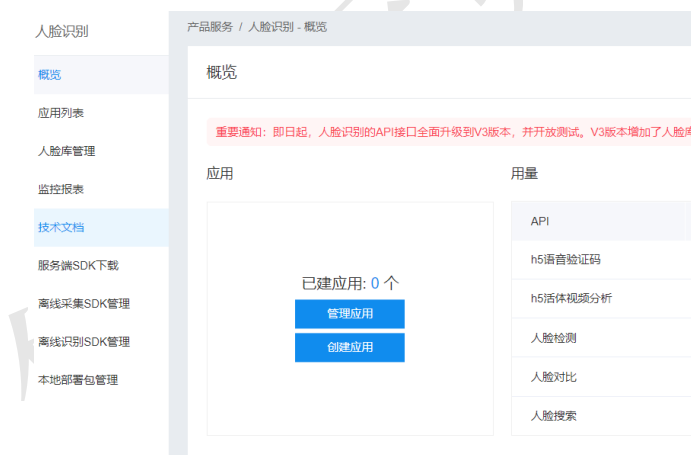
[短信快捷登录](#)

登录

[立即注册](#) [忘记密码](#)

温馨提示:
与百度搜索、百度贴吧、百度云盘、百度知道、百度文库等产品通用。

3、选择创建应用



4、依次输入应用名称，选择类型，接口默认选择人脸识别，添加应用描述，点击立即创建。



应用名称:

第一个人脸识别应用

应用类型:

游戏娱乐

接口选择:

勾选以下接口, 使此应用可以请求已勾选的接口服务, 注意人脸识别服务已默认勾选并不可取消。

人脸识别

h5语音验证码

h5活体视频分析

人脸检测

人脸对比

人脸搜索

人脸库管理

在线活体检测

人脸搜索-M-N识别

百度语音

文字识别

自然语言处理

内容审核

UNIT

知识图谱

图像识别

智能呼叫中心

图像搜索

人体分析

图像处理

应用描述:

第一个人脸识别应用

立即创建

取消

5、至此, 应用创建完毕, 现在就可以使用该应用了。

创建完毕

返回应用列表

查看应用详情

查看文档

下载SDK

6、下载 SDK, 下载 Python 的 SDK (当前版本: 2.2.4)。



人脸识别					
Java HTTP SDK	Java	4.3.2 人脸v3一些问题修复 2018-05-15	使用说明	↓ 下载	
PHP HTTP SDK	PHP	2.2.4 人脸v3一些问题修复 2018-05-10	使用说明	↓ 下载	
Python HTTP SDK	Python	2.2.4 人脸v3一些问题修复 2018-05-10	使用说明	↓ 下载	
C++ HTTP SDK	C++	0.7.4 修复一个token过期无法重现获取的... 2018-08-23	使用说明	↓ 下载	
C# HTTP SDK	C#	3.5.1 人脸v3一些问题修复 2018-05-10	使用说明	↓ 下载	
Node.js HTTP SDK	Node.js	2.3.1 人脸v3一些问题修复 2018-04-28	使用说明	↓ 下载	
HTTP SDK V2版本汇总		1.0.0 人脸v2版本最新版本所有语言SDK汇总 2018-05-02	使用说明	↓ 下载	

7、创建完毕界面，点击返回应用列表，将 AppID、API Key、Secret Key 记下。

应用列表						
+ 创建应用						
应用名称	AppID	API Key	Secret Key	创建时间	操作	
1 第一个人脸识别应用	15804109	Gx6ILUY0aA1d51BQmno9ATw	***** 显示	2019-03-20 14:52:52	报表 管理 查看人脸库 删除	

II 调用 API 实现识别

解压下载好的 SDK。

使用 pip 安装 baidu-aip 的包。

```
pip install baidu-aip
```

接下来我们开始实现人脸识别。

AipSpeech 是语音识别的 Python SDK 客户端，为使用图像识别的开发人员提供了一系列的交互方法。我们在解压的目录下新建一个 main.py。

baidu_api_face				
名称	修改日期	类型	大小	
aip	2018/5/10 14:08	文件夹		
bin	2018/5/10 14:08	文件夹		
LICENSE	2018/5/10 14:08	文件	12 KB	
main.py	2019/3/20 14:57	PY 文件	1 KB	
setup.py	2018/5/10 14:08	PY 文件	1 KB	

打开 main.py，加入代码（注意：在以下的实验中，所有的实验都拥有该信息）：



Python Code

```
1 from aip import AipFace
2 """ 你的 APPID AK SK """
3 APP_ID = '你的 App ID'
4 API_KEY = '你的 Api Key'
5 SECRET_KEY = '你的 Secret Key'
6 client = AipFace (APP_ID, API_KEY, SECRET_KEY)
```

```
1 from aip import AipFace
2
3 """ 你的 APPID AK SK """
4 APP_ID = '你的 App ID'
5 API_KEY = '你的 Api Key'
6 SECRET_KEY = '你的 Secret Key'
7
8 client = AipFace(APP_ID, API_KEY, SECRET_KEY)
```

APP_ID、API_KEY、SECRET_KEY 这三个内容为申请完毕后记录的三个内容。

III 人脸检测方式

在百度人脸识别 API 上，我们有三种方式上传人脸图片：

```
10 image = "取决于image_type参数, 传入BASE64字符串或URL字符串或FACE_TOKEN字符串"
11 imageType = "image_type参数"
12
13 """ 调用人脸检测 """
14 client.detect(image, imageType)
```

image = "取决于 image_type 参数，传入 BASE64 字符串或 URL 字符串或 FACE_TOKEN 字符串"

imageType = "image_type 参数"

即上传的图像三种方式：BASE64 字符串、URL 字符串、FACE_TOKEN 字符串。

BASE64:图片的 base64 值，base64 编码后的图片数据，需 urlencode，编码后的图片大小不超过 2M；

URL:图片的 URL 地址(可能由于网络等原因导致下载图片时间过长)；

FACE_TOKEN: 人脸图片的唯一标识，调用人脸检测接口时，会为每个人脸图片赋予一个唯一的 FACE_TOKEN，同一张图片多次检测得到的 FACE_TOKEN 是同一个

首先使用 URL 方式进行检测。当然，就想上面的例程一样，除了能直接返回人脸检测结果之外，我们还可以添加参数以及返回更为详细的信息。



```

10 image = "取决于image_type参数, 传入BASE64字符串或URL字符串或FACE_TOKEN字符串"
11 imageType = "image_type参数"
12
13 """ 调用人脸检测 """
14 # client.detect(image, imageType)
15
16 """ 如果有可选参数 """
17 options = {}
18 options["face_field"] = "age"
19 options["max_face_num"] = 2
20 options["face_type"] = "LIVE"
21
22 """ 带参数调用人脸检测 """
23 client.detect(image, imageType, options)
24

```

请求参数

参数	必选	类型	说明
image	是	string	base64 编码后的图片数据, 需 urlencode, 编码后的图片大小不超过 2M
max_face_num	否	uint32	最多处理人脸的数目, 默认值为 1, 仅检测图片中面积最大的那个人脸
face_fields	否	string	包括 age、beauty、expression、faceshape、gender、glasses、landmark、race、qualities 信息, 逗号分隔, 默认只返回人脸框、概率和旋转角度

face_fields 参数, 默认只返回人脸框、概率和旋转角度, age 等更多属性, 请在此参数中添加。

返回参数

参数	类型	必选	说明
log_id	uint64	是	日志 id
result_num	uint32	是	人脸数目
result	object[]	是	人脸属性对象的集合
+age	double	否	年龄。face_fields 包含 age 时返回
+beauty	double	否	美丑打分, 范围 0-100, 越大表示越美。face_fields 包含 beauty 时返回
+location	object	是	人脸在图片中的位置
++left	uint32	是	人脸区域离左边界的距离
++top	uint32	是	人脸区域离上边界的距离
++width	uint32	是	人脸区域的宽度



参数	类型	必选	说明
++height	uint32	是	人脸区域的高度
+face_probability	double	是	人脸置信度，范围 0-1
+rotation_angle	int32	是	人脸框相对于竖直方向的顺时针旋转角，[-180,180]
+yaw	double	是	三维旋转之左右旋转角[-90(左), 90(右)]
+pitch	double	是	三维旋转之俯仰角度[-90(上), 90(下)]
+roll	double	是	平面内旋转角[-180(逆时针), 180(顺时针)]
+expression	uint32	否	表情，0，不笑；1，微笑；2，大笑。face_fields 包含 expression 时返回
+expression_probability	double	否	表情置信度，范围 0~1。face_fields 包含 expression 时返回
+faceshape	object[]	否	脸型置信度。face_fields 包含 faceshape 时返回
++type	string	是	脸型：square/triangle/oval/heart/round
++probability	double	是	置信度：0~1
+gender	string	否	male、female。face_fields 包含 gender 时返回
+gender_probability	double	否	性别置信度，范围[0~1]，face_fields 包含 gender 时返回
+glasses	uint32	否	是否带眼镜，0-无眼镜，1-普通眼镜，2-墨镜。face_fields 包含 glasses 时返回
+glasses_probability	double	否	眼镜置信度，范围[0~1]，face_fields 包含 glasses 时返回
+landmark	object[]	否	4 个关键点位置，左眼中心、右眼中心、鼻尖、嘴中心。face_fields 包含 landmark 时返回
++x	uint32	否	x 坐标
++y	uint32	否	y 坐标
+landmark72	object[]	否	72 个特征点位置，face_fields 包含 landmark 时返回
++x	uint32	否	x 坐标
++y	uint32	否	y 坐标
+race	string	否	yellow、white、black、arabs。face_fields 包含 race 时返回
+race_probability	double	否	人种置信度，范围[0~1]，face_fields 包含 race 时返回
+qualities	object	否	人脸质量信息。face_fields 包含 qualities 时返回



参数	类型	必选	说明
++occlusion	object	是	人脸各部分遮挡的概率，范围[0~1]，0 表示完整，1 表示不完整
+++left_eye	double	是	左眼遮挡比例
+++right_eye	double	是	右眼遮挡比例
+++nose	double	是	鼻子遮挡比例
+++mouth	double	是	嘴巴遮挡比例
+++left_cheek	double	是	左脸颊遮挡比例
+++right_cheek	double	是	右脸颊遮挡比例
+++chin	double	是	下巴遮挡比例
++blur	double	是	人脸模糊程度，范围[0~1]，0 表示清晰，1 表示模糊
++illumination	-	是	取值范围在[0~255],表示脸部区域的光照程度
++completeness	-	是	人脸完整度，0 或 1, 0 为人脸溢出图像边界，1 为人脸都在图像边界内
++type	object	是	真实人脸/卡通人脸置信度
+++human	-	是	真实人脸置信度，[0~1]，大于 0.5 可以判断为人脸
+++cartoon	-	是	卡通人脸置信度，[0~1]

IV 使用 URL 方式进行人脸检测

1、该识别主要用于 URL 方式上传网络照片，从而进行人脸检测，在 main.py 中添加代码如下：

```

1 image = "http://imgtu.5011.net/uploads/content/20170509/3639831494323304.jpg"
2 imageType = "URL"
3
4 """ 调用人脸检测 """
5 # client.detect(image, imageType)
6
7 """ 如果有可选参数 """
8 options = {}
9 options["face_field"] = "age,beauty"
10 options["max_face_num"] = 1
11 options["face_type"] = "LIVE"

```



```
14 """ 带参数调用人脸检测 """
    print (client.detect(image, imageType, options))
```

2、修改完成，添加年龄检测和颜值检测：

```
1 from aip import AipFace
2
3 """ 你的 APPID AK SK """
4 APP_ID = '你的 App ID'
5 API_KEY = '你的 Api Key'
6 SECRET_KEY = '你的 Secret Key'
7
8 client = AipFace(APP_ID, API_KEY, SECRET_KEY)
9
10 image = "http://imgtu.5011.net/uploads/content/20170509/3639831494323304.jpg"
11 imageType = "URL"
12
13 """ 调用人脸检测 """
14 # client.detect(image, imageType)
15
16 """ 如果有可选参数 """
17 options = {}
18 options["face_field"] = "age,beauty"
19 options["max_face_num"] = 1
20 options["face_type"] = "LIVE"
21
22 """ 带参数调用人脸检测 """
23 print (client.detect(image, imageType, options))
24
```

3、执行：

python main.py

4、得到结果：26 岁，75.4 分（可能满分 75 吧）：

```
1 {
    'error_code': 0,
    'error_msg': 'SUCCESS',
    'log_id': 304592830666201311,
    'timestamp': 1553066620,
    'cached': 0,
    'result': {'face_num': 1,
    'face_list': [
        {'face_token': '8c50547c8d3d245fa4d47839f33606ee',
        'location': {'left': 53.75, 'top': 72.19, 'width': 75, 'height': 68, 'rotation': -4},
        'face_probability': 1,
        'angle': {'yaw': 10.14, 'pitch': 24.95, 'roll': -10.52},
        'age': 26,
        'beauty': 75.4}}]}
}
```




V 使用 BASE64 方式进行人脸检测

1、该识别主要用于 BASE64 方式上传本地照片，从而进行人脸检测，在 main.py 中添加代码如下：

Python Code

```
1 """ 选择 BASE64 """
2 filePath = "huge.png"
3 with open(filePath, "rb") as f:
4 # b64encode 是编码
5     base64_data = base64.b64encode(f.read())
6 image = str(base64_data, 'utf-8')
7 imageType = "BASE64"
8
9 """ 调用人脸检测 """
10 # client.detect(image, imageType)
11
12 """ 如果有可选参数 """
13 options = {}
14 options["face_field"] = "age,beauty"
15 options["max_face_num"] = 1
16 options["face_type"] = "LIVE"
17
18 """ 带参数调用人脸检测 """
19 print (client.detect(image, imageType, options))
```

2、修改完成，添加年龄检测和颜值检测：



```

1  from aip import AipFace
2  import base64
3
4  """ 你的 APPID AK SK """
5  APP_ID = '你的 App ID'
6  API_KEY = '你的 Api Key'
7  SECRET_KEY = '你的 Secret Key'
8
9  client = AipFace(APP_ID, API_KEY, SECRET_KEY)
10
11 """ 选择URL """
12 # image = "http://imgtu.5011.net/uploads/content/20170509/3639831494323304.jpg"
13 # imageType = "URL"
14
15 """ 选择BASE64 """
16 filePath = "huge.png"
17 with open(filePath, "rb") as f:
18     # b64encode是编码
19     base64_data = base64.b64encode(f.read())
20     image = str(base64_data, 'utf-8')
21     imageType = "BASE64"
22
23 """ 调用人脸检测 """
24 # client.detect(image, imageType)
25
26 """ 如果有可选参数 """
27 options = {}
28 options["face_field"] = "age,beauty"
29 options["max_face_num"] = 1
30 options["face_type"] = "LIVE"
31
32 """ 带参数调用人脸检测 """
33 print (client.detect(image, imageType, options))
34

```

3、执行：

python main.py

4、得到结果：

```

1 {
    'error_code': 0,
    'error_msg': 'SUCCESS',
    'log_id': 305486830671835851,
    'timestamp': 1553067183,
    'cached': 0,
    'result': {
        'face_num': 1,
        'face_list': [
            {'face_token': 'ecb468f2e58ef53e052141d99d39e149',
             'location':
                {
                    'left': 150.2, 'top': 211.73, 'width': 205, 'height': 183, 'rotation': -4},

```

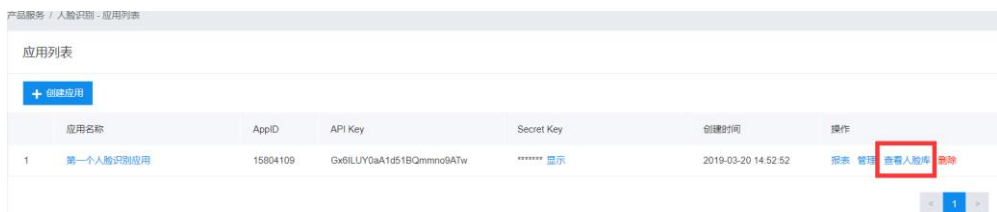


```
'face_probability': 1,
'angle': {'yaw': -3.98, 'pitch': 24.38, 'roll': -6.13},
'age': 25,
'beauty': 84.93}}}]
}
```

VI 添加人脸库

人脸库在百度上，也就是人脸识别想识别谁，就放谁的照片上去就可以了。

先进入人脸库：



点击新建组，输入 ID 新建。

新建用户组

×

组ID:

由数字、字母和下划线组成

0/48

确认

取消

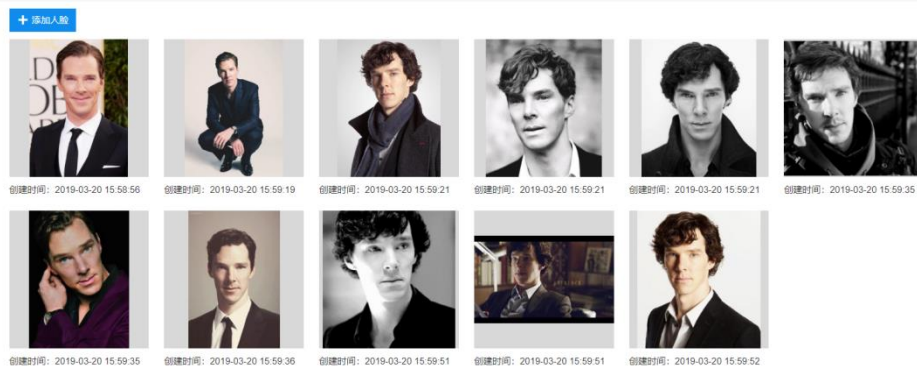
我们新建一个神探夏洛克（Sherlock）

用户组	用户数	修改时间	详情
1 Sherlock	0	2019-03-20 15:44:25	0张人脸

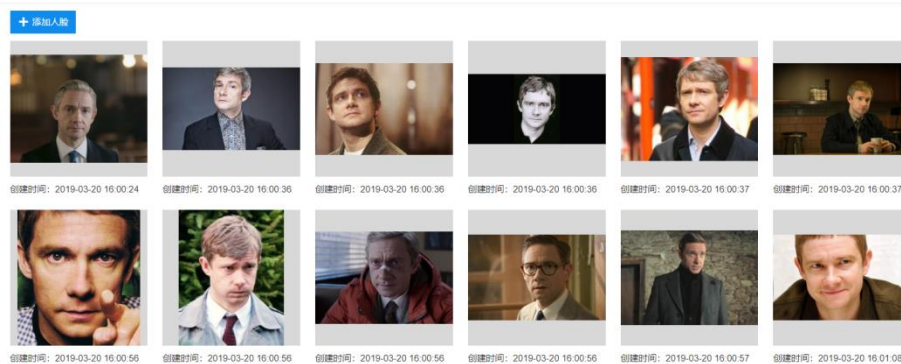
点击进去，然后新建用户，这里我们新建了两个用户，卷福（Benedict Cumberbatch）和花生（Martin Freeman），每一个用户都上传了十几张照片上去。



第一个人脸识别应用 / Sherlock / BenedictCumberbatch



第一个人脸识别应用 / Sherlock / MartinFreeman



VII 开始识别

1、我们用户组 ID 名叫做“Sherlock”，在编程时我们只需向 groupIdList 参数中传入“Sherlock”即可。新建 main_detection.py，并加入代码：

Python Code

```
1 from aip import AipFace
2 import base64
3
4 """ 你的 APPID AK SK """
5 APP_ID = '你的 App ID'
6 API_KEY = '你的 Api Key'
7 SECRET_KEY = '你的 Secret Key'
8
9 client = AipFace(APP_ID, API_KEY, SECRET_KEY)
10
11 """ 选择BASE64 """
12 filePath ="huge.png"
13 with open(filePath,"rb") as f:
14 # b64encode 是编码
15     base64_data = base64.b64encode(f.read())
16 image = str(base64_data,'utf-8')
```



```

17
18 imageType = "BASE64"
19
20 groupIdList = "Sherlock"
21
22 """ 调用人脸搜索 """
23 print (client.search(image, imageType, groupIdList))
24
25

```

2、修改完成:

```

1  from aip import AipFace
2  import base64
3
4  """ 你的 APPID AK SK """
5  APP_ID = '你的 App ID'
6  API_KEY = '你的 Api Key'
7  SECRET_KEY = '你的 Secret Key'
8
9  client = AipFace(APP_ID, API_KEY, SECRET_KEY)
10
11 """ 选择BASE64 """
12 filePath ="huge.png"
13 with open(filePath,"rb") as f:
14     # b64encode是编码
15     base64_data = base64.b64encode(f.read())
16     image = str(base64_data,'utf-8')
17
18     imageType = "BASE64"
19
20     groupIdList = "Sherlock"
21
22 """ 调用人脸搜索 """
23 print (client.search(image, imageType, groupIdList))

```

3、执行:

python main_detection.py

4、得到结果:

```

1 {
  'error_code': 0,
  'error_msg': 'SUCCESS',
  'log_id': 305486830694867641,
  'timestamp': 1553069486,
  'cached': 0,

```



```
'result': {
  'face_token': 'ecb468f2e58ef53e052141d99d39e149',
  'user_list': [{
    'group_id': 'Sherlock',
    'user_id': 'BenedictCumberbatch',
    'user_info': '',
    'score': 30.385366439819}]]}
}
```

这里我们还是拿胡歌的照片做识别，发现与卷福更为相近，但是'score'，即相似度只有 30 左右。接下来我们把图片换成卷福的试试。代码：

```
10
11 """ 选择BASE64 """
12 filePath ="Benedict.png"
```

结果：

```
1 {'error_code': 0,
  'error_msg': 'SUCCESS',
  'log_id': 304569230698551801,
  'timestamp': 1553069855,
  'cached': 0,
  'result': {
    'face_token': '87dc6189f0c53af307c44606015b25b5',
    'user_list': [{
      'group_id': 'Sherlock',
      'user_id': 'BenedictCumberbatch',
      'user_info': '',
      'score': 96.727256774902}]]}
}
```

可以看出，'score'为 96 接近卷福。

再换成花生的试试：

```
10
11 """ 选择BASE64 """
12 filePath ="Martin.png"
13 with open(filePath,"rb") as f:
```



结果:

```
1 {'error_code': 0,
  'error_msg': 'SUCCESS',
  'log_id': 304592830699718771,
  'timestamp': 1553069971,
  'cached': 0,
  'result': {
    'face_token': 'f7f45bcd55a97f542535289a0e4cab72',
    'user_list': [{
      'group_id': 'Sherlock',
      'user_id': 'MartinFreeman',
      'user_info': '',
      'score': 93.161392211914}]}
}
```

可以看出, 'score'为 93 接近花生。

VIII改进返回值

之前我们进行检测或者进行识别都是直接返回的结果信息(字典),我们这次利用这个结果进行一下人脸的检测以及性别的检测。我们还是使用我们的人脸检测代码(BASE64)的进行修改。

我们这次试用 OpenCV 打开图片并转为 base64,但是前提要导入 OpenCV 的包哦。

```
3 import cv2
```

```
12 img1 = cv2.imread("Sherlock.png")
13 img = cv2.imencode('.jpg',img1)[1]
14 image = str(base64.b64encode(img))[2:-1]
15 imageType = "BASE64"
16
```

首先,可选参数向我们选择 age、beauty、gender 三项进行检测,脸部数量最大的值设定为 9。

```
19 """ 如果有可选参数 """
20 options = {}
21 options["face_field"] = "age,beauty,gender"
22 options["max_face_num"] = 9
23 options["face_type"] = "LIVE"
```



之后我们进行检测

```
27 result = client.detect(image, imageType, options)
```

我们输出这个 result 发现，一旦成功检测到，那他会返回 {'error_code': 0, 'error_msg': 'SUCCESS', '****'}

那么我们认为 error_msg 为 'SUCCESS' 的时候，检测成功，我们在这里进行处理。

```
if result['error_msg'] == 'SUCCESS':
```

接下来我们想：可能照片中只有一个人，也可能有好几个人，我们这里最大设置为 9，所以还需要一个循环来循环画出脸部位置以及信息。

```
32 for i in range(result['result']['face_num']):
```

然后我们需要拿到不同人脸的相应位置。

```
33 x = result['result']['face_list'][i]['location']['left']
34 y = result['result']['face_list'][i]['location']['top']
35 w = result['result']['face_list'][i]['location']['width']
36 h = result['result']['face_list'][i]['location']['height']#
```

然后我们利用 opencv 在图中画出人脸，并且将信息显示出来。

```
37 cv2.rectangle(img1, (int(x), int(y)), (int(x+w), int(y+h)), (0, 255, 0), 2)
38 cv2.putText(img1, "beauty:" + str(result['result']['face_list'][i]['beauty']), (int(x), int(y+h)+15), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,0,255), 2)
39 cv2.putText(img1, "age:" + str(result['result']['face_list'][i]['age']), (int(x), int(y+h)+35), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,0,255), 2)
40 cv2.putText(img1, "gender:" + str(result['result']['face_list'][i]['gender']['type']), (int(x), int(y+h)+55), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,0,255), 2)
```

之后我们显示照片并且让他一直在，除非我们自己关闭。

```
41 cv2.imshow('Face Recognition', img1)
42 cv2.waitKey(0)
```

那么如果没检测到，我们做个打印容错。

```
43 else:
44     print("no answer")
```

这样我们就结束了这次改进。

整体源码：

Python Code

```
1 from aip import AipFace
2 import base64
3 import cv2
4
5 """ 你的 APPID AK SK """
6 APP_ID = '你的 App ID'
```




```

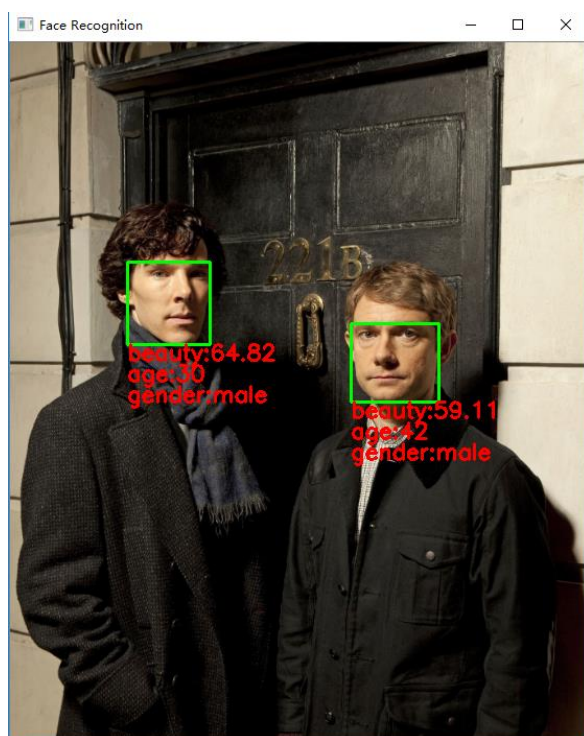
7 API_KEY = '你的 Api Key'
8 SECRET_KEY = '你的 Secret Key'
9
10 client = AipFace(APP_ID, API_KEY, SECRET_KEY)
11
12 img1 = cv2.imread("Sherlock.png")
13 img = cv2.imencode('.jpg',img1)[1]
14 image = str(base64.b64encode(img))[2:-1]
15 imageType = "BASE64"
16
17 """ 如果有可选参数 """
18 options = {}
19 options["face_field"] = "age,beauty,gender"
20 options["max_face_num"] = 9
21 options["face_type"] = "LIVE"
22
23 result = client.detect(image, imageType, options)
24 print (result)
25 print (result['result']['face_num'])
26 type(result['result']['face_num'])
27 if result['error_msg']=='SUCCESS':
28     for i in range(result['result']['face_num']):
29         x = result['result']['face_list'][i]['location']['left']
30         y = result['result']['face_list'][i]['location']['top']
31         w = result['result']['face_list'][i]['location']['width']
32         h = result['result']['face_list'][i]['location']['height']#
33         cv2.rectangle(img1, (int(x), int(y)), (int(x+w), int(y+h)), (0, 255, 0), 2)
34         cv2.putText(img1, "beauty:" +str(result['result']['face_list'][i]['beauty']), \
35             (int(x), int(y+h)+15), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,0,255), 2)
36         cv2.putText(img1, "age:" +str(result['result']['face_list'][i]['age']), \
37             (int(x), int(y+h)+35), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,0,255), 2)
38         cv2.putText(img1, "gender:" +str(result['result']['face_list'][i]['gender']['type']), \
39             (int(x), int(y+h)+55), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,0,255), 2)
40     cv2.imshow('Face Recognition', img1)
41     cv2.waitKey(0)
42 else:
43     print ("no answer")

```

然后我们直接执行

python main.py

得到结果:



6.2 语音识别

6.2.1 实验目的

通过调用百度 AI 开放平台的接口，实现语音识别项目。

6.2.2 实验原理

百度短语音识别可以将 60 秒以下的音频识别为文字。适用于语音对话、语音控制、语音输入等场景。

接口类型：通过 RESTAPI 的方式提供的通用的 HTTP 接口。适用于任意操作系统，任意编程语言

接口限制：需要上传完整的录音文件，录音文件时长不超过 60 秒。浏览器由于无法跨域请求百度语音服务器的域名，因此无法直接调用 API 接口。

支持音频格式：pcm、wav、amr、m4a

音频编码要求：采样率 16000、8000，16 bit 位深，单声道。

6.2.3 实验步骤

本节实验代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种人工智能实验



I 申请步骤

1、首先登陆 <http://ai.baidu.com/tech/speech/asr>，进入到通用语音识别的界面。选择立即使用。



2、登陆自己的百度账号，可用手机号申请。

百度账号 云账号(推广账号)

短信快捷登录

登录

立即注册 忘记密码

温馨提示：
与百度搜索、百度贴吧、百度云盘、百度知道、百度文库
等产品通用。

3、选择创建应用



4、依次输入应用名称，选择类型，接口默认选择语音识别，添加应用描述，点击立即创建。

* 应用名称: 第一个语音识别应用

* 应用类型: 游戏娱乐

* 接口选择: 勾选以下接口，使此应用可以请求已勾选的接口服务，注意百度语音服务已默认勾选并不可取消

☒ 百度语音 ☒ 语音识别 ☒ 语音合成

☐ 文字识别

☐ 人脸识别

☐ 自然语言处理

☐ 内容审核 1

☐ UNIT 1

☐ 知识图谱

☐ 图像识别

☐ 智能呼叫中心

☐ 图像搜索

☐ 人体分析

☐ 图像处理

* 语音包名: ☐ iOS ☐ Android ☒ 不需要

* 应用描述: 第一个语音识别应用

7、至此，应用创建完毕，现在就可以使用该应用了。

创建完毕

[返回应用列表](#)

[查看应用详情](#)

[查看文档](#)

[下载SDK](#)



8、下载 SDK，下载 Python 的 SDK（当前版本：2.0.0）。

离线融合SDK		3.0.10 1. 添加UNIT 2.0 语义解析功能。 U...	使用说明	下载
Linux SDK		3.0.5 1. 修正长语音识别的部分问题	使用说明	下载
离线融合SDK		3.0.5 增加模拟器支持 NEW	使用说明	下载
识别、合成 RESTful API JAVA SDK		4.1.1 语音合成问题修复 NEW	使用说明	下载
识别、合成 RESTful API PHP SDK		1.6.0 新增功能	使用说明	下载
识别、合成 RESTful API Python SDK		2.0.0 修复问题	使用说明	下载
识别、合成 RESTful API C++ SDK		0.7.4 修复一些问题	使用说明	下载

7、创建完毕界面，点击返回应用列表，将 AppID、API Key、Secret Key 记下。

应用列表

+ 创建应用						
应用名称	AppID	API Key	Secret Key	创建时间	操作	
1 第一个语音识别应用	15803120	1oOPaqrXyIQD5NBzVrUy49	***** 显示	2019-03-20 13:45:37	报表 管理 删除	

< 1 >

II 调用 API 实现识别

解压下载好的 SDK，查看目录结构。下载下来的 SDK 在 aip 目录下还有其他文件，用于做其他识别，图像识别只用到这几个文件。

```

├── README.md
├── aip                                //SDK目录
│   ├── __init__.py                 //导出类
│   ├── base.py                     //aip基类
│   ├── http.py                     //http请求
│   └── speech.py                   //语音识别
└── setup.py                         //setuptools安装
    
```

使用 pip 安装 baidu-aip 的包。

```
pip install baidu-aip
```

接下来我们开始实现语音识别。

AipSpeech 是语音识别的 Python SDK 客户端，为使用图像识别的开发人员提供了一系列的交互方法。我们在解压的目录下新建一个 main.py。



baidu_api_voice				
名称	修改日期	类型	大小	
aip	2017/12/28 13:45	文件夹		
bin	2017/12/28 13:45	文件夹		
LICENSE	2017/12/28 13:45	文件	12 KB	
setup.py	2017/12/28 13:45	PY 文件	1 KB	
main.py	2019/3/20 13:54	PY 文件	0 KB	

打开 main.py，加入代码（注意：在以下的实验中，所有的实验都拥有该信息）：

Python Code

```
1 from aip import AipSpeech
2 """ 你的 APPID AK SK """
3 APP_ID = '你的 App ID'
4 API_KEY = '你的 Api Key'
5 SECRET_KEY = '你的 Secret Key'
6 client = AipSpeech (APP_ID, API_KEY, SECRET_KEY)
7
```

```
1 from aip import AipSpeech
2
3 """ 你的 APPID AK SK """
4 APP_ID = '你的 App ID'
5 API_KEY = '你的 Api Key'
6 SECRET_KEY = '你的 Secret Key'
7
8 client = AipSpeech(APP_ID, API_KEY, SECRET_KEY)
```

APP_ID、API_KEY、SECRET_KEY 这三个内容为申请完毕后记录的三个内容。

III 语音识别

1、该识别主要用于语音识别，语音时长上限为 60s，在 main.py 中添加代码如下：

Python Code

```
1 # 读取文件
2 def get_file_content(filePath):
3     with open(filePath, 'rb') as fp:
4         return fp.read()
5
6 # 识别本地文件
7 print (client.asr(get_file_content("test.wav"), 'wav', 16000, {'dev_pid': 1536,}))
```

2、修改完成：



```
1 from aip import AipSpeech
2
3 """ 你的 APPID AK SK """
4 APP_ID = '你的 App ID'
5 API_KEY = '你的 Api Key'
6 SECRET_KEY = '你的 Secret Key'
7
8 client = AipSpeech(APP_ID, API_KEY, SECRET_KEY)
9
10 # 读取文件
11 def get_file_content(filePath):
12     with open(filePath, 'rb') as fp:
13         return fp.read()
14
15 # 识别本地文件
16 print (client.asr(get_file_content("test.wav"), 'wav', 16000, {'dev_pid': 1536,}))
17
```

3、执行:

python main.py

4、得到结果:

```
1 {
    'corpus_no': '6670351178587786449',
    'err_msg': 'success.',
    'err_no': 0,
    'result': ['今天天气很好'],
    'sn': '228403153301553062158'
}
```

dev_pid 参数列表

dev_pid	语言	模型	是否有标点	备注
1536	普通话(支持简单的英文识别)	搜索模型	无标点	支持自定义词库
1537	普通话(纯中文识别)	输入法模型	有标点	支持自定义词库
1737	英语		有标点	不支持自定义词库
1637	粤语		有标点	不支持自定义词库
1837	四川话		有标点	不支持自定义词库
1936	普通话远场	远场模型	有标点	不支持

语音识别-返回数据参数详情



参数	类型	是否一定输出	描述
err_no	int	是	错误码
err_msg	int	是	错误码描述
sn	int	是	语音数据唯一标识，系统内部产生，用于 debug
result	int	是	识别结果数组，提供 1-5 个候选结果，string 类型为识别的字符串，utf-8 编码

6.3 通用物体和场景识别

6.3.1 实验目的

通过调用百度 AI 开放平台的接口，实现通用物体和场景识别项目。

6.3.2 实验原理

I 接口描述

该请求用于通用物体及场景识别，即对于输入的一张图片（可正常解码，且长宽比适宜），输出图片中的多个物体及场景标签。

II 请求说明

请求示例

HTTP 方法：POST

请求 URL：https://aip.baidubce.com/rest/2.0/image-classify/v2/advanced_general

URL 参数：

参数	值
access_token	通过 API Key 和 Secret Key 获取的 access_token,参考“Access Token 获取”

Header 如下：

参数	值
Content-Type	application/x-www-form-urlencoded

Body 中放置请求参数，参数详情如下：

请求参数



参数	是否必选	类型	可选值范围	说明
image	true	string	-	图像数据，base64 编码，要求 base64 编码后大小不超过 4M，最短边至少 15px，最长边最大 4096px,支持 jpg/png/bmp 格式。 注意：图片需要 base64 编码、去掉编码头（data:image/jpg;base64,）后，再进行 urlencode。
baike_num	否	integer	0	返回百科信息的结果数，默认不返回

6.3.3 实验步骤

本节实验代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种人工智能实验

1、该识别主要用于通用物体以及通用场景的识别，主要是针对输入的图片而言（由于每天有免费次数的限制，不建议使用摄像头捕捉循环识别），图片与 main.py 放在同一目录下。



main.py



pic.jpg

2、在 main.py 下加入代码：

```

1  from aip import AipImageClassify
2
3  """ 你的 APPID AK SK """
4  APP_ID = ''
5  API_KEY = ''
6  SECRET_KEY = ''
7
8  client = AipImageClassify(APP_ID, API_KEY, SECRET_KEY)
9
10 """ 读取图片 """
11 def get_file_content(filePath):
12     with open(filePath, 'rb') as fp:
13         return fp.read()
14
15 image = get_file_content('pic.jpg')
16
17 """ 如果有可选参数 """
18 options = {}
19 options["baike_num"] = 1
20
21 """ 带参数调用通用物体和场景识别 """
22 print (client.advancedGeneral(image, options))

```

3、执行：

python main.py



4、得到结果:

```
1 {'result_num': 5, 'log_id': 4966932872521916972, 'result': [{'root': '建筑-现代建筑',
2 'baike_info': {'baike_url': 'http://baike.baidu.com/item/%E5%BB%BA%E7%AD%91/405099',
3 'description': '指人工建筑而成的资产，属于固定资产范畴，包括房屋和构筑物两大类。房屋是指供人居住、工
4 作、学习、生产、经营、娱乐、储藏物品以及进行其他社会活动的工程建筑。与建筑物有区别的是构筑物，构筑物指房
5 屋以外的工程建筑，如围墙、道路、水坝、水井、隧道、水塔、桥梁和烟囱等。', 'image_url':
6 'https://bking.cdn.bcebos.com/pic/6159252dd42a283431973c7454b5c9ea15cebf9f'}, 'keyword': '建筑',
7 'score': 0.868034}, {'root': '建筑-建筑夜景', 'keyword': '上海外滩', 'score': 0.686723}, {'root':
8 '自然风景-天空', 'keyword': '天空', 'score': 0.416487}, {'root': '建筑-其他', 'keyword': '街道',
9 'score': 0.226844}, {'root': '建筑-建筑夜景', 'keyword': '夜景照明', 'score': 0.030591}]}
```

通用物体识别，请求参数详情

参数名称	是否必选	类型	默认值	说明
image	是	string		图像数据，base64 编码，要求 base64 编码后大小不超过 4M，最短边至少 15px，最长边最大 4096px,支持 jpg/png/bmp 格式
baike_num	否	string	0	返回百科信息的结果数，默认不返回

通用物体识别-返回数据参数详情

字段	是否必选	类型	说明
log_id	是	uint64	唯一的 log id，用于问题定位
result_num	是	uint32	返回结果数目，及 result 数组中的元素个数
result	是	array(object)	标签结果数组
+keyword	是	string	图片中的物体或场景名称
+score	是	float	置信度，0-1
+root	是	string	识别结果的上层标签，有部分钱币、动漫、烟酒等 tag 无上层标签
+baike_info	object	否	对应识别结果的百科词条名称
++baike_url	string	否	对应识别结果百度百科页面链接
++image_url	string	否	对应识别结果百科图片链接
++description	string	否	对应识别结果百科内容描述



6.4 蔬菜识别

6.4.1 实验目的

通过调用百度 AI 开放平台的接口，实现蔬菜识别项目。

6.4.2 实验原理

I 接口描述

该请求用于识别果蔬类食材，即对于输入的一张图片（可正常解码，且长宽比适宜），输出图片中的果蔬食材结果。

II 请求说明

HTTP 方法：POST

请求 URL： <https://aip.baidubce.com/rest/2.0/image-classify/v1/classify/ingredient>

6.4.3 实验步骤

本节实验代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种人工智能实验

1、该识别主要用于蔬菜识别，主要是针对输入的图片而言（由于每天有免费次数的限制，不建议使用摄像头捕捉循环识别），图片与 main.py 放在同一目录下。



2、在 main.py 下加入代码：



```
1 from aip import AipImageClassify
2
3 """ 你的 APPID AK SK """
4 APP_ID = ''
5 API_KEY = ''
6 SECRET_KEY = ''
7
8 client = AipImageClassify(APP_ID, API_KEY, SECRET_KEY)
9
10 """ 读取图片 """
11 def get_file_content(filePath):
12     with open(filePath, 'rb') as fp:
13         return fp.read()
14
15 image = get_file_content('pic.jpg')
16
17 """ 如果有可选参数 """
18 options = {}
19 options["baike_num"] = 1
20
21 """ 带参数调用蔬菜识别 """
22 print (client.ingredient(image, options))
```

3、执行:

python main.py

4、得到结果:

```
1 {'result': [{'score': 0.910444974899292, 'name': '大白菜'}, {'score': 0.05064169317483902,
2 'name': '白菜'}, {'score': 0.024156631901860237, 'name': '小白菜'}, {'score':
3 0.01419985480606556, 'name': '娃娃菜'}, {'score': 0.00024176544684451073, 'name': '生菜'}],
4 'result_num': 5, 'log_id': 1816837886997887436}
```

6.5 动物识别

6.5.1 实验目的

通过调用百度 AI 开放平台的接口，实现动物识别项目。

6.5.2 实验原理

I 接口描述

该请求用于识别一张图片，即对于输入的一张图片（可正常解码，且长宽比较合适），输出动物识别结果。

II 请求说明

HTTP 方法: POST

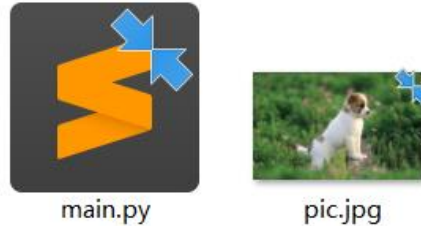


请求 URL: <https://aip.baidubce.com/rest/2.0/image-classify/v1/animal>

6.5.3 实验步骤

本节实验代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种人工智能实验

1、该识别主要用于动物识别,主要是针对输入的图片而言(由于每天有免费次数的限制,不建议使用摄像头捕捉循环识别),图片与 main.py 放在同一目录下。



2、在 main.py 下加入代码:

```
from aip import AipImageClassify

""" 你的 APPID AK SK """
APP_ID = ''
API_KEY = ''
SECRET_KEY = ''

client = AipImageClassify(APP_ID, API_KEY, SECRET_KEY)

""" 读取图片 """
def get_file_content(filePath):
    with open(filePath, 'rb') as fp:
        return fp.read()

image = get_file_content('pic.jpg')

""" 如果有可选参数 """
options = {}
options["baike_num"] = 1

""" 带参数调用动物识别 """
print (client.animalDetect(image, options))
```

3、执行:

python main.py

4、得到结果:

```
1{'log_id': 9095108377896366956, 'result': [{'name': '荷兰牧羊犬', 'baike_info': {}, 'score':
2'0.219801'}, {'name': '杰克罗素梗', 'score': '0.196446'}, {'name': '库达犬', 'score': '0.14319'},
3{'name': '蝴蝶犬', 'score': '0.11156'}, {'name': '比利牛斯獒犬', 'score': '0.0667426'}, {'name':
'边境牧羊犬', 'score': '0.0461472'}]}
```



6.6 植物识别

6.6.1 实验目的

通过调用百度 AI 开放平台的接口，实现植物识别项目。

6.6.2 实验原理

I 接口描述

该请求用于识别一张图片，即对于输入的一张图片（可正常解码，且长宽比较合适），输出植物识别结果。

II 请求说明

HTTP 方法：POST

请求 URL： <https://aip.baidubce.com/rest/2.0/image-classify/v1/plant>

6.6.3 实验步骤

本节实验代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种人工智能实验

1、该识别主要用于植物识别，主要是针对输入的图片而言（由于每天有免费次数的限制，不建议使用摄像头捕捉循环识别），图片与 main.py 放在同一目录下。



2、在 main.py 下加入代码：



```

1  from aip import AipImageClassify
2
3  """ 你的 APPID AK SK """
4  APP_ID = ''
5  API_KEY = ''
6  SECRET_KEY = ''
7
8  client = AipImageClassify(APP_ID, API_KEY, SECRET_KEY)
9
10 """ 读取图片 """
11 def get_file_content(filePath):
12     with open(filePath, 'rb') as fp:
13         return fp.read()
14
15 image = get_file_content('yumi.png')
16
17 """ 调用植物识别 """
18 # print (client.plantDetect(image))
19
20 """ 如果有可选参数 """
21 options = {}
22 options["baike_num"] = 1
23
24 """ 带参数调用植物识别 """
25 print (client.plantDetect(image, options))

```

3、执行:

python main.py

4、得到结果:

```

1 {'result': [{ 'baike_info': { 'baike_url':
2 'http://baike.baidu.com/item/%E7%8E%89%E8%9C%80%E9%BB%8D/1855461', 'image_url':
3 'https://bking.cdn.bcebos.com/pic/d6ca7bcb0a46f21fe189b45af0246b600c33ae7e', 'description': '玉
4 蜀黍，中药名。为禾本科玉蜀黍属植物玉蜀黍 Zea mays L.的种子。遍及全国。具有调中开胃，利尿消肿之功效。常用于
   食欲不振，小便不利，水肿，尿路结石。'}, 'name': '玉蜀黍', 'score': 0.7373589873313904}, { 'name':
   '玉米', 'score': 0.014158247970044613}, { 'name': '高粱', 'score': 0.005641318392008543}],
   'log_id': 5578962653828935052}

```

6.7 果蔬识别

6.7.1 实验目的

通过调用百度 AI 开放平台的接口，实现果蔬识别项目。

6.7.2 实验原理

I 接口描述

该请求用于识别果蔬类食材，即对于输入的一张图片（可正常解码，且长宽比适宜），输出图片中的果蔬食材结果。



II 请求说明

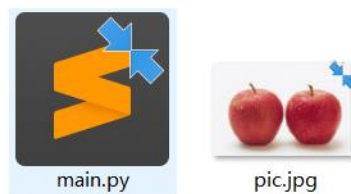
HTTP 方法: POST

请求 URL: <https://aip.baidubce.com/rest/2.0/image-classify/v1/classify/ingredient>

6.7.3 实验步骤

本节实验代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种人工智能实验

1、该识别主要用于果蔬识别,主要是针对输入的图片而言(由于每天有免费次数的限制,不建议使用摄像头捕捉循环识别),图片与 main.py 放在同一目录下。



2、在 main.py 下加入代码:

```
1 from aip import AipImageClassify
2
3 """ 你的 APPID AK SK """
4 APP_ID = '15028993'
5 API_KEY = 'G8G8mUpngdQPPVxNe3dfed08'
6 SECRET_KEY = 'RruWmpD5ck5nVHi714UzEkVXWf1wyIk4'
7
8 client = AipImageClassify(APP_ID, API_KEY, SECRET_KEY)
9
10 """ 读取图片 """
11 def get_file_content(filePath):
12     with open(filePath, 'rb') as fp:
13         return fp.read()
14
15 image = get_file_content('pic.jpg')
16
17 """ 如果有可选参数 """
18 options = {}
19 options["baike_num"] = 1
20
21 """ 带参数调用果蔬识别 """
22 print (client.ingredient(image, options))
23
```

3、执行:

python main.py

4、得到结果:



```
1 {'log_id': 6145303806463895628, 'result_num': 5, 'result': [{'score': 0.8844612240791321,
2 'name': '苹果'}, {'score': 0.054053883999586105, 'name': '蛇果'}, {'score':
3 0.045061103999614716, 'name': '翠伏梨'}, {'score': 0.0055670770816504955, 'name': '非果蔬食材'},
4 {'score': 0.003817841410636902, 'name': '梨'}]}
```

6.8 图像主体检测

6.8.1 实验目的

通过调用百度 AI 开放平台的接口，实现图像主体检测项目。

6.8.2 实验原理

I 接口描述

用户向服务请求检测图像中的主体位置。

II 请求示例

HTTP 方法: POST

请求 URL: https://aip.baidubce.com/rest/2.0/image-classify/v1/object_detect

6.8.3 实验步骤

本节实验代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的各种人工智能实验

1、该识别主要用于图像主体识别，主要是针对输入的图片而言（由于每天有免费次数的限制，不建议使用摄像头捕捉循环识别），图片与 main.py 放在同一目录下。



main.py



pic.jpg

2、在 main.py 下加入代码：



```
1 from aip import AipImageClassify
2
3 """ 你的 APPID AK SK """
4 APP_ID = ''
5 API_KEY = ''
6 SECRET_KEY = ''
7
8 client = AipImageClassify(APP_ID, API_KEY, SECRET_KEY)
9
10 """ 读取图片 """
11 def get_file_content(filePath):
12     with open(filePath, 'rb') as fp:
13         return fp.read()
14
15 image = get_file_content('pic.jpg')
16
17 """ 如果有可选参数 """
18 options = {}
19 options["baike_num"] = 1
20
21 """ 带参数调用图像主体识别 """
22 print (client.objectDetect(image, options))
```

3、执行：

python main.py

4、得到结果：

```
1 {'result': {'height': 465, 'left': 449, 'top': 339, 'width': 455}, 'log_id':
2 3490380876388379308}
```

6.9 地标识别

6.9.1 实验目的

通过调用百度 AI 开放平台的接口，实现地标识别项目。

6.9.2 实验原理

I 接口描述

该请求用于识别地标，即对于输入的一张图片（可正常解码，且长宽比适宜），输出图片中的地标识别结果。

II 请求示例

HTTP 方法：POST

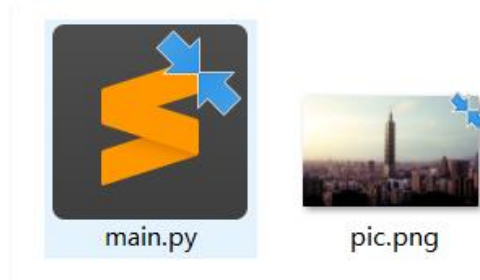
请求 URL： <https://aip.baidubce.com/rest/2.0/image-classify/v1/landmark>



6.9.3 实验步骤

本节实验代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于Python 的各种人工智能实验

1、该识别主要用于地标识别，主要是针对输入的图片而言（由于每天有免费次数的限制，不建议使用摄像头捕捉循环识别），图片与 main.py 放在同一目录下。



2、在 main.py 下加入代码：

```
1 from aip import AipImageClassify
2
3 """ 你的 APPID AK SK """
4 APP_ID = '15028993'
5 API_KEY = 'G8G8mUpngdQPPVxNe3dfed08'
6 SECRET_KEY = 'RruWmpD5cK5nVHi714UzEkVXWf1wyIk4'
7
8 client = AipImageClassify(APP_ID, API_KEY, SECRET_KEY)
9
10 """ 读取图片 """
11 def get_file_content(filePath):
12     with open(filePath, 'rb') as fp:
13         return fp.read()
14
15 image = get_file_content('pic.png')
16
17 """ 如果有可选参数 """
18 options = {}
19 options["baike_num"] = 1
20
21 """ 带参数调用地标识别 """
22 print (client.landmark(image, options))
```

3、执行：

```
python main.py
```

4、得到结果：

```
1 {'result': {'landmark': '台北 101'}, 'log_id': 7245230094854273036}
```



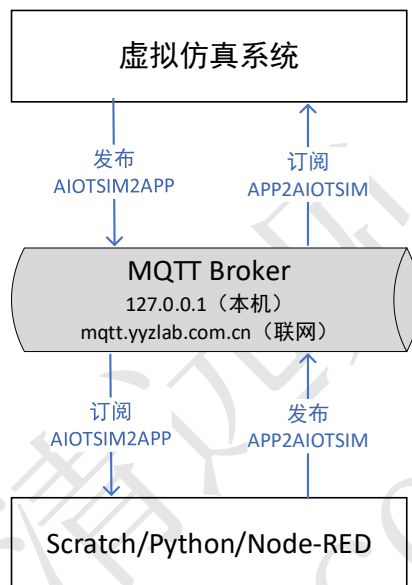
第 7 章 基于 Python 的智能系统实验

7.1 智能火焰报警系统

7.1.1 实验目的

通过 Python 获取物联网虚拟仿真系统里的火焰传感器数据，并根据数据自动控制声光报警器。

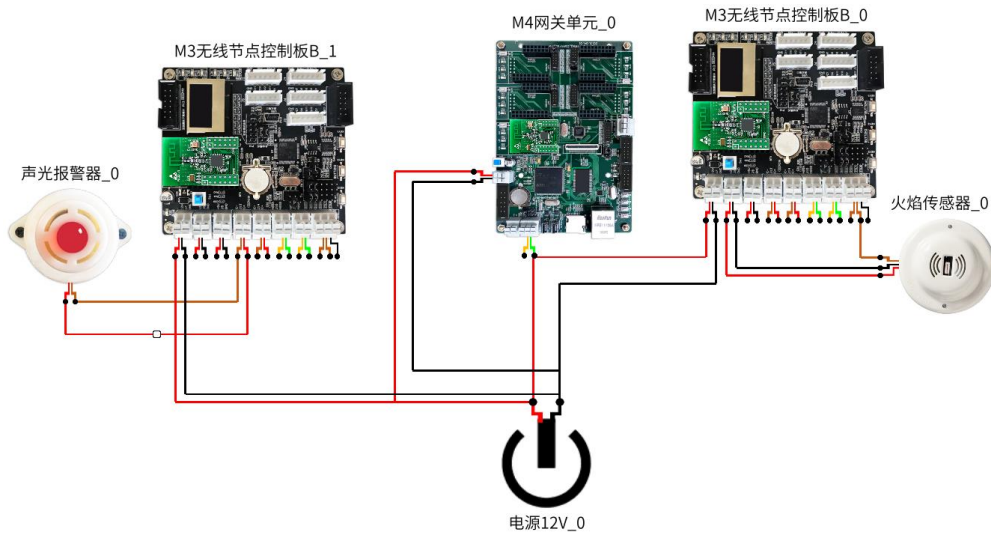
7.1.2 实验原理



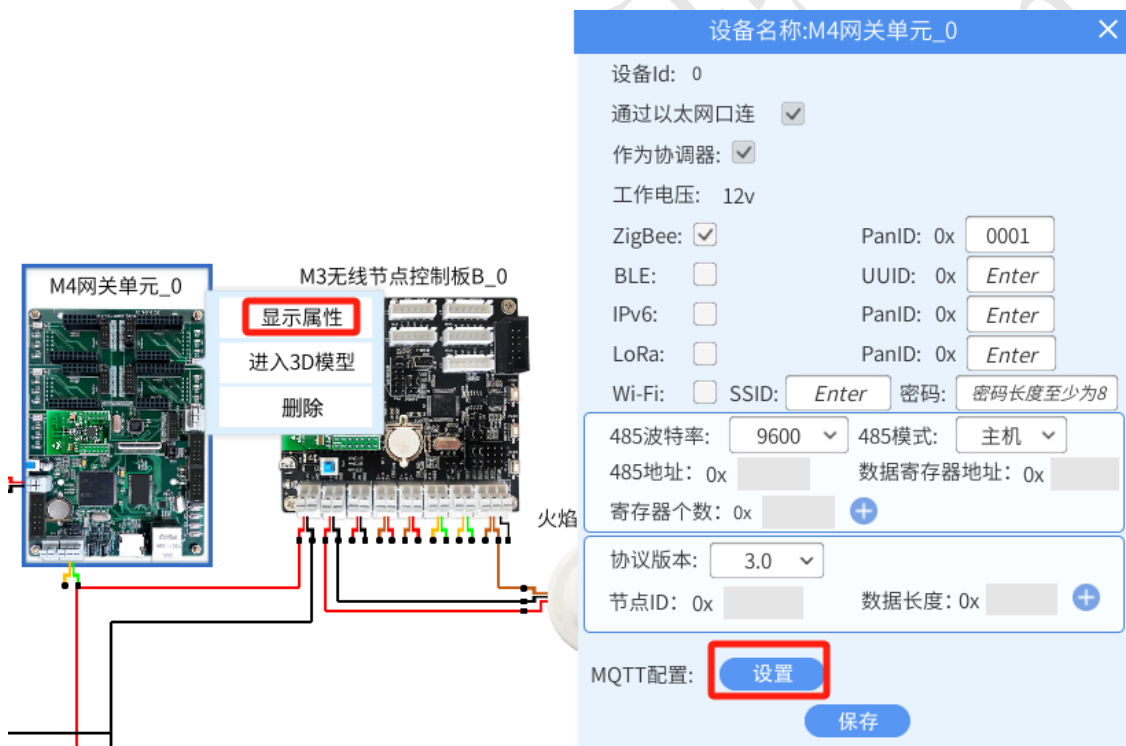
在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

7.1.3 实验步骤

用物联网虚拟仿真系统打开 01- 资料光盘镜像 \01- 程序源码 \Python 源码 \python_and_aiotsim\基于 Python 的智能系统实验下的对应文件。



左键点击显示属性，在弹出的页面中点击设置。



在弹出的 MQTT 配置页面中，设置 MQTT broker 的 IP 地址、端口、话题等，点击连接和订阅。然后关闭窗口后保存设置。



MQTT配置

MQTT:

IP地址: 127.0.0.1 端口: 1883

ClientID: 请输入或者生成ClientID 生成

用户名: 请输入用户名

密码: 请输入密码

断开

与应用层、云端交互的Topic:

订阅: APP2AIOTSIM 复制

发布: AIOTSIM2APP 复制

生成Topic 取消订阅

与硬件交互的Topic:

点击数据列表，打开数据。

数据列表

关闭数据 打开数据

火焰传感器_0数据

数据来源: 打开数据 虚拟设备 实际硬件

曲线名称: 定值曲线

曲线类型: 定值

数据周期: 10 修改

定值曲线/name 火焰传感器_0数据模拟器

1

火焰

然后点击上方的运行按钮，即可让数据自动产生并向 MQTT 发送数据。

在 Python 中打开 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验\code 下的智能火焰报警系统_scratch.py。

```

1  # -*- coding:utf-8 -*-
2  import paho.mqtt.client as mqtt
3  from queue import Queue
4  import json
5  import base64
6  import requests
7  import time
8
9  class HQYJMQTTClient:

```

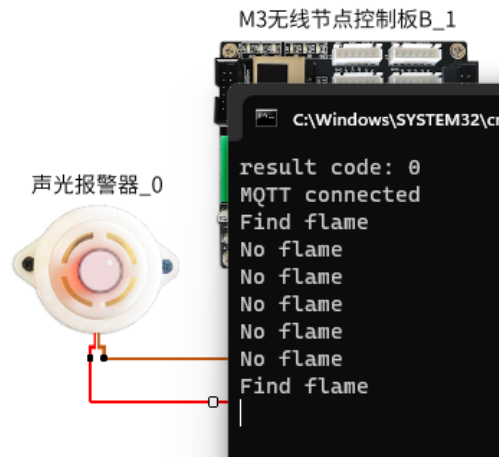


```

10 def __init__(self, broker_ip: str, broker_port: int):
11     self.mqtt_queue = Queue(255)
12     self.is_connected = False
13     self.client = mqtt.Client()
14     self.client.on_message = self.on_message
15     self.client.on_connect = self.on_connect
16     self.rc = 100
17     try:
18         self.client.connect(broker_ip, broker_port, 3)
19     except Exception as e:
20         print(e)
21
22 def on_message(self, client, userdata, message):
23     msg = json.loads(message.payload.decode())
24     self.mqtt_queue.put(msg)
25
26 def on_connect(self, client, userdata, flags, rc):
27     print("result code:", rc)
28     self.rc = rc
29
30 # 注意: 在 websocket 端口是 9001, 如使用 Python 代码, 端口需手动改成 1883。
31 hqyj_mqtt_clt = HQYJMqttClient('127.0.0.1', 1883)
32 hqyj_mqtt_clt.client.loop_start()
33 time.sleep(3)
34 if hqyj_mqtt_clt.rc == 0:
35     print('MQTT connected')
36     hqyj_mqtt_clt.client.subscribe('AIOTSIM2APP', qos=0)
37
38 while True:
39     mqtt_data = hqyj_mqtt_clt.mqtt_queue.get()
40
41     if ('flame' in mqtt_data) and ('id' in mqtt_data) and (mqtt_data['id'] == 0):
42         if (float(mqtt_data["flame"])) == 1:
43             print('Find flame ')
44             hqyj_mqtt_clt.client.publish('APP2AIOTSIM', payload=json.dumps({"alarm":True, "id":
45 0}),ensure_ascii=False))
46         if (float(mqtt_data["flame"])) == 0:
47             print('No flame')
48             hqyj_mqtt_clt.client.publish('APP2AIOTSIM', payload=json.dumps({"alarm":False, "id":
49 0}),ensure_ascii=False))

```

在运行虚拟仿真的情况下运行 Python 程序, 可以看到命令行界面会识别到相关的数据, 并将控制命令发送给虚拟仿真系统。



我们也可以利用 Python 里的 tkinter 框架设计界面，来显示和控制相关的传感器。代码如下：

```
1 from tkinter import *
2 import tkinter.messagebox
3 import paho.mqtt.client as mqtt
4 import threading
5 import json
6
7 class MQTTClient:
8     """
9     mqtt
10    """
11    def __init__(self):
12        self.message = ""
13
14    def param_set(self, target_ip, default_port, sub_topic, pub_topic):
15        """
16        初始化
17        :param target_ip: mqtt 服务器 IP
18        :param default_port: mqtt 端口号
19        :param sub_topic: sub 的 topic
20        :param pub_topic: pub 的 topic
21        """
22        self.client = mqtt.Client()
23        self.client.on_connect = self.on_connect
24        self.client.on_message = self.on_message
25        self.client.connect(target_ip, default_port, 5)
26        self.pub_topic = pub_topic
27        self.client.subscribe(sub_topic, qos=0)
28        self.client.loop_start()
29
30    def on_connect(self, client, userdata, flags, rc):
```




```

31         """
32         连接功能
33         :param client:
34         :param userdata:
35         :param flags:
36         :param rc:
37         :return:
38         """
39         tkinter.messagebox.showinfo("MQTT 初始化", "MQTT 连接成功")
40
41     def on_message(self, client, userdata, msg):
42         """
43         接收数据
44         :param client:
45         :param userdata:
46         :param msg: 接收到的信息
47         :return: none
48         """
49         msg = json.loads(str(msg.payload.decode('utf-8')))
50         try:
51             self.message = msg['flame']
52         except Exception as e:
53             pass
54
55     def send_mqtt(self, data):
56         """
57         发送 mqtt
58         :param data: 发送的数据, 字符串
59         :return: none
60         """
61         self.client.publish(self.pub_topic, payload="{}".format(data), qos=0)
62
63     class InterfaceStyle:
64         def __init__(self):
65             self.ip = ""
66             self.end_of_code = True
67             self.lower = 0
68             self.upper = 0
69
70         def on_closing(self):
71             if tkinter.messagebox.askokcancel("退出", "确定要退出吗?"):
72                 self.end_of_code = False
73                 self.root.destroy()
74
75 
```



```

76     def type(self):
77         self.root = Tk()
78         self.root.title("智能火焰报警系统")
79         self.root.geometry("430x320")
80         self.type_text = "智能火焰报警"
81         # 背景设置
82         background_img = PhotoImage(file="image/背景.png")
83         background_label = Label(self.root, image=background_img)
84         background_label.pack()
85
86         frame = Frame(self.root, bd=10, width=270, height=200, bg="ivory")
87         frame.place(x=80, y=42)
88
89         self.var = StringVar(frame, "火焰状态: 无火焰")
90
91         name = Label(frame, text=self.type_text, bg="ivory")
92         name.place(x=10, y=135)
93
94         value = Label(frame, textvariable=self.var, bg="ivory")
95         value.place(x=10, y=165)
96
97         img_alarm_close = PhotoImage(file="image/报警器.png")
98         img_alarm_open = PhotoImage(file="image/报警器开.png")
99         self.alarm_close = Label(frame, image=img_alarm_close, height=100, width=100, bg="i
100 vory")
101         self.alarm_close.place(x=0, y=30, height=100, width=100)
102         self.alarm_open = Label(frame, image=img_alarm_open, height=100, width=100, bg="ivo
103 ry")
104         #self.alarm_open.place(x=0, y=30, height=100, width=100)
105
106         mqtt_broker = Label(frame, text="MQTT Broker IP:", bg="ivory")
107         mqtt_broker.place(x=0, y=0)
108
109         self.mqtt_broker_ip = Entry(frame, width=15, bg="ivory")
110         self.mqtt_broker_ip.place(x=110, y=0)
111         self.mqtt_broker_ip.insert(0, "127.0.0.1")
112         mqtt_subtopic = Label(frame, text="订阅: AIOTSIM2APP", bg="ivory")
113         mqtt_subtopic.place(x=120, y=50)
114         mqtt_pubtopic = Label(frame, text="发布: APP2AIOTSIM", bg="ivory")
115         mqtt_pubtopic.place(x=120, y=70)
116         mqtt_button = Button(frame, text="连接", width=5, command=self.mqtt_init)
117         mqtt_button.place(x=210, y=-5)
118
119         self.root.protocol("WM_DELETE_WINDOW", self.on_closing)
120

```

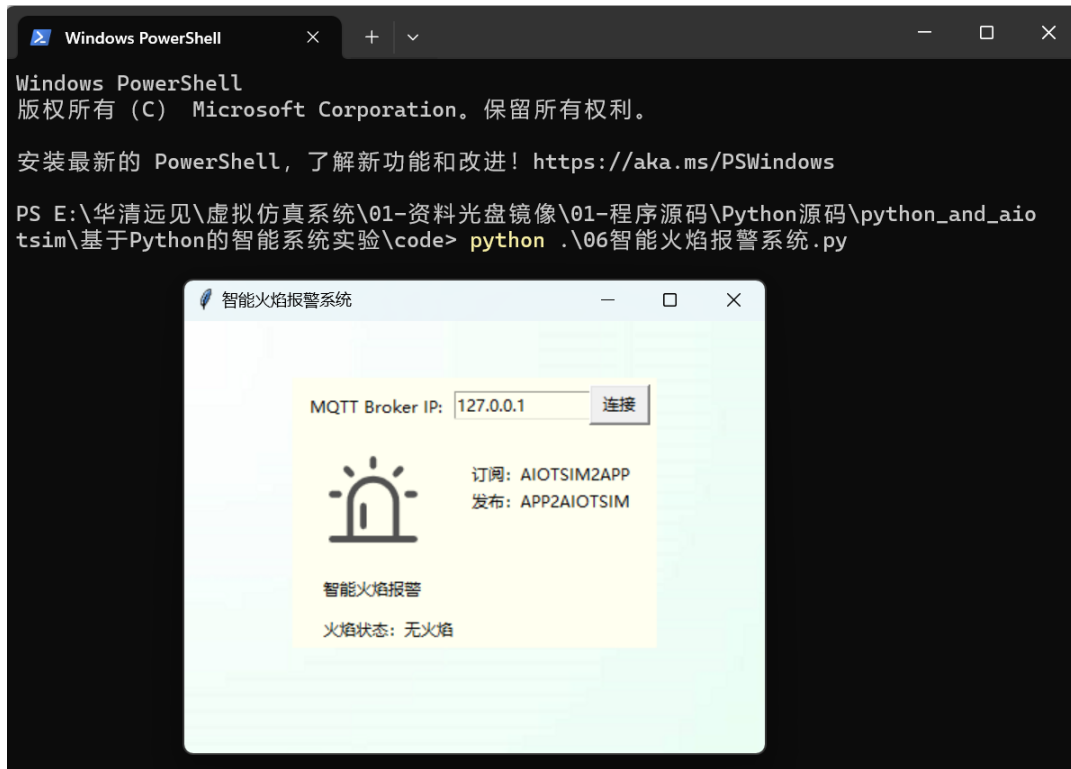


```

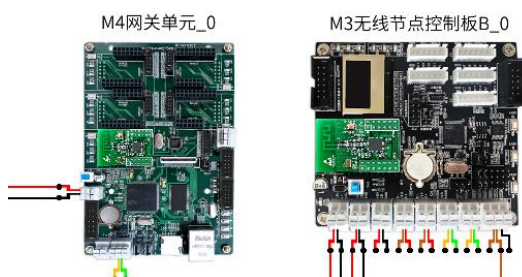
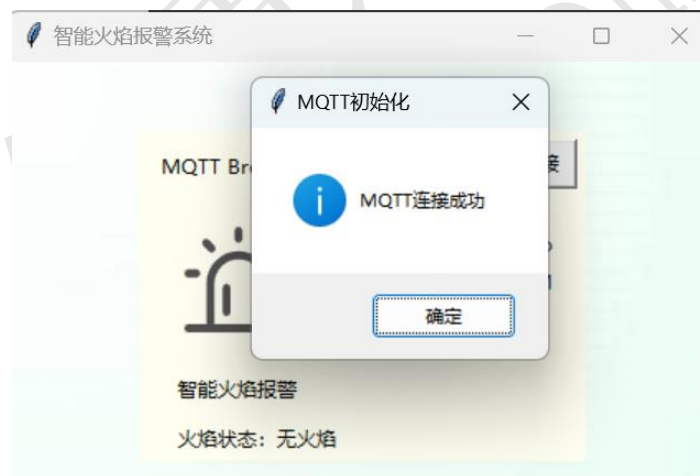
121         self.root.mainloop()
122
123     def mqtt_init(self):
124         self.ip = self.mqtt_broker_ip.get()
125
126     def display_and_control_device(self, status):
127         if status:
128             self.alarm_close.place_forget()
129             self.alarm_open.place(x=10, y=10, height=110, width=110)
130         else:
131             self.alarm_open.place_forget()
132             self.alarm_close.place(x=10, y=10, height=110, width=110)
133
134 if __name__ == '__main__':
135     interface_style = InterfaceStyle()
136     t_interface_style = threading.Thread(target=interface_style.type)
137     t_interface_style.start()
138     mqtt_client = MQTTClient()
139     while interface_style.end_of_code:
140         if interface_style.ip != "":
141             try:
142                 mqtt_client.param_set(interface_style.ip, 1883, "AIOTSIM2APP", "APP2AIOTSIM"
143 )
144             except Exception as e:
145                 tkinter.messagebox.showwarning("MQTT 初始化失败", e)
146                 interface_style.ip = ""
147             if mqtt_client.message != "":
148                 tempval = "火焰状态: {}".format(mqtt_client.message)
149                 interface_style.var.set(tempval)
150                 if int(mqtt_client.message) == 0:
151                     interface_style.display_and_control_device(False)
152                     mqtt_client.send_mqtt("{}alarm:False")
153                 elif int(mqtt_client.message) >= interface_style.upper:
154                     interface_style.display_and_control_device(True)
155                     mqtt_client.send_mqtt("{}alarm:True")
156                     mqtt_client.message = ""

```

该文件为 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验\code 下的 06 智能火焰报警系统.py。在 Python 编译器或命令提示行中运行该 Python 文件，可显示下列窗口。

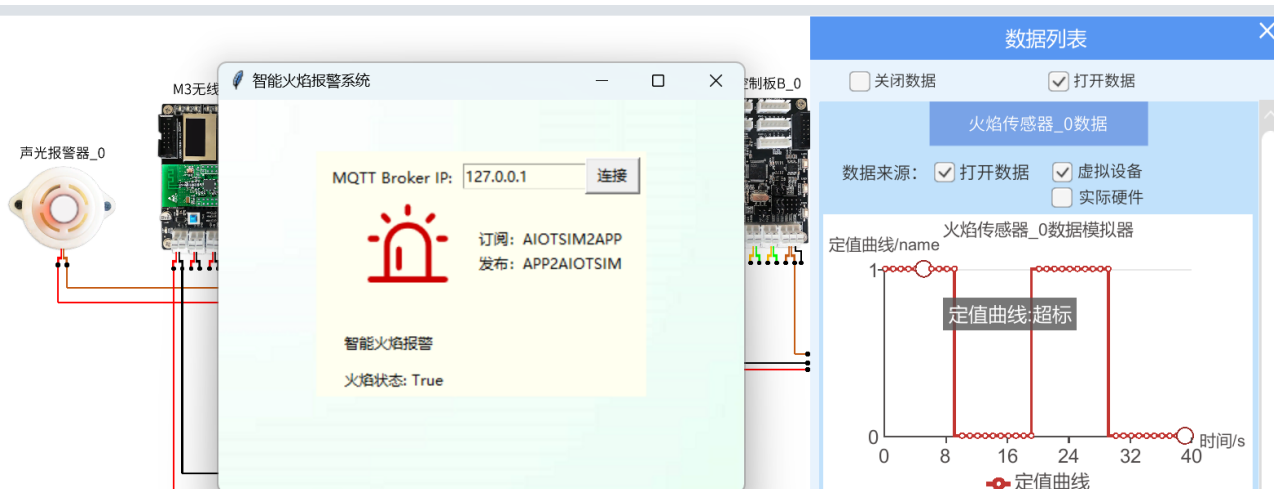


点击连接并显示连接成功后，运行虚拟仿真系统。





可以看到程序会根据传感器的数据自动控制报警器开关。

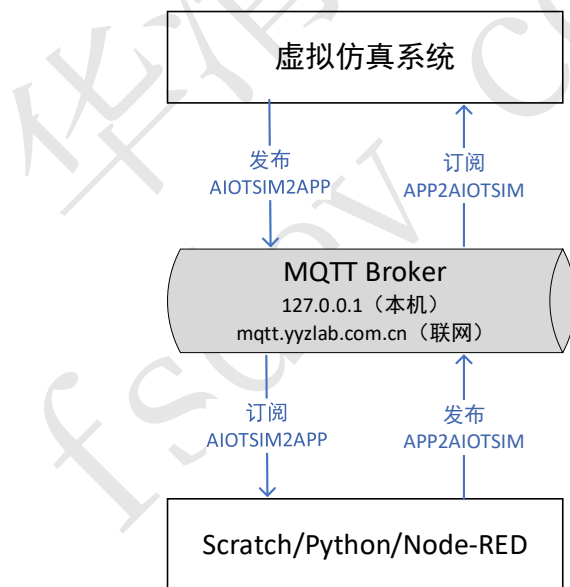


7.2 智能烟雾报警系统

7.2.1 实验目的

通过 Python 获取物联网虚拟仿真系统里的烟雾传感器数据，并根据数据自动控制声光报警器。

7.2.2 实验原理

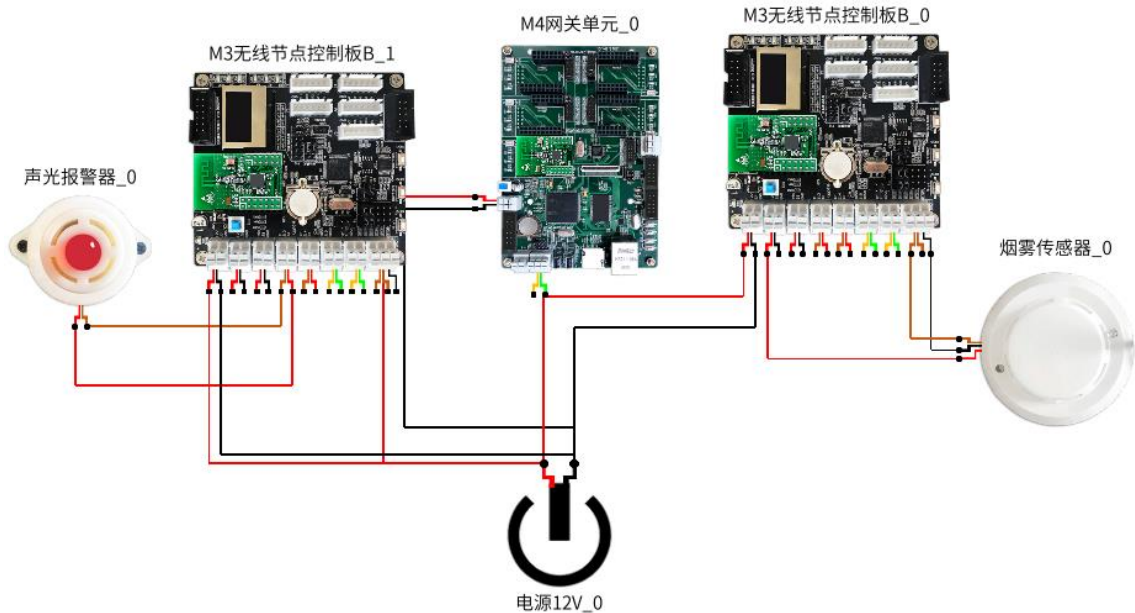


在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

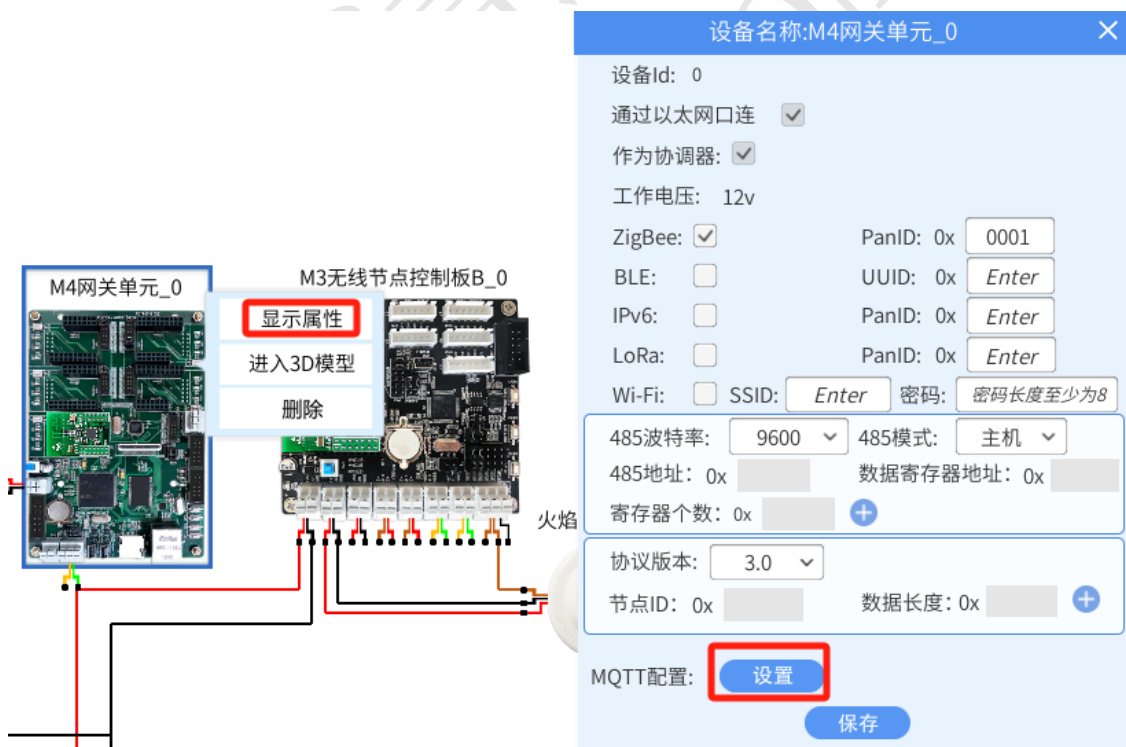


7.2.3 实验步骤

用物联网虚拟仿真系统打开 01- 资料光盘 镜像 \01- 程序源码 \Python 源码 \python_and_aiotsim\基于 Python 的智能系统实验下的对应文件。



左键点击显示属性，在弹出的页面中点击设置。



在弹出的 MQTT 配置页面中，设置 MQTT broker 的 IP 地址、端口、话题等，点击连接和订阅。然后关闭窗口后保存设置。



MQTT配置

MQTT:

IP地址: 127.0.0.1 端口: 1883

ClientID: 请输入或者生成ClientID 生成

用户名: 请输入用户名

密码: 请输入密码

断开

与应用层、云端交互的Topic:

订阅: APP2AIOTSIM 复制

发布: AIOTSIM2APP 复制

生成Topic 取消订阅

与硬件交互的Topic:

点击数据列表，打开数据。

数据列表

☐ 关闭数据 ☒ 打开数据

烟雾传感器_0数据

数据来源: ☒ 打开数据 ☒ 虚拟设备 ☐ 实际硬件

曲线名称: 定值曲线

曲线类型: 定值

数据周期: 20 修改

定值曲线/name 烟雾传感器_0数据模拟器

1

然后点击上方的运行按钮，即可让数据自动产生并向 MQTT 发送数据。

在 Python 中打开 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验\code 下的智能烟雾报警系统_scratch.py。代码如下所示：

```
1# -*- coding:utf-8 -*-
2import paho.mqtt.client as mqtt
3from queue import Queue
4import json
5import base64
6import requests
7import time
8
9class HQYJMQTTClient:
10    def __init__(self, broker_ip: str, broker_port: int):
11        self.mqtt_queue = Queue(255)
```




```

12     self.is_connected = False
13     self.client = mqtt.Client()
14     self.client.on_message = self.on_message
15     self.client.on_connect = self.on_connect
16     self.rc = 100
17     try:
18         self.client.connect(broker_ip, broker_port, 3)
19     except Exception as e:
20         print(e)
21
22     def on_message(self, client, userdata, message):
23         msg = json.loads(message.payload.decode())
24         self.mqtt_queue.put(msg)
25
26     def on_connect(self, client, userdata, flags, rc):
27         print("result code:", rc)
28         self.rc = rc
29
30 # 注意: 在 websocket 端口是 9001, 如使用 Python 代码, 端口需手动改成 1883。
31 hqyj_mqtt_clt = HQYJMqttClient('127.0.0.1', 1883)
32 hqyj_mqtt_clt.client.loop_start()
33 time.sleep(3)
34 if hqyj_mqtt_clt.rc == 0:
35     print('MQTT connected')
36     hqyj_mqtt_clt.client.subscribe('AIOTSIM2APP', qos=0)
37
38     while True:
39         mqtt_data = hqyj_mqtt_clt.mqtt_queue.get()
40
41         if ('smog' in mqtt_data) and ('id' in mqtt_data) and (mqtt_data['id'] == 0):
42             if (float(mqtt_data["smog"])) == 1:
43                 print('Find smoke ')
44                 hqyj_mqtt_clt.client.publish('APP2AIOTSIM', payload=json.dumps({"alarm":True, "id":
45 0}),ensure_ascii=False))
46             if (float(mqtt_data["smog"])) == 0:
47                 print('No smog')
48                 hqyj_mqtt_clt.client.publish('APP2AIOTSIM', payload=json.dumps({"alarm":False, "id":
49 :0}),ensure_ascii=False))

```

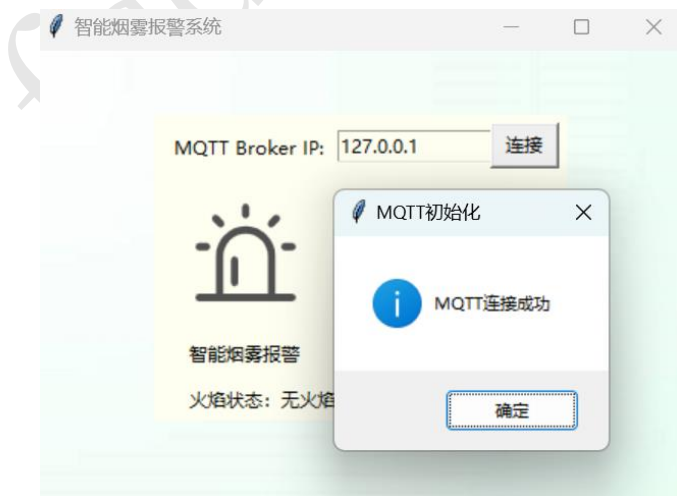
在运行虚拟仿真的情况下运行 Python 程序, 可以看到命令行界面会识别到相关的数据, 并将控制命令发送给虚拟仿真系统。

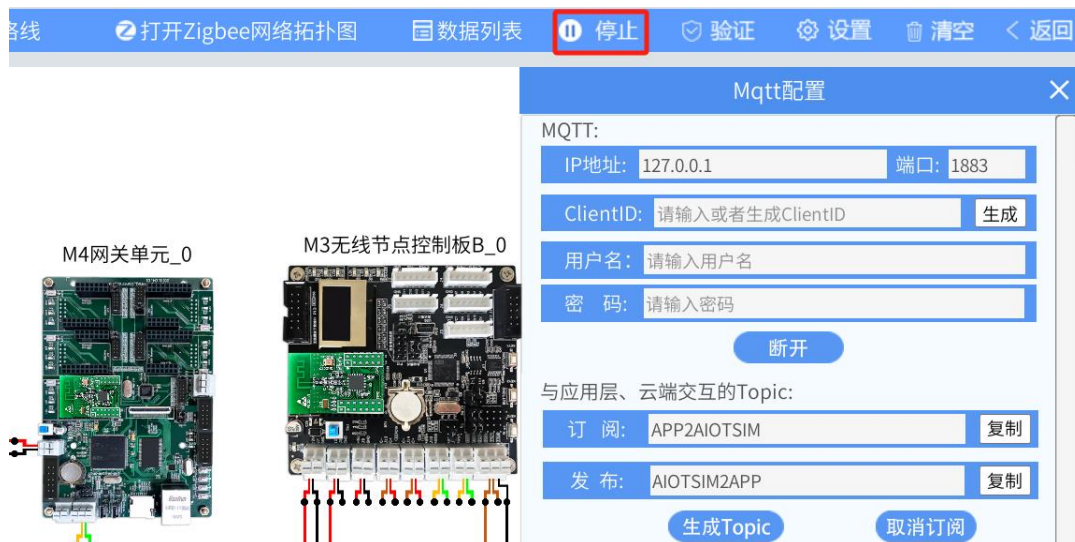


我们也可以利用 Python 里的 tkinter 框架设计界面，来显示和控制相关的传感器。该文件为 01-资料光盘镜像\01-程序源码\Python 源码\python_and aiotsim\基于 Python 的智能系统实验\code 下的 07 智能烟雾报警系统.py。在 Python 编译器或命令提示行中运行该 Python 文件，可显示下列窗口。

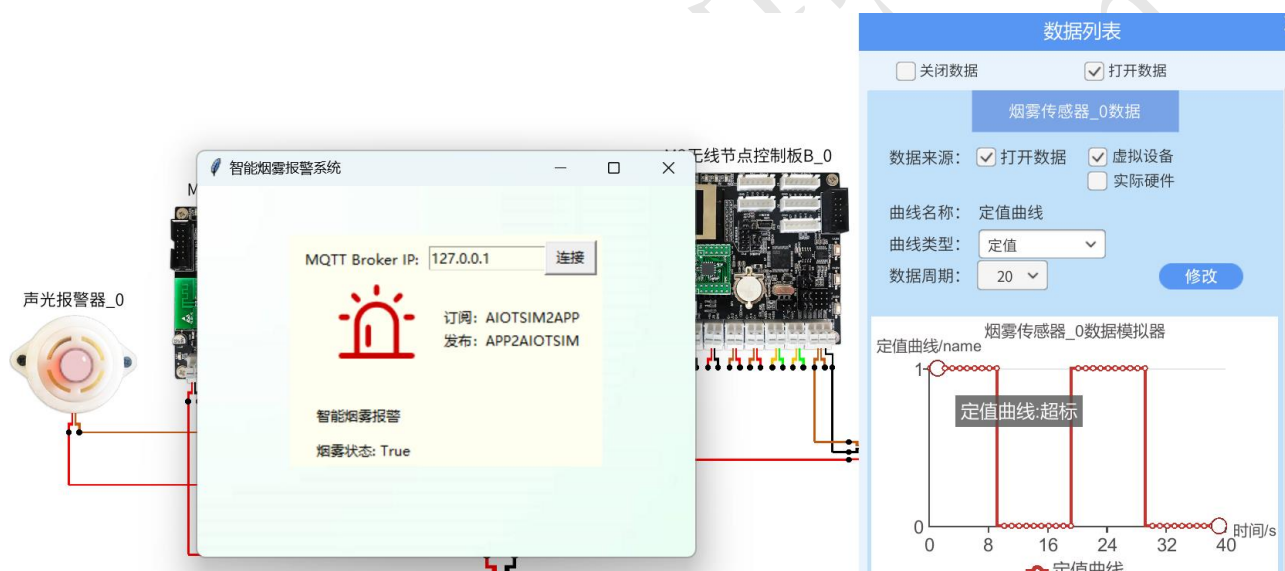


点击连接后，运行虚拟仿真系统。





即可看到程序会根据传感器的状态自动控制执行器。



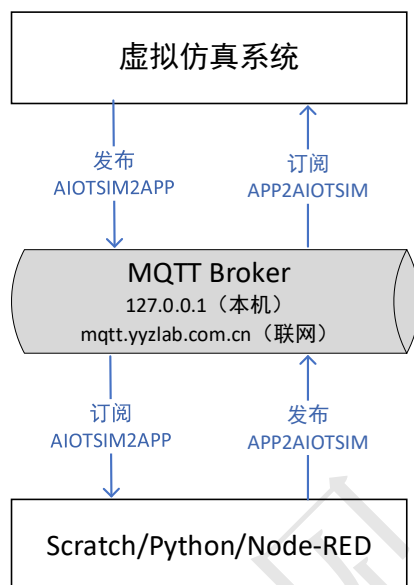
7.3 智能光照控制系统

7.3.1 实验目的

利用 Python 通过 MQTT 获取光照传感器信息，并根据状态进行电动窗帘的控制。当户外的光照传感器识别到光线强度高时，打开窗帘；强度低时，拉上窗帘。



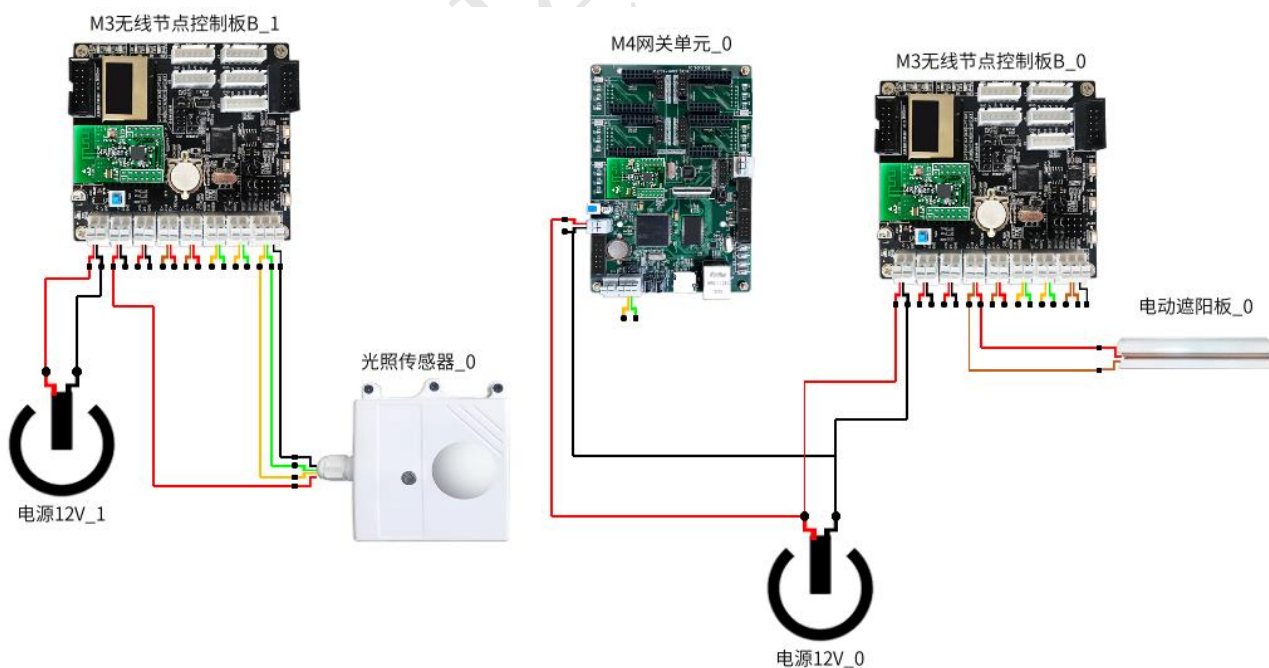
7.3.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

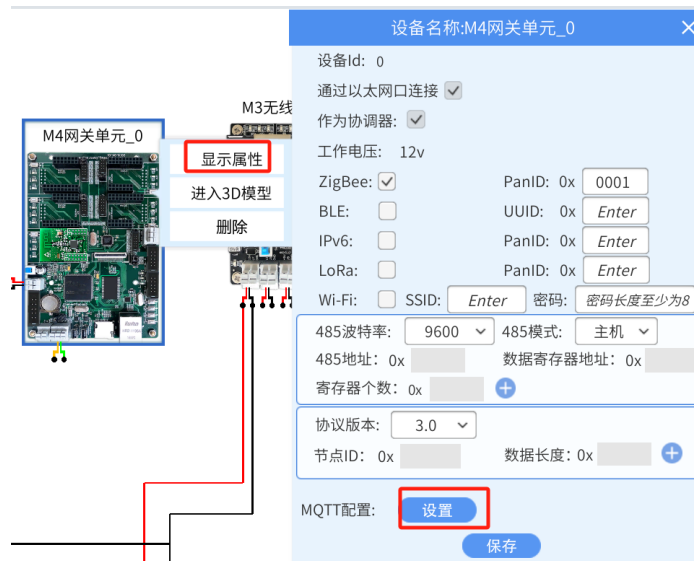
7.3.3 实验步骤

用物联网虚拟仿真系统打开 01- 资料光盘镜像 \01- 程序源码 \Python 源码 \python_and_aiotsim\基于 Python 的智能系统实验下的对应文件。





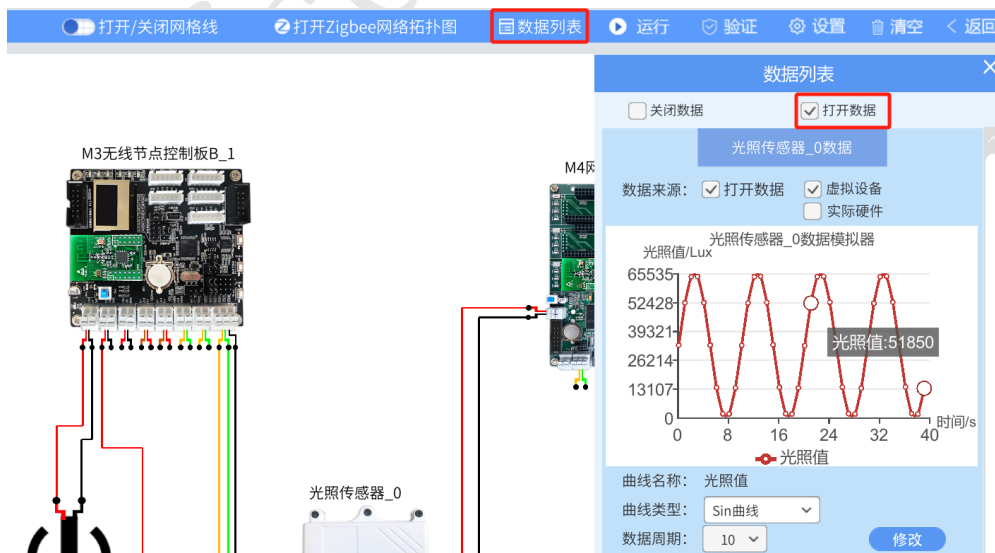
左键点击网关单元，选择属性



连接本地 MQTT Broker，订阅相关话题。



点击数据列表，打开数据。然后点击数据列表旁边的运行按钮。





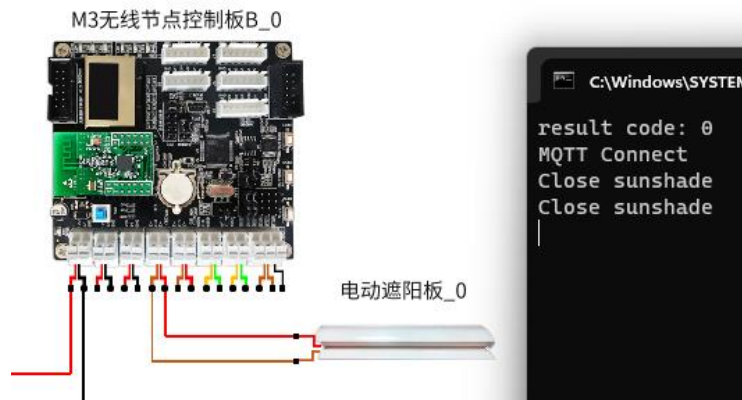
在 Python 中打开 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验\code 下的智能光照控制系统_scratch.py。代码如下所示：

```
# -*- coding:utf-8 -*-
2 import paho.mqtt.client as mqtt
3 from queue import Queue
4 import json
5 import base64
6 import requests
7
8 import time
9
10 class HQYJMqttClient:
11     def __init__(self, broker_ip: str, broker_port: int):
12         self.mqtt_queue = Queue(255)
13         self.is_connected = False
14         self.client = mqtt.Client()
15         self.client.on_message = self.on_message
16         self.client.on_connect = self.on_connect
17         self.rc = 100
18         try:
19             self.client.connect(broker_ip, broker_port, 3)
20         except Exception as e:
21             print(e)
22
23     def on_message(self, client, userdata, message):
24         msg = json.loads(message.payload.decode())
25         self.mqtt_queue.put(msg)
26
27     def on_connect(self, client, userdata, flags, rc):
28         print("result code:", rc)
29         self.rc = rc
30
31 # 注意：在 websocket 端口是 9001，如使用 Python 代码，端口需手动改成 1883。
32 hqyj_mqtt_clt = HQYJMqttClient('127.0.0.1', 1883)
33 hqyj_mqtt_clt.client.loop_start()
34 time.sleep(3)
35 if hqyj_mqtt_clt.rc == 0:
36     print('MQTT Connect')
37     hqyj_mqtt_clt.client.subscribe('AIOTSIM2APP', qos=0)
38
39     while True:
40         mqtt_data = hqyj_mqtt_clt.mqtt_queue.get()
41
```



```
42 if ('light' in mqtt_data) and ('id' in mqtt_data) and (mqtt_data['id'] == 0):
43     if (float(mqtt_data["light"])) > 30000:
44         hqyj_mqtt_clt.client.publish('APP2AIOTSIM', payload=json.dumps({"sunshade":False, "i
45 d":0}),ensure_ascii=False))
46     if (float(mqtt_data["light"])) < 30000:
47         hqyj_mqtt_clt.client.publish('APP2AIOTSIM', payload=json.dumps({"sunshade":True, "id"
48 :0}),ensure_ascii=False))
```

虚拟仿真系统和 Python 都运行后，可以看见如下效果。



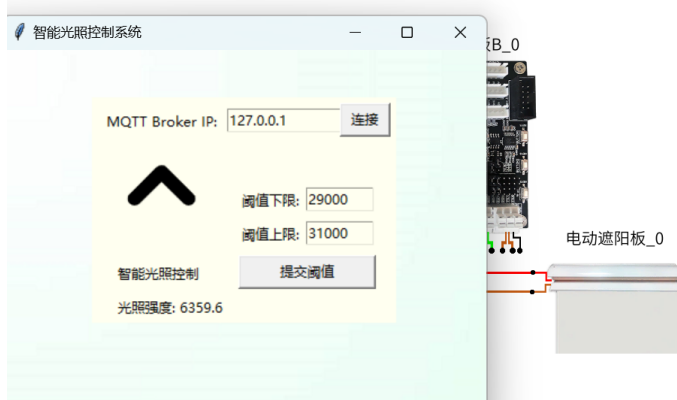
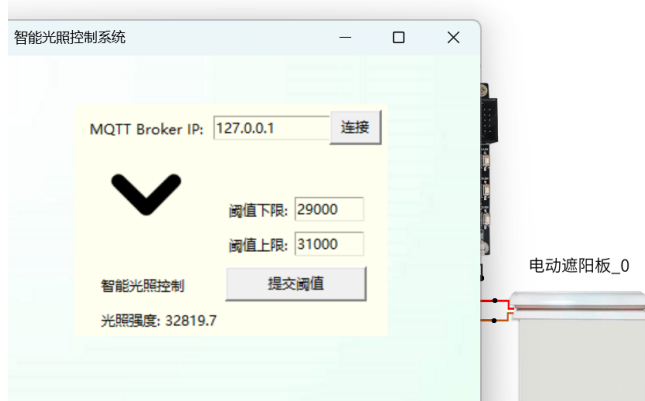
我们也可以利用 Python 里的 tkinter 框架设计界面，来显示和控制相关的传感器。该文件为 01-资料光盘镜像\01-程序源码\Python 源码python_and_aiotsim\基于 Python 的智能系统实验\code 下的智能光照控制系统.py。在 Python 编译器或命令提示行中运行该 Python 文件，可显示下列窗口。



点击连接后，点击提交阈值按钮，然后运行虚拟仿真系统。



即可看到光照过强时，会让遮阳板下放；光照过弱时，会让遮阳板上拉。



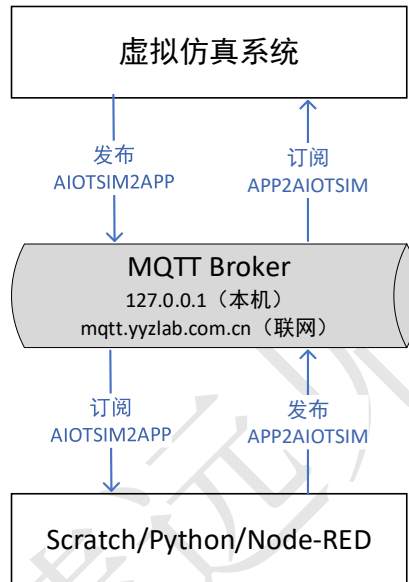


7.4 智能风扇控制系统

7.4.1 实验目的

通过 Python 实现虚拟仿真软件上的智能风扇控制系统。

7.4.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

7.4.3 实验步骤

打开虚拟仿真软件的预设实验中的智能风扇控制系统，设置好 MQTT 并打开数据后，运行系统。

华清远见
HQYJ.COM
物联网虚拟仿真系统

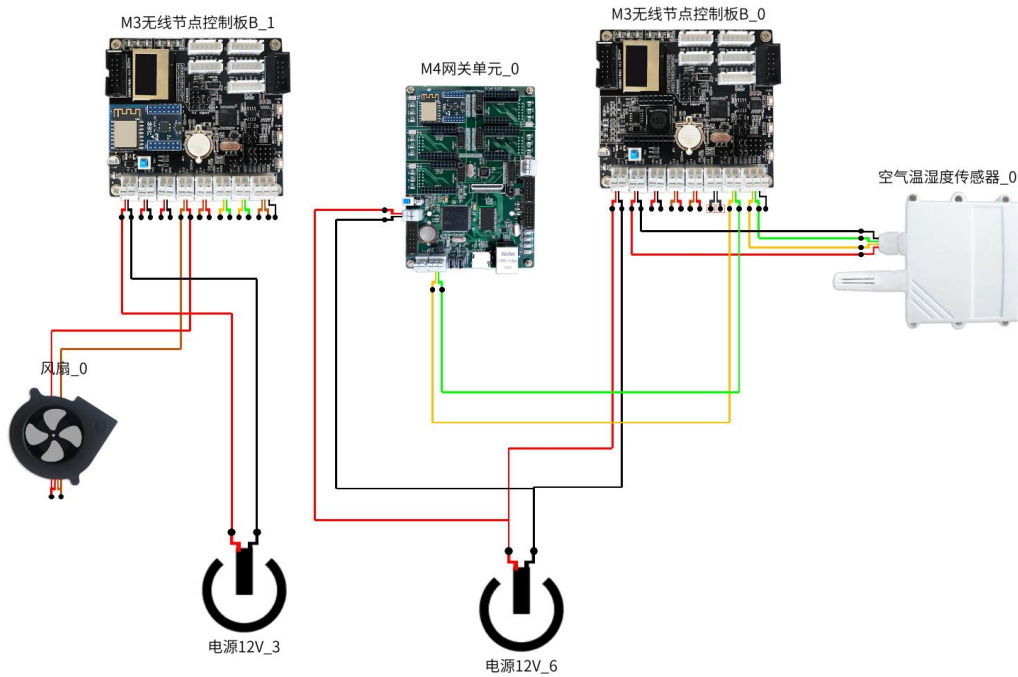
我的实验

预设实验

关于我们

预设实验

实验名称		
1. 智能风扇控制系统 检测室内温度并控制风扇开关，可演示传感节点的虚实结合。 D:/FS_AIOTSIM/HQYJ/AIOTSIM/FS_AIOTSIM_Data/StreamingAssets/PreData/1_智能风扇控制系统.AIOTSIM	打开	移除
2. 智能家居 实现智能家居系统场景，检测家居环境状态，联动智能家居设备，可结合智能家居实训套件实现虚实结合。 D:/FS_AIOTSIM/HQYJ/AIOTSIM/FS_AIOTSIM_Data/StreamingAssets/PreData/2_智能家居.AIOTSIM	打开	移除
3. 智能农业		



Mqtt配置

MQTT:

IP地址: 127.0.0.1 端口: 1883

ClientID: 请输入或者生成ClientID 生成

用户名: 请输入用户名

密码: 请输入密码

断开

与应用层、云端交互的Topic:

订阅: APP2AIOTSIM 复制

发布: AIOTSIM2APP 复制

生成Topic 取消订阅

数据列表

运行 验证 设置 清空 返回

数据列表

☐ 关闭数据 ☒ 打开数据

空气温湿度传感器_0数据

数据来源: ☒ 打开数据 ☒ 虚拟设备 ☐ 实际硬件

曲线名称: 温度

曲线类型: Sin曲线

数据周期: 20

数据范围: -40 - 80

数据范围-40-80

曲线名称: 湿度

曲线类型: Sin曲线

数据周期: 20

修改



运行 Python 代码或者执行智能风扇控制系统数据采集的 exe 文件。文件位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验\dist

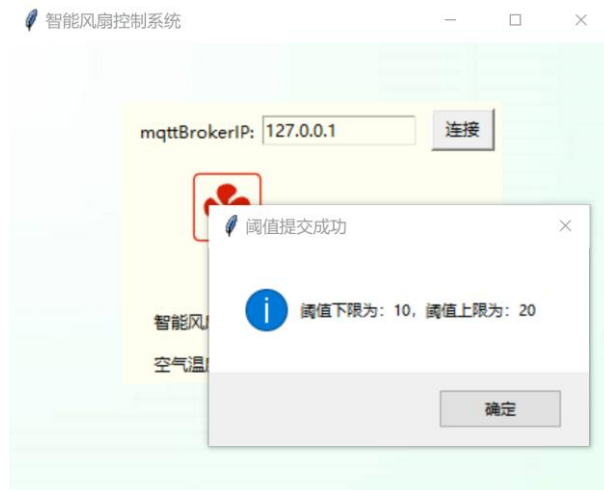


输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



在程序中分别输入阈值下限和上限,点击提交阈值,弹出阈值提交成功,如果不成功,请根据具体弹出警告解决。



之后会发现,如果程序中的”空气温度”高于 20℃,那么风扇标志会变红,如果程序中的”空气温度”低于 10℃,那么风扇标志会变黑



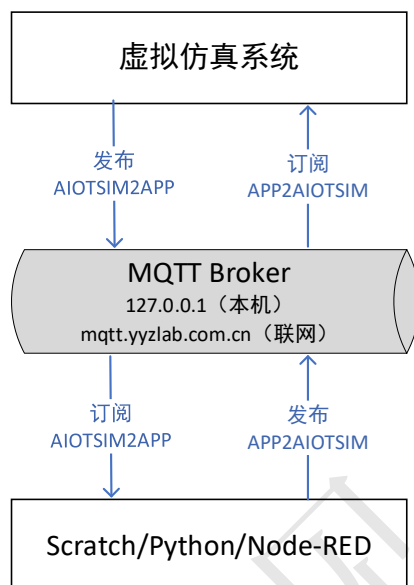
7.5 智能土壤灌溉系统

7.5.1 实验目的

通过 Python 实现虚拟仿真软件上的智能土壤灌溉系统。



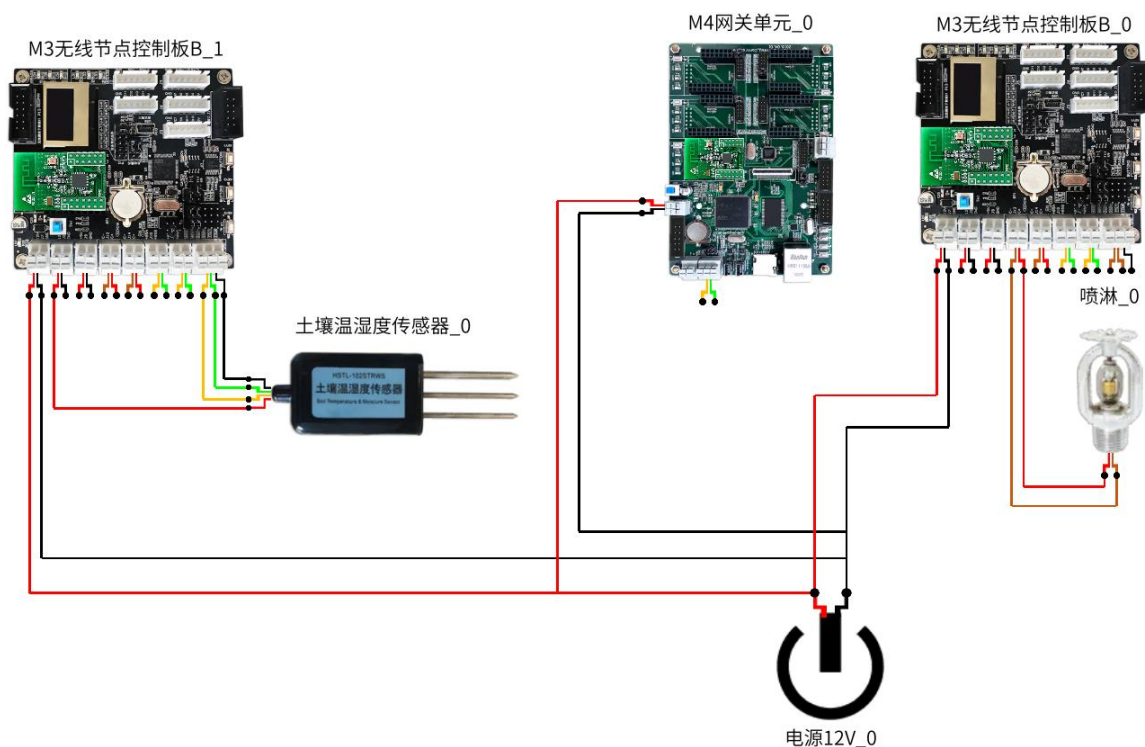
7.5.2 实验原理



在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

7.5.3 实验步骤

打开虚拟仿真软件，导入 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验运行系统目录下的智能土壤灌溉系统.AIOTSIM。打开数据并设置好 MQTT 后点击运行按钮。

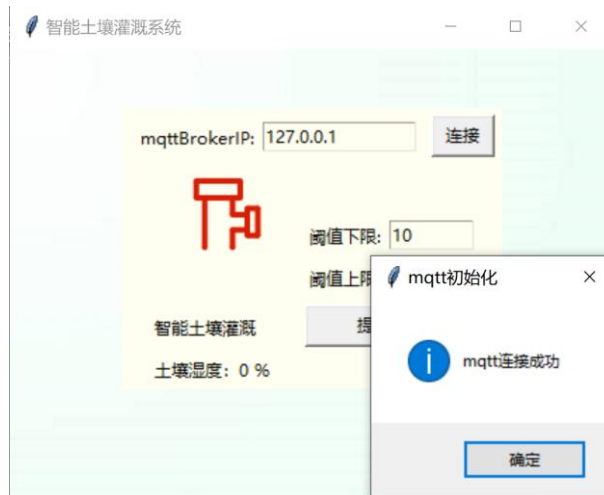


运行 Python 代码或者执行智能土壤灌溉系统数据采集的 exe 文件。文件位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验\dist。其代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验\code。

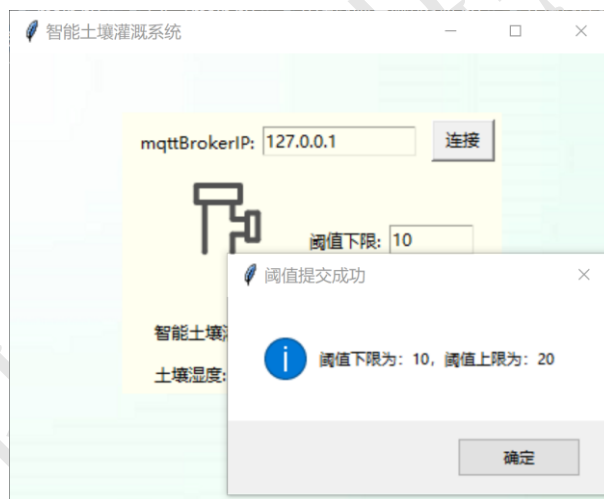


输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



在程序中分别输入阈值下限和上限，点击提交阈值，弹出阈值提交成功，如果不成功，请根据具体弹出警告解决。



之后会发现，如果程序中的“土壤湿度”高于 20℃，那么水泵标志会变黑，如果程序中的“土壤湿度”低于 10℃，那么风扇标志会变红。



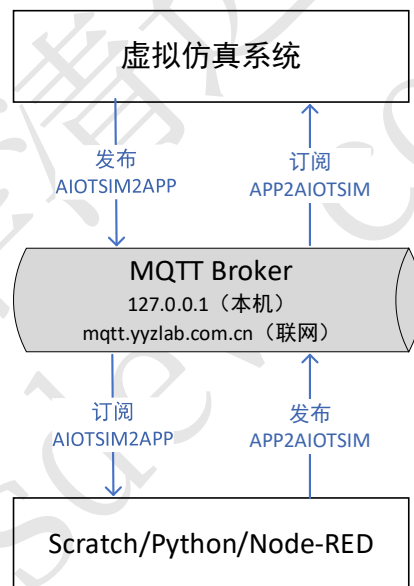


7.6 智能灯光控制系统

7.6.1 实验目的

通过 Python 实现对虚拟仿真软件上的智能灯光控制系统。

7.6.2 实验原理



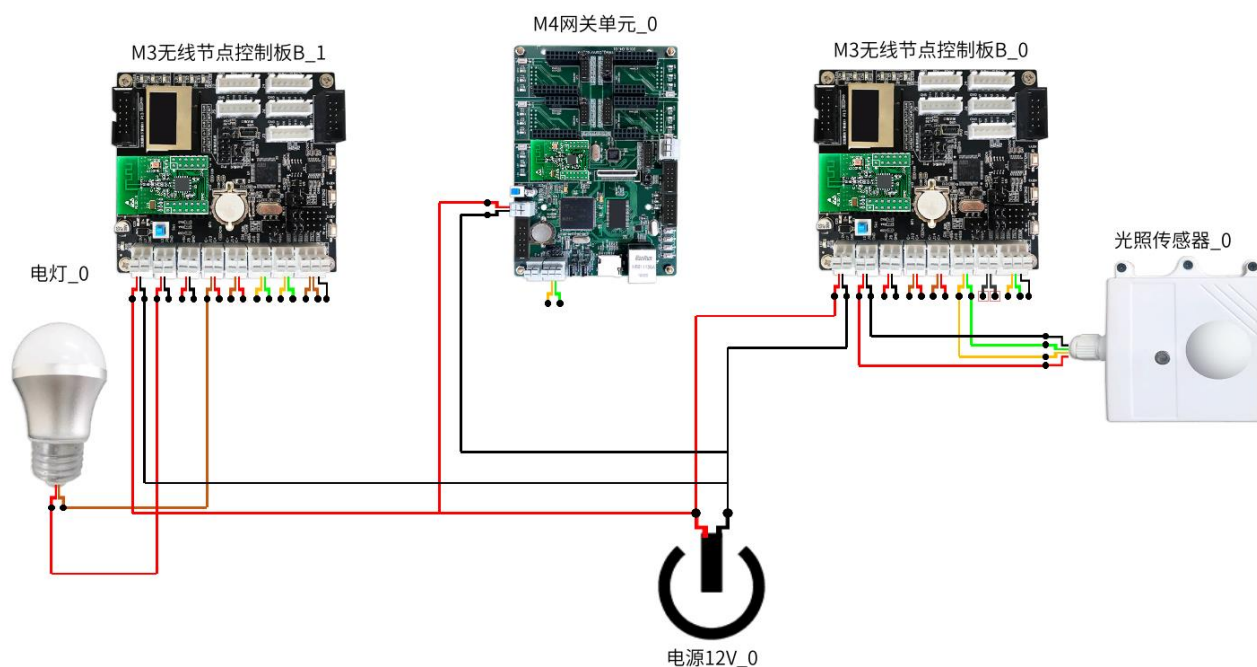
在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

7.6.3 实验步骤

打开虚拟仿真软件，导入 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验运行系统目录下的智能灯光控制系统.AIOTSIM。打开数据并设



置好 MQTT 后点击运行按钮。

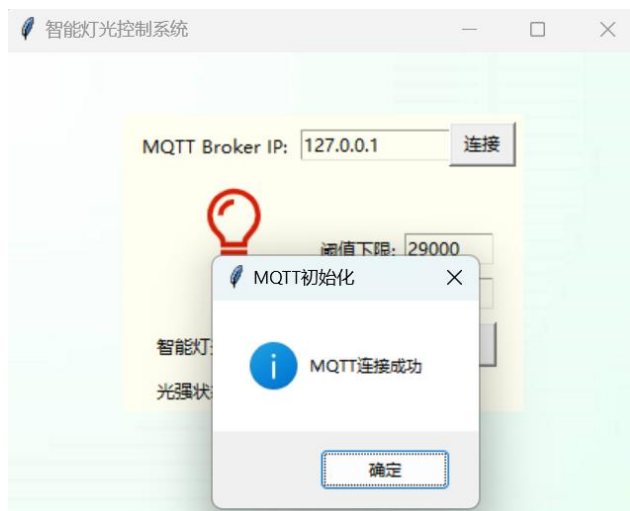


运行 Python 代码或者执行智能灯光控制系统数据采集的 exe 文件。文件位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验\dist。其代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验\code。

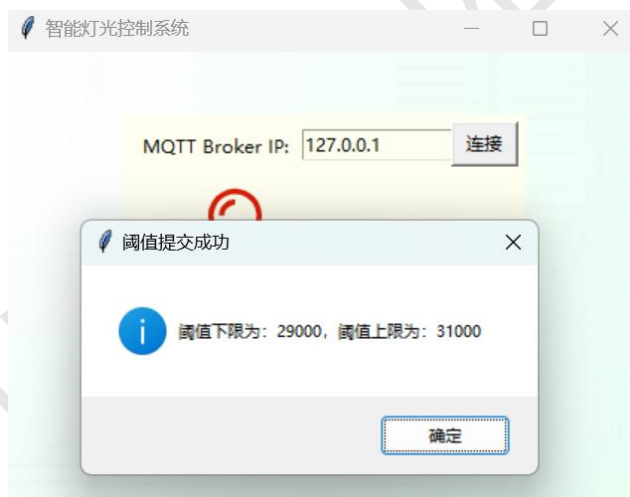


输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



在程序中分别输入阈值下限和上限,点击提交阈值,弹出阈值提交成功,如果不成功,请根据具体弹出警告解决。



之后会发现,如果程序中的”光强状态”高于 3100 Lux,那么灯泡标志会变黑,如果程序中的”光强状态”低于 2900Lux,那么灯泡标志会变红



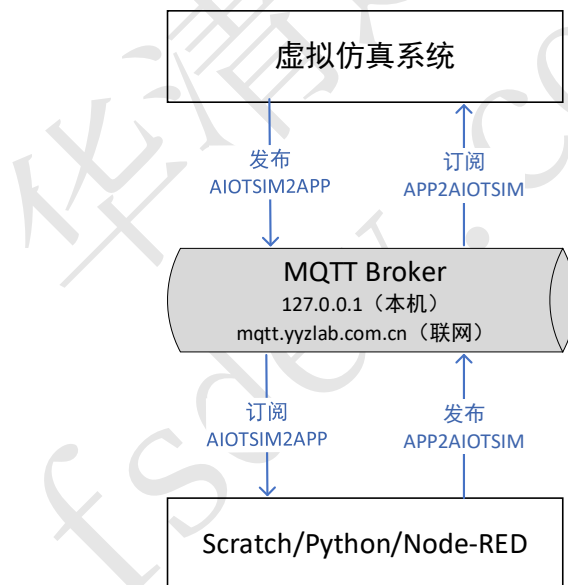


7.7 智能窗帘控制系统

7.7.1 实验目的

通过 Python 实现对虚拟仿真软件上的智能窗帘控制系统。当人体红外传感器检测到有人存在时，关闭窗帘；无人存在时，打开窗帘。

7.7.2 实验原理



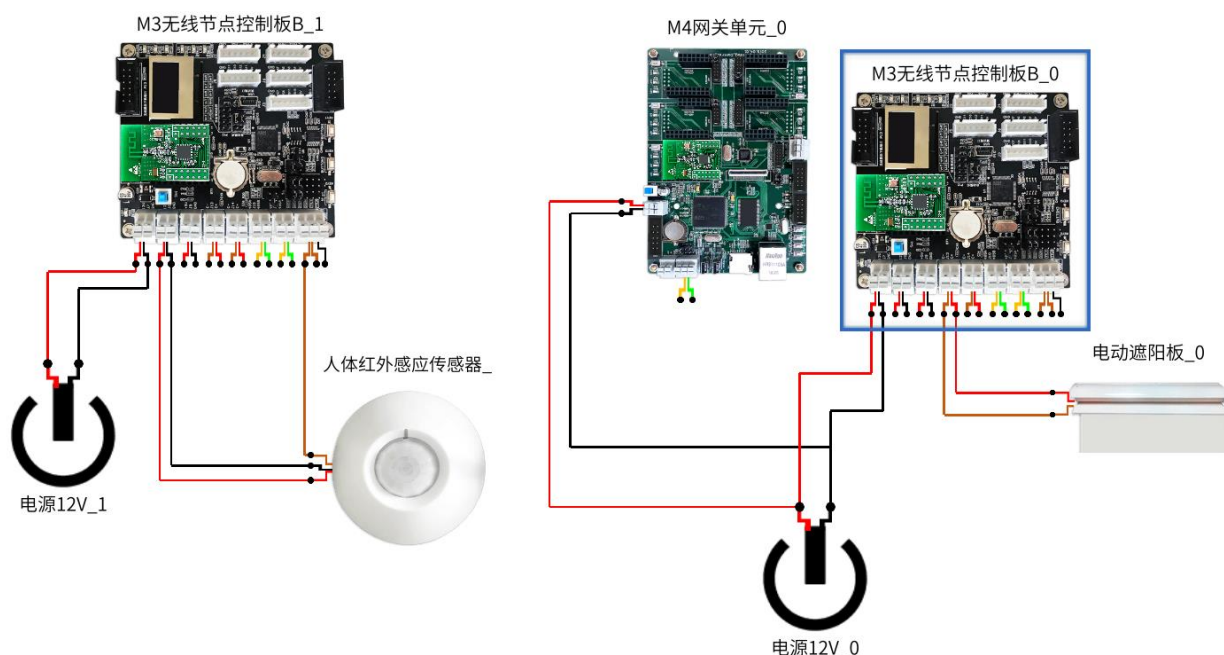
在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

7.7.3 实验步骤

打开虚拟仿真软件，导入 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\



基于 Python 的智能系统实验运行系统目录下的智能窗帘控制系统.AIOTSIM。打开数据并设置好 MQTT 后点击运行按钮。



运行 Python 代码或者执行智能光照控制系统数据采集的 exe 文件。文件位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验\dist。其代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验\code。

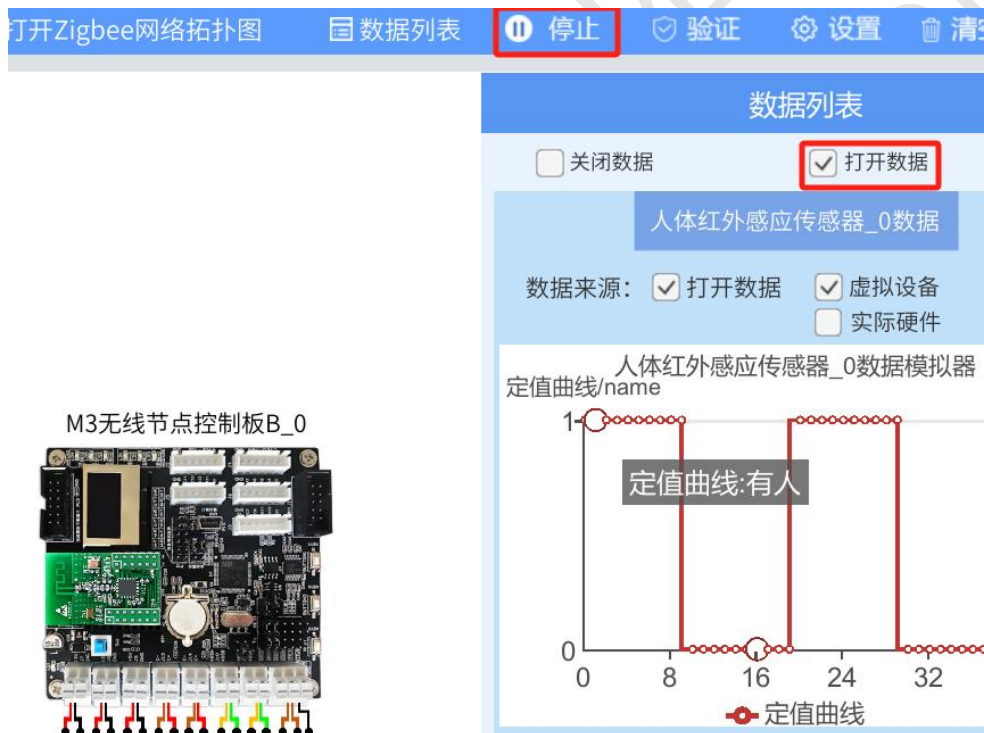


输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

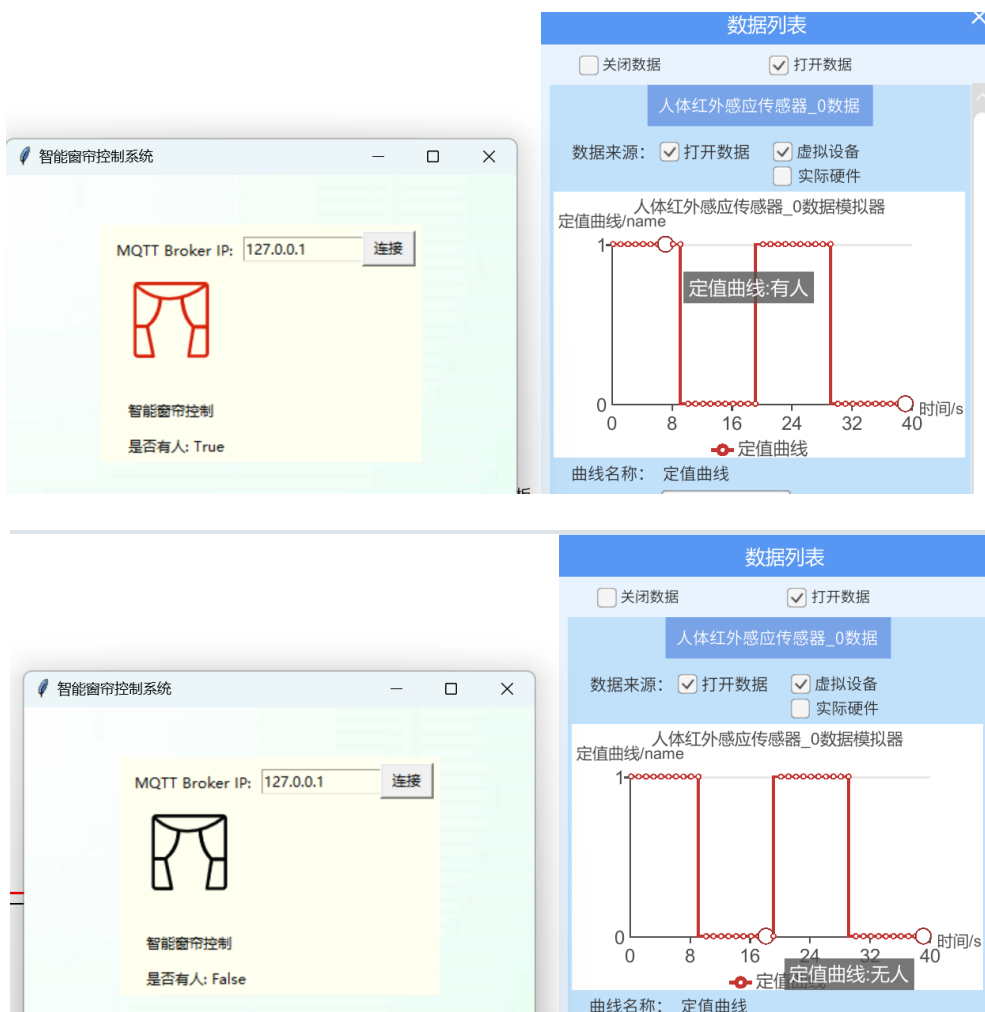
点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。



打开虚拟仿真系统的数据，设置好 MQTT 的话题，连接 MQTT，运行系统。



程序即可接收到虚拟仿真系统中产生的传感器数据，然后控制窗帘。当有人存在时，正向转动窗帘。当无人存在时，反向转动窗帘。

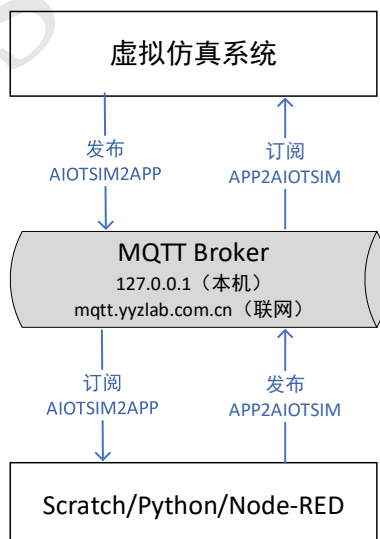


7.8 智能燃气报警系统

7.8.1 实验目的

通过 Python 实现对虚拟仿真软件上的智能燃气报警控制系统。

7.8.2 实验原理

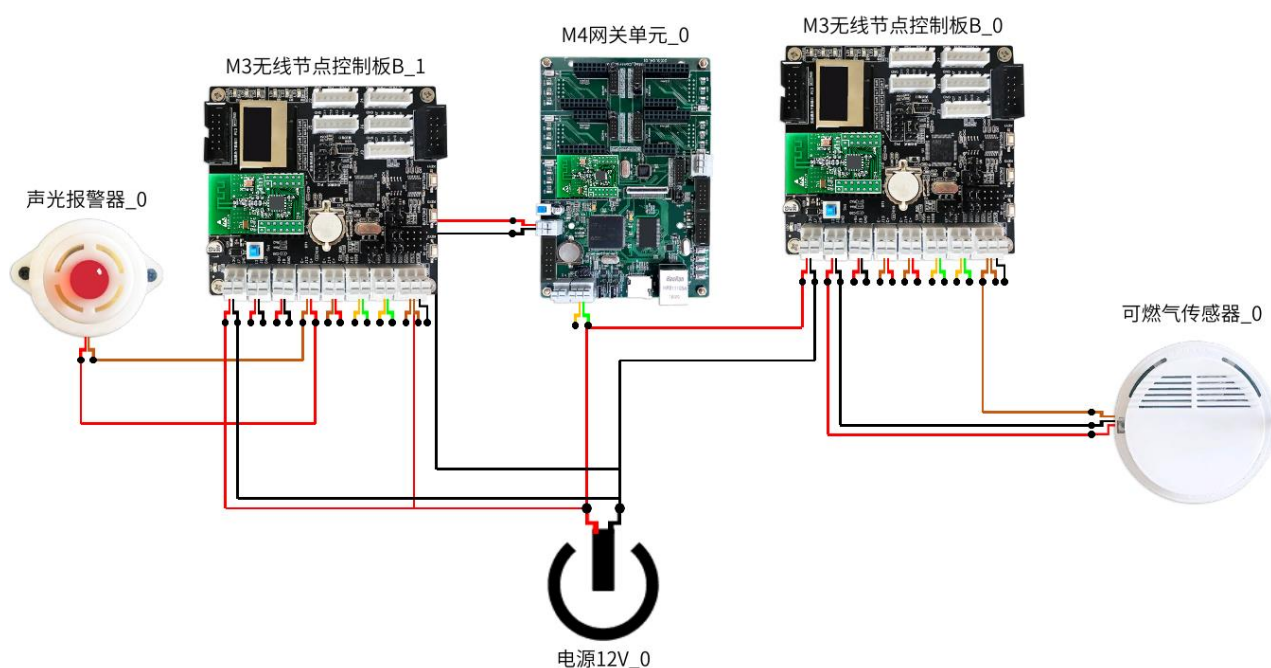




在虚拟仿真系统上连接传感器并运行后，传感器产生的数据会发送到 MQTT broker 的相关的话题上面（此处为 AIOTSIM2APP，即 Scratch 订阅的话题）。Scratch 程序会连接到 MQTT broker 上，订阅相关话题。这样 Scratch 程序即可收到该话题下的数据。Scratch 在接收到数据并进行解析和判断后，会向 APP2AIOTSIM 话题发送指令消息。虚拟仿真系统上的网关会订阅该话题并接收到该话题下的消息，并判断下达了什么指令。

7.8.3 实验步骤

打开虚拟仿真软件，导入 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验运行系统目录下的智能燃气报警控制系统.AIOTSIM。打开数据并设置好 MQTT 后点击运行按钮。

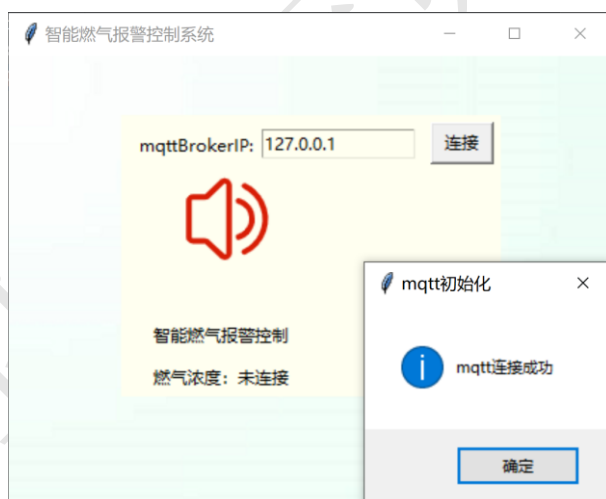


运行 Python 代码或者执行智能燃气报警系统数据采集的 exe 文件。文件位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验\dist。其代码位于 01-资料光盘镜像\01-程序源码\Python 源码\python_and_aiotsim\基于 Python 的智能系统实验\code。

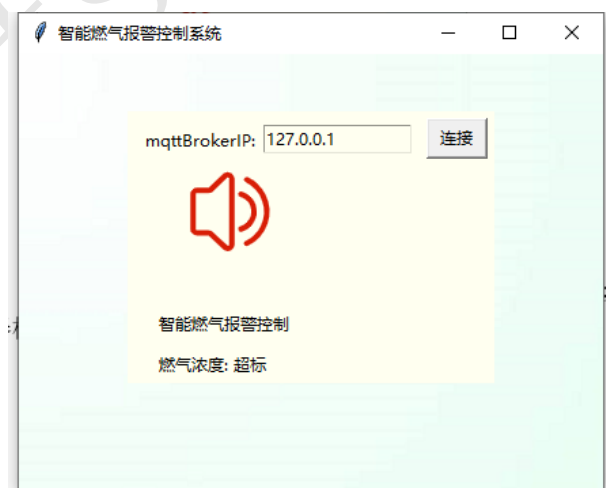


输入 MQTT 服务器的 IP 地址，即运行虚拟的仿真的计算机，如果在运行虚拟仿真和运行 Python 在同一台计算机上，则无需更改。

点击连接，弹出 MQTT 初始化成功的界面，如果不成功，请根据具体弹出警告解决。

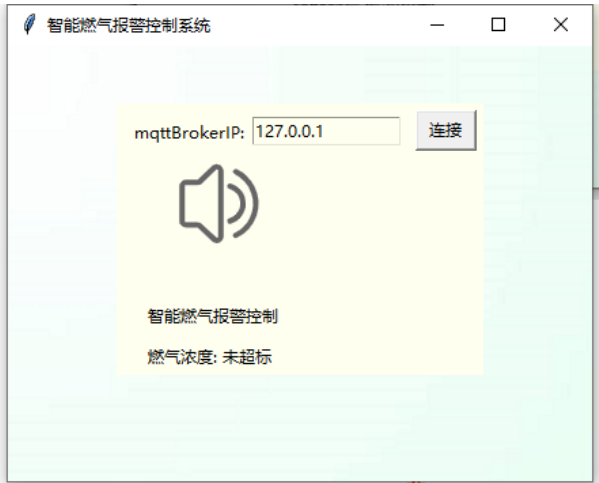
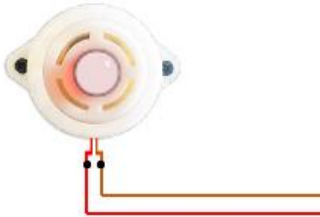


之后会发现,如果“燃气浓度”为超标，则蜂鸣器标志变红；如果“燃气浓度”为未超标,则报警器标志变黑。

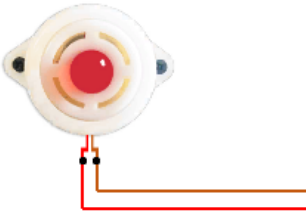




声光报警器_0



声光报警器_0



版本控制

版本号	修改内容	修改人	修改日期
V1.2	重新梳理所有程序。	AHN	2023.12.21